

Universidad de Alcalá

Escuela Politécnica Superior

MÁSTER UNIVERSITARIO EN INGENIERÍA INDUSTRIAL



Trabajo Fin de Máster

Diseño del sistema de posicionamiento y percepción de un robot de competición mediante visión artificial y ROS

ESCUELA POLITECNICA
SUPERIOR

Autor: Carlos José Mamblona Nieto

Tutor: Manuel Ocaña Miguel

2023

UNIVERSIDAD DE ALCALÁ

Escuela Politécnica Superior

Máster en Ingeniería Industrial

Trabajo Fin de Máster

Diseño del sistema de posicionamiento y percepción de un robot de competición mediante visión artificial y ROS

Autor: Carlos José Mamblona Nieto

Tutor/es: Manuel Ocaña Miguel

TRIBUNAL:

Presidente: Rafael Barea Navarro

Vocal 1º: Ignacio Parra Alonso

Vocal 2º: Manuel Ocaña Miguel

FECHA: 18/09/2023

Agradecimientos

“Extraño juego, el único movimiento para ganar es no jugar”

WOPR (Joshua) en Juegos de guerra

En primer lugar, les doy las gracias a todas aquellas personas que me han apoyado y colaborado conmigo durante los 5 años que llevo participando en Eurobot.

Especial agradecimiento a mi familia: Danielle Muñoz Barbero, mis padres y hermana sin los que este trabajo, y por extensión, todos mis estudios no habrían sido posible y también agradecer a mis compañeros del equipo Red Wire por todas las experiencias vividas a lo largo de estos años.

Agradezco también el granito de arena que han dejado mis profesores inspirándome:

Cristina Alén Cordero, Fidel Fiz Criado, Sonia Rodríguez y Manuel Ocaña Miguel.

A todos, muchísimas gracias.

Índice

Índice	Pág. 1
Resumen	Pág. 3
Abstract	Pág. 5
Palabras Clave	Pág. 7
Resumen Extendido	Pág. 9
1. Memoria	Pág. 11
1.1. Introducción	Pág. 12
1.2. Objetivos	Pág. 15
1.3. Materiales y Métodos	Pág. 17
1.4. Desarrollo del Trabajo/Resultados y Discusión	Pág. 24
1.5. Conclusiones	Pág. 56
2. Planos y Diagramas	Pág. 59
2.1. Planos Técnicos del sistema	Pág. 59
2.2. Códigos	Pág. 73
3. Presupuesto	Pág. 83
4. Anexos	Pág. 85
5. Bibliografía	Pág. 141

Índice de figuras

- <i>Figura 0.0.0.1 - Identificación de un conflicto.</i>	<i>Pág. 10</i>
- <i>Figura 1.0.0.1 - Sistema de visión montado sobre el campo.</i>	<i>Pág. 11</i>
- <i>Figura 1.1.0.1 - Reconocimiento de ArUcos durante partido de eurobot 2023</i>	<i>Pág. 13</i>
- <i>Figura 1.1.0.2 - Robot Robonaut basado en ROS - extraído de: https://www.nasa.gov</i>	<i>Pág. 14</i>
- <i>Figura 1.2.0.1 - Esquema del proyecto.</i>	<i>Pág. 16</i>
- <i>Figura 1.3.1.1 - Esquema conceptual del sistema.</i>	<i>Pág. 17</i>
- <i>Figura 1.3.1.2 - Ejemplo de Soporte - Final Francia Eurobot 2023.</i>	<i>Pág. 19</i>
- <i>Figura 1.3.1.3 - Elgato Facecam.</i>	<i>Pág. 20</i>
- <i>Figura 1.3.1.4 - RaspiCam V3 Wide.</i>	<i>Pág. 20</i>
- <i>Figura 1.3.2.1 - Tabla de características del OS cargado (Ubuntu 22.04.3 LTS).</i>	<i>Pág. 22</i>
- <i>Figura 1.4.1.1 - Pasos para la descarga del OS (https://ubuntu-mate.org/download/).</i>	<i>Pág. 25</i>
- <i>Figura 1.4.1.2 - Pasos para cargar el Ubuntu en la MicroSD.</i>	<i>Pág. 25</i>
- <i>Figura 1.4.1.3 - Menú Raspi-config.</i>	<i>Pág. 28</i>
- <i>Figura 1.4.1.4 - Win32 Disk Imager.</i>	<i>Pág. 29</i>
- <i>Figura 1.4.1.5 - Raspberry Py con disipador y ventilador.</i>	<i>Pág. 30</i>
- <i>Figura 1.4.2.1 - Prueba de funcionamiento de ROS2.</i>	<i>Pág. 33</i>
- <i>Figura 1.4.3.1 - Composición de las imágenes de calibración.</i>	<i>Pág. 35</i>
- <i>Figura 1.4.3.2 - Comprobación de la calibración.</i>	<i>Pág. 36</i>
- <i>Figura 1.4.3.3 - Posiciones de las referencias del campo (Normativa Eurobot 2023).</i>	<i>Pág. 37</i>
- <i>Figura 1.4.3.4 – Secciones del programa</i>	<i>Pág. 38</i>
- <i>Figura 1.4.3.5 - Flujograma de procesos.</i>	<i>Pág. 39</i>
- <i>Figura 1.4.3.6 - Ejemplo de funcionamiento de las "áreas de seguridad".</i>	<i>Pág. 40</i>
- <i>Figura 1.4.3.7 - Ejemplo de funcionamiento del programa de visión.</i>	<i>Pág. 41</i>
- <i>Figura 1.4.4.1 – Gráfico "rqt_graph" del proyecto.</i>	<i>Pág. 45</i>
- <i>Figura 1.4.5.1 - Simulación del mapa de competición.</i>	<i>Pág. 46</i>
- <i>Figura 1.4.5.2 - Representación de los límites de visión del sistema.</i>	<i>Pág. 47</i>
- <i>Figura 1.4.5.3 - Plano de dimensiones generales.</i>	<i>Pág. 48</i>
- <i>Figura 1.4.5.4 - Diseño de conexiones por carril de cola de milano.</i>	<i>Pág. 48</i>
- <i>Figura 1.4.6.1 - Marcadores y usos.</i>	<i>Pág. 49</i>
- <i>Figura 1.4.6.2 - Marcadores de pruebas.</i>	<i>Pág. 50</i>
- <i>Figura 1.4.6.3 - Calidad de detección de objetos.</i>	<i>Pág. 51</i>
- <i>Figura 1.4.6.4 - Rango de detección de los ArUcos</i>	<i>Pág. 51</i>
- <i>Figura 1.4.6.5 - Programa de cálculo de puntos dinámico</i>	<i>pág. 52</i>
- <i>Figura 1.4.6.6 - Prueba sobre el robot en campo.</i>	<i>Pág. 53</i>
- <i>Figura 1.4.6.7 - Comprobación de las medidas del sistema.</i>	<i>Pág. 53</i>
- <i>Figura 1.4.6.8 - Errores de detección.</i>	<i>Pág. 53</i>
- <i>Figura 1.4.6.9 - Resultados.</i>	<i>Pág. 54</i>
- <i>Figura 1.4.6.10 - Mapa de resultados.</i>	<i>Pág. 54</i>
- <i>Figura 1.4.6.11 - Pruebas con focos de luz directa.</i>	<i>Pág. 55</i>

Resumen

Se presenta un proyecto de visión artificial escrito en Python3 con OpenCV, enfocado en la robótica deportiva para la competición “Eurobot” a través del equipo “Red Wire”, por el cual se desarrolla un sistema de detección y estimación de la “pose” de códigos ArUco vinculados a cada uno de los robots y los artículos de interés del campo, implementando esta funcionalidad en el entorno de ROS2 para que los resultados obtenidos sean accesibles por el resto de sistemas del equipo para trabajar de manera conjunta en la realización de las tareas demandadas por la normativa de la competición vigente.

Abstract

An artificial vision project related to robot sports for the "Eurobot" competition through "Red Wire" team is created in Python3 with OpenCV. The objective of this project is the creation of a detection system and a "pose" estimation of ArUco markers attached to each of the robots and important items in the competition field. This functionality will be implemented in ROS2 environment so that the results obtained are accessible to the rest of the existing systems in order to perform the demanded tasks written in the competition rules.

Palabras clave

Robótica

Robótica móvil

Visión por computador

OpenCV

Visión artificial

Python

ROS2 (Robot Operating System)

Raspberry Pi 4

Procesamiento de imágenes

Estimación de posición

ArUco

Reconocimiento de patrones

Nodo ROS

Comunicación entre dispositivos

Algoritmos de visión por computador

Cámara

Resumen extendido

Este proyecto se desarrolla dentro del ámbito de la robótica deportiva con aplicación directa en la competición internacional “Eurobot” y a través del equipo “Red Wire”, fundado en 2021 a partir de algunos miembros del antiguo equipo de robótica de la Universidad de Alcalá (el “Team UAH Robotics”) del que el autor del presente proyecto es miembro desde 2019.

Continuando con esta carrera de superación que representa el perfeccionamiento de los robots móviles que el equipo debe presentar en cada edición para poder conseguir victorias, se plantea la investigación y desarrollo de nuevas herramientas que estos dispositivos puedan utilizar para realizar las tareas propuestas por la competición de manera más eficiente y efectiva, por lo que, el propósito de este proyecto fin de máster será precisamente desarrollar una de las líneas de investigación que el equipo lleva barajando desde su primera participación: los sistemas de visión artificial. De tal forma que se pueda aportar información útil a los robots, ya sea la detección de objetos y enemigos o incluso la posición del propio robot, así como otras muchas aplicaciones, siempre teniendo en cuenta las necesidades reales del equipo tanto principales como transversales.

Existen diversas formas de detección por medio de visión por ordenador que la normativa de la competición contempla y por ello existen puntos estratégicamente colocados para la utilización de estas herramientas de detección, como balizas activas que pueden implementar diversas tecnologías (ultrasonidos, infrarrojos, laser...), además de, por supuesto, dentro del propio robot diseñado. Sin embargo, el campo cuenta con una posición que destaca principalmente por su altura y es la denominada “TORRE”, un punto excepcionalmente elevado un metro por encima de la superficie del campo y que permite, según la normativa, instalar un dispositivo de altura máxima de 600mm dando como resultado un punto a casi 1.6 m con respecto a la pista de juego. Este lugar está expresamente pensado para la colocación de algún sistema de visión que permita tener control de todo el campo y de esta forma entregar información a los robots que pueda servirles en tareas como el posicionamiento, la identificación de los enemigos o la estimación de puntos.

En este proyecto se implementará el uso de marcadores ArUco “Augmented Reality University of Cordoba”, unos patrones fácilmente detectables en una imagen por medio de la detección de los contornos que forman el marcador. Estos Aruco llevan asociados un identificador y son asimétricos, por lo que es posible reconocer su orientación y, de esta forma, la orientación del objeto sobre el que va impreso.

En la competición, todos los robots deben ir identificados con un marcador que se coloca en un soporte a una altura fija en el techo de cada robot, que muestra el equipo al que pertenece este, por lo que es posible y será la principal tarea de este proyecto fin de master, estimar la posición de los robots aliados con gran precisión para contribuir en el sistema de posicionamiento interno del robot, y también detectar la posición de los robots enemigos para identificar posibles situaciones de conflicto.

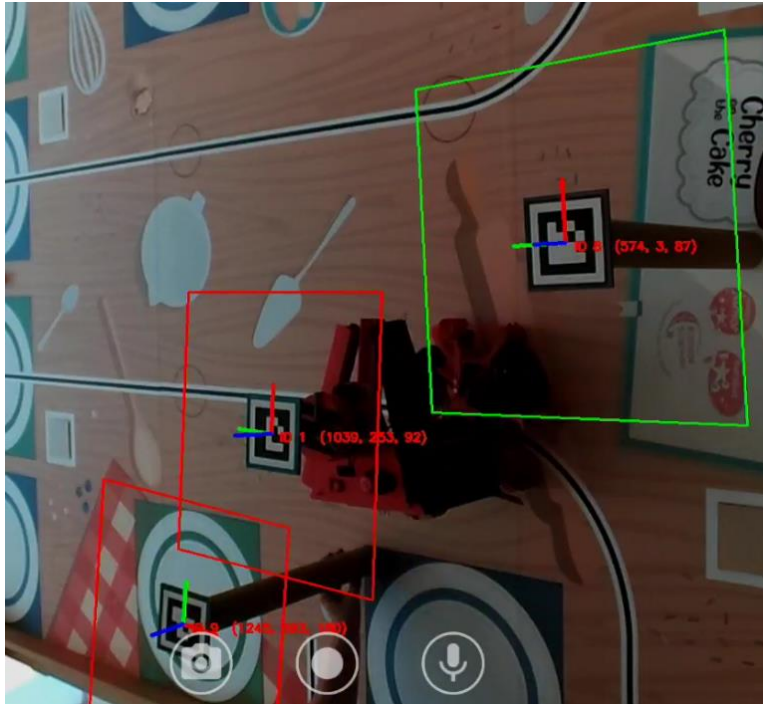


Figura 0.0.0.1 - Identificación de un conflicto

Una de las contribuciones más significativas de este proyecto para el equipo será la implementación de este sistema de visión en el entorno de ROS2, lo cual permitirá trabajar con el sistema de visión de manera independiente al resto del proyecto, finalmente concluyendo en la generación de un nodo de ROS dentro de un paquete creado para albergar las funciones de visión artificial que se desarrollen y que en este caso es capaz de publicar la información resultante en un topic accesible para el resto de sistemas del equipo a traves de un nodo remitente el cual permitirá manejar estos datos de manera sencilla y eficaz.

1. Memoria

En este capítulo se va a exponer el desarrollo del proyecto, desde el planteamiento inicial donde se comenzó a realizar el estudio de los componentes necesarios para llevar a cabo el trabajo, hasta la obtención y el análisis de los resultados finales pasando por el desarrollo de los programas y metodologías para el correcto funcionamiento del sistema diseñado.



Figura 1.0.0.1 - Sistema de visión montado sobre el campo

1.1. Introducción

El proyecto realizado se construye en el entorno de Eurobot, una competición de robótica móvil que surge en Francia en el año 1998 como “Coupe de France de Robotique”, un desafío amateur internacional que reúne a aficionados del mundo de la robótica, donde los participantes deben presentar soluciones para la realización de una serie de tareas de manera automática especificada previamente en una normativa que varía anualmente.

La copa se compone de varios partidos de clasificación donde cada encuentro se desarrolla entre dos equipos que, hasta la fecha, pueden competir con hasta dos robots cada uno, siendo el objetivo del encuentro la obtención de puntos por medio de la realización de diversas tareas en un tiempo máximo de 90 segundos.

Las tareas que han de realizar los robots varían cada año según la normativa, pero dada la morfología que presentan los robots, las tareas principales no suelen variar en esencia, ya que en todas las ediciones es necesaria la navegación autónoma junto con la evitación de obstáculos para la manipulación de objetos de diversos pesos y formas, además de transportarlos de un lugar a otro del campo de tres metros de largo y dos de ancho.

También se plantean tareas secundarias para conseguir puntos extra, las cuales suelen implicar la elaboración de estrategias concretas, el desarrollo de herramientas específicas o la detección de ciertos objetos, aumentando así la dificultad del evento.

Todos estos puntos se realizan teniendo en cuenta el reducido tiempo disponible mientras el equipo contendiente trabaja en el mismo espacio y en gran medida, con elementos compartidos.

De esta forma es preciso que los robots se muevan de manera autónoma por el campo reconociendo los objetivos disponibles para adquirir puntos y evitando la colisión con otros robots o incluso con los límites del campo, acción que está prohibida y cuya infracción conlleva sanciones para el equipo.

El objetivo final es la acumulación de puntos partido tras partido para poder superar las rondas eliminatorias. Primeramente, se llevan a cabo partidos a nivel nacional, donde se clasifican los 3 equipos con mayor puntuación de cada país. Posteriormente, se reúnen todos ellos a competir en la final celebrada en Francia. Cabe destacar que hay dos modalidades: junior y senior. La principal diferencia que se puede apreciar entre las dos modalidades es la automatización robótica de los seniors y el control remoto de los juniors.

- **Los Sistemas de visión artificial y la detección de objetos**

Estos sistemas se han visto beneficiados gracias a los avances en el campo de la inteligencia artificial, revolucionando numerosas áreas de la robótica. Se ha dotado a los sistemas automatizados la capacidad de reconocer el entorno por medio del análisis de imágenes. Cuya podrían ser: en el entorno de la conducción autónoma, el reconocimiento de obstáculos, detección de líneas de carril, señales, etc. En este punto, los marcadores ArUco han surgido como una herramienta a tener en cuenta.

Estos códigos bidimensionales se utilizan en numerosas áreas para la identificación y posicionamiento de objetos por medio de visión artificial a través de cámaras gracias a la gran sencillez de esta solución que se ha convertido en un elemento fundamental para ciertas aplicaciones

Algunos ejemplos de uso de estos marcadores podrían ser el control de drones ya que estos van equipados con cámaras que les permiten identificar un código ArUco y utilizarlo para orientarse durante un aterrizaje, o la automatización de centros logísticos donde robots elevadores son capaces de circular por las instalaciones y posicionarse correctamente por medio de estos marcadores colocados en el suelo.

Esta tecnología, en un entorno de características controladas, mejora la precisión y eficiencia de los sistemas de reconocimientos de objetos y abre nuevas posibilidades en términos de innovación por lo que este proyecto busca beneficiarse de estos elementos para obtener resultados que sirvan de interés a la hora de coordinar los robots de la competición Eurobot. Ya que, como se verá más adelante, existen numerosos códigos ArUco colocados en objetos y zonas de interés e incluso sobre los robots a una altura específica permitiendo así estudiar sus movimientos para tareas de depuración como se muestra en la siguiente figura.

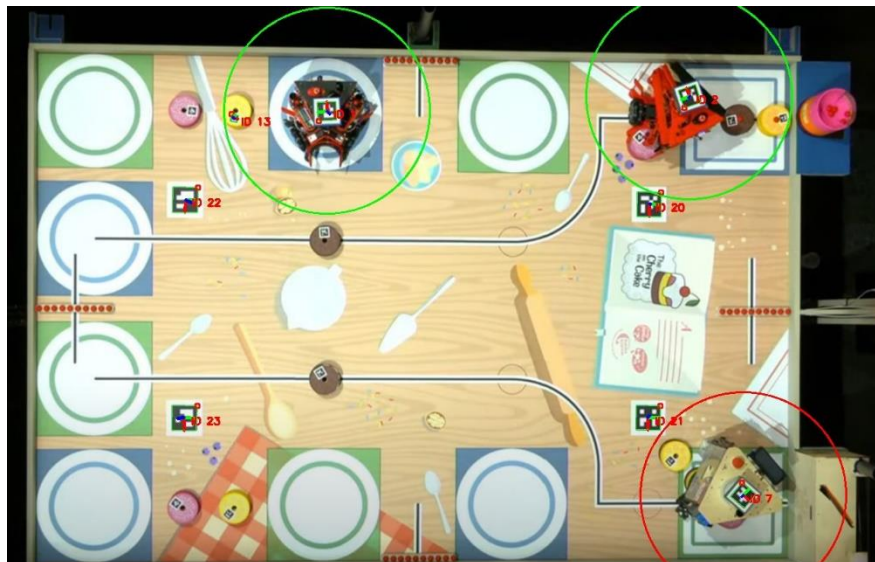


Figura 1.1.0.1 Reconocimiento de ArUcos durante partido de eurobot 2023

- **ROS (Robot Operating System)**

Esta plataforma de software abierto ha supuesto un gran avance dentro del área de la robótica, y más concretamente, en la tarea de control de robots ya que una de las principales características de este software es desgajar este problema en tareas más sencillas por medio de la incorporación de “nodos”, unas unidades de procesamiento independientes que se asignan a cada una de estas tareas más sencillas, de tal forma que, gracias a la arquitectura de ROS, estos nodos pueden testearse de manera independiente y organizarse en paquetes junto con todas sus dependencias y bibliotecas para permitir su uso en otros proyectos.

Este sistema da como resultado un entorno flexible y modular desde el que poder desarrollar soluciones complejas por medio de subtareas más sencillas y la comunicación entre estos nodos.

Actualmente, ROS se puede encontrar en prácticamente todas las áreas donde la robotica tenga aplicación, desde medicina como es el caso del robot quirúrgico “Da Vinci” hasta en numerosos proyectos de la NASA (National Aeronautics and Space Administration) como el rover “Mars Curiosity” donde el sistema de ROS se utiliza para tareas de navegación y control autónomo o el humanoide “Robonaut 2” de la estación espacial internacional.



Figura 1.1.0.2 - Robot Robonaut basado en ROS - extraído de: <https://www.nasa.gov>

Este proyecto se centrará en la versión dos de este entorno de desarrollo que fue presentada en 2017 y trae numerosas mejoras como la utilización de DDS (Data distribution Service) en lugar de TCP/IP (Transmission Control Protocol/Internet Protocol) como la versión uno. El DDS está enfocado en el desarrollo de aplicaciones en tiempo real y se utiliza en sistemas con gran cantidad de nodos que necesitan intercambiar datos de manera eficiente.

Además, en esta versión se realiza un cambio significativo en el tipo de modelo jerárquico que se utilizaba. Si bien en ROS1 existía un “master” que centralizaba todo el entorno y administraba la distribución y conexión entre nodos, en la versión dos de ROS ya no hay un “master” y cada nodo es capaz de administrar su entrada en la red.

1.2. Objetivos

Este proyecto plantea los siguientes objetivos principales:

1. El diseño y desarrollo de un sistema de posicionamiento del robot que ha sido diseñado por uno de los equipos que ha participado y participará en futuras entregas de Eurobot, como se explicaba en la introducción del trabajo, haciendo uso de un sistema de visión por computador, detectando los marcadores ArUco repartidos por el campo, los robots y los objetos de interés.

El sistema estará compuesto por un sensor, unos sistemas de procesamiento de las imágenes y transmisión de los datos resultantes al robot y las estructuras de soporte que integren toda la solución en un único prototipo robusto y fácil de utilizar.

Para ello, ha de estudiarse cuál es la mejor alternativa de hardware y software para adquirir además de hacer uso de programas de diseño 3D que permitan estudiar las implicaciones del proyecto a nivel de entorno (el campo de competición como mundo cerrado y conocido) y diseñar las piezas para el montaje de la estructura de soporte mencionada. Se tendrán en cuenta las condiciones de su obtención por medio de fabricación aditiva de tal forma que el resultado final permita al sensor contar con una vista completa de toda el área de juego desde el soporte destinado a su uso, “la torre” y obtener de esta manera información de relevancia para el equipo. El sistema debe contar con un cierto grado de adaptabilidad de manera que permita su utilización en futuras ediciones de la competición.

2. La implementación de este sistema dentro del entorno de ROS 2 (Robot Operating System), haciendo que los datos resultantes del mecanismo de visión puedan pertenecer a un nodo publicador al que el robot pueda subscribirse para extraer en cada momento la información correspondiente a su posición, o incluso, la de los robots del equipo contrario y la de ciertos puntos de interés en el campo; cruzando la información obtenida con el resto de sensores con los que cuenta el robot (odometría, LIDAR, ultrasonidos...) de tal forma que el nodo planificador pueda desarrollar la ruta a seguir.

Para conseguir estos objetivos principales y de forma lateral, se conseguirán los siguientes sub-objetivos:

1. El diseño de una herramienta de recuento de puntos dinámica de forma que, al final de la competición, pueda darse de manera más precisa una estimación de la puntuación realizada por el equipo obteniéndose una cierta bonificación sobre esta puntuación en función de la exactitud de este sistema.

2. Detectar la posición relativa de determinados puntos y tareas con respecto al robot en el campo para que éste pueda recurrir a esos datos para determinar qué actividad resulta más beneficiosa en cada momento

3. Conocer la orientación de los robots del equipo contrario en cada momento. Existe una gran cantidad de robots que utilizan sistemas de tracción diferencial, por lo que cuentan con una orientación determinada que puede ser útil conocer de cara a guardar las distancias con el mismo en cada momento evitándose así encuentros que puedan desembocar en un choque que provocaría la descalificación de ambos robots.

Como objetivo final se conseguirá una estructura como se muestra en la siguiente figura, donde la solución hardware (sensor y microcontrolador) transmita a través del software de ROS, los datos relevantes del entorno una vez hayan sido procesados por medio de un programa escrito en Python.

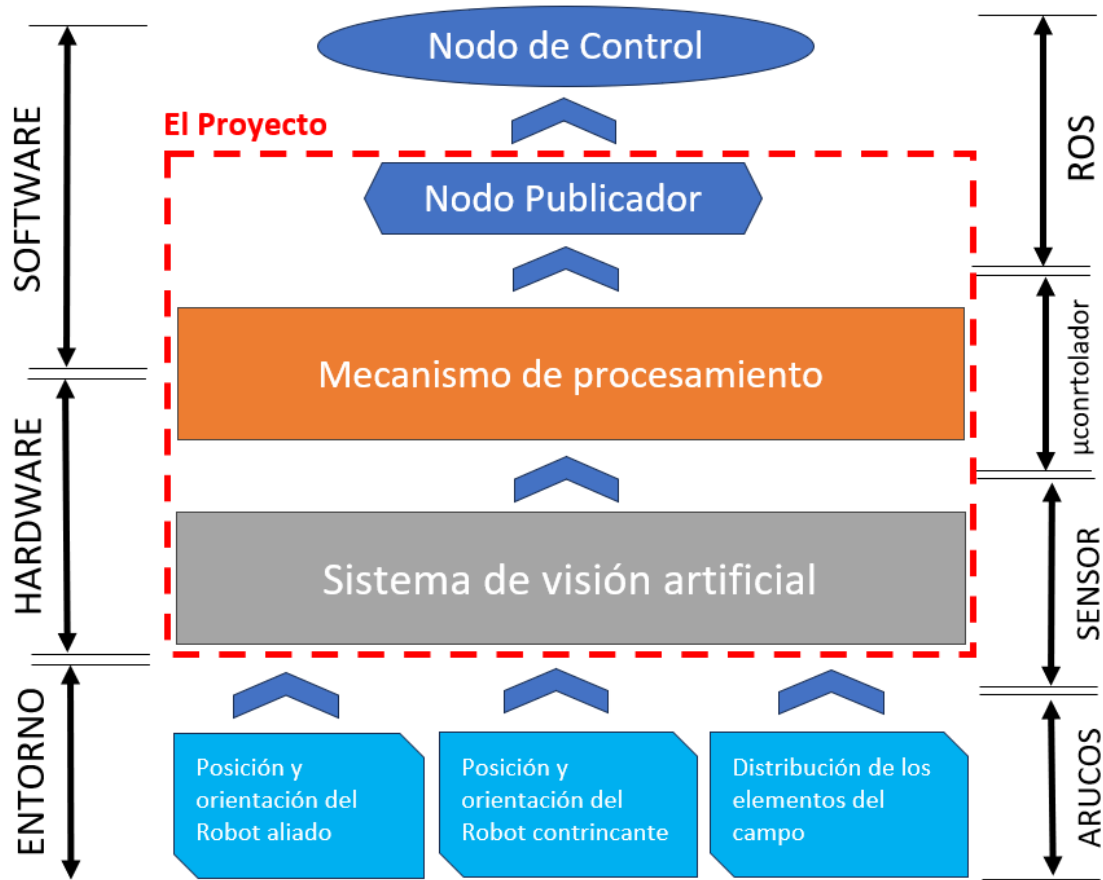


Figura 1.2.0.1 - Esquema del proyecto

1.3. Material y Métodos

En este apartado, se recogen todas las necesidades estudiadas y la selección de los componentes detallada.

Para llevar a cabo el proyecto fue preciso realizar primero un estudio de las características del sistema recogidas en la normativa de la competición y, a través del estudio de la geometría del campo con programas de diseño 3D, comprobar que soluciones se adaptaban mejor al entorno del proyecto para poder así seleccionar y adquirir los materiales y herramientas más apropiados e instalar el software necesario para el desarrollo de la solución.

1.3.1 Selección de Materiales

El sistema a desarrollar debe ser capaz de, mediante un sistema de visión, transmitir la posición de un robot móvil en un determinado espacio, detectando un marcador ArUco que obligatoriamente se encontrará en la parte superior de éste. Por lo que el sistema estará formado por los siguientes componentes:

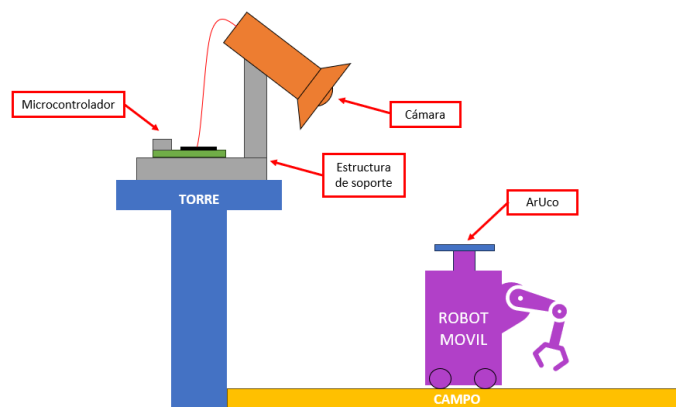


Figura 1.3.1.1 - Esquema conceptual del sistema

- **El sensor:**

Como sensor principal se usará una cámara con unas determinadas características que permitan obtener un plano de imagen lo suficientemente amplio como para abarcar todo el mapa de competición desde la "TORRE" donde debe colocarse el sistema y una resolución de imagen que permita distinguir correctamente los marcadores, incluso si estos se encuentran en el punto más alejado del mapa, por lo que serán importantes las siguientes características:

1. Ángulo de visión FOV "Field of view", el cual está directamente relacionado con la cantidad de espacio del entorno que se captura en la imagen a una distancia determinada. Dado que la cámara ha de encontrarse a una distancia concreta del plano del campo y no es posible alejarla tanto como podría ser necesario, es importante que el ángulo de visión de la cámara sea lo bastante amplio como para capturar todo el campo minimizando las zonas ciegas donde se perdería la referencia con los objetos a detectar. No obstante, un mayor ángulo de visión genera fenómenos de distorsión de la imagen como el estiramiento de las equinas, aumentos de la perspectiva o curvaturas

de la imagen en la periferia, lo que complica mucho más el reconocimiento de la posición real del objeto en la imagen.

A esta característica hay que añadirle la relación de aspecto que va a determinar las dimensiones de la imagen a una determinada distancia juntos con el ángulo de visión ya explicado.

2. Resolución de vídeo, que como se indicó anteriormente, debe permitir una correcta visualización de los marcadores para su detección, pero teniendo en cuenta que a mayor resolución mayor es la cantidad de recursos necesarios para procesar la imagen, y el dispositivo designado a este procesamiento debe ser lo bastante potente como para llevar a cabo esta tarea sin que se ralentice la función de todo el sistema. Por ello, la resolución de vídeo debe encontrarse entre valores de media y alta resolución desde VGA (640x480) hasta Full HD (1920x1080).

Otras características secundarias del sensor son:

1. Conectividad y compatibilidad con el dispositivo de procesamiento. Una característica muy importante dado que existen diferentes maneras de comunicación entre el procesador y el sensor, las cuales pueden ser más generales como USB o más específicas, pero rápidas como la conexión CSI "Camera Serial Interface".
2. Montaje de la cámara, así como sus dimensiones y peso, que va a determinar la manera en la que se diseñen los soportes de esta.

- **El Procesador:**

Para el procesamiento de las imágenes se decide utilizar una Raspberry Py 4B, una placa de desarrollo que permite ejecutar programas basados en Python junto con la librería de OpenCV y lanzar servidores de ROS sobre los que se publicará la información requerida.

Dado que se desea llevar a cabo el procesamiento de imágenes en tiempo real de manera efectiva y precisa, se requerirá de un dispositivo con gran capacidad de procesamiento y memoria RAM. Esto es esencial, ya que el proceso de analizar y tomar decisiones basadas en imágenes en tiempo real, como la detección de objetos, el seguimiento de movimiento o la visión por computador, demandan una gran capacidad de procesamiento para garantizar resultados precisos y una respuesta inmediata ante cambios en la escena, por lo que muchos dispositivos cuentan con elementos específicos para el procesamiento de imagen, pero en el caso de esta aplicación puede que no sea posible contar con estas características, por lo que el dispositivo, al contar con una CPU potente y una cantidad elevada de memoria RAM permitirá realizar estas tareas de manera fluida y sin retrasos, siendo fundamental dado que los marcadores se encuentran montados sobre los robots móviles que se desplazan por el campo y a veces es necesaria dar una respuesta rápida para evitar colisiones.

Además, es importante que este procesador no supere unas determinadas dimensiones ni pesos que marca la normativa de la competición para su implementación. No solo es importante la cantidad de procesamiento del ordenador que se quiera utilizar, sino además de si físicamente es apto para la competición ya que en este caso tampoco es posible contar con todas las posibilidades de alimentación o refrigeración que ciertos dispositivos necesitan. Más concretamente, el dispositivo permite la alimentación con una corriente continua proveniente de una batería, ya sea utilizando reguladores de tensión o de forma directa a esta, pero el uso de corriente alterna está prohibido.

- **El soporte del sistema:**

La competición establece unas normas muy estrictas con respecto a los límites de espacio, peso y seguridad de las soluciones que se llevan al evento, por lo que no se cuenta siempre con todo el espacio que se desea y en este caso todo el mecanismo debe poderse montar en el soporte o “TORRE” que el campo presenta para esta concreta aplicación. Si bien es cierto que no necesariamente han de encontrarse el sensor y el procesador en el mismo lugar, ya que hasta cierto punto pueden realizarse conexiones por cable con otras zonas del campo, puede que esta sea la manera más eficiente de desarrollar la aplicación porque cuanto más complejo sea el sistema, más dificultades tendrá su montaje y uso en la competición donde se cuentan con escasos minutos para toda la preparación previa al partido, lo que incluye el encendido y configuración de los robots, balizas, otras tareas secundarias de la competición y este sistema de visión.



Figura 1.3.1.2 - Ejemplo de Soporte - Final Francia Eurobot 2023

Por ello, el soporte del sistema debe facilitar esta tarea de montaje y configuración siendo lo más compacto e intuitivo de usar posible, disminuyéndose así los errores de uso.

Finalmente, comparando las distintas consideraciones explicadas anteriormente se decide el uso de una Raspberry Pi Modelo 4B+ para el desarrollo de la aplicación por diversas razones:

Esta placa de desarrollo es actualmente de las más potentes y escogidas para su uso en aplicaciones que incluyen sistemas de visión por computador si obviamos el uso de ordenadores específicos los cuales pueden no cumplir con las restricciones de tamaño y peso de la competición. En cambio, la Raspberry Pi son pequeñas y ligeras.

Esta última generación de Raspberry Pi cuenta con una CPU multicore y GPU integrada, lo que permitirá manejar tareas de procesamiento de imágenes con mayor eficiencia.

Ya existen diversas cámaras y sensores diseñados específicamente para la Raspberry Pi que facilitan la captura de imágenes además de contar con numerosos puertos USB tanto 2.0 como 3.0 y los puertos específicos para cámaras “CSI”.

Este dispositivo es compatible con numerosas bibliotecas y frameworks como OpenCV, TensorFlow y PyTorch. Además de poder ejecutar ROS para llevar a cabo la comunicación con el resto de los sistemas que el equipo necesita para la competición

Otro principal atractivo para su uso es el contar con una comunidad activa de usuarios y desarrolladores que comparten proyectos y recursos, por lo que se espera que el desarrollo cuente con menores dificultades que en el caso de utilizar una plataforma distinta, además de que el equipo ya cuenta con cierta experiencia en el uso de esta tarjeta de desarrollo, que es la principalmente utilizada como procesador principal de los robots que se presentan en la competición desde hace ya algunos años.

Dadas las necesidades de procesamiento de la aplicación, las recomendaciones son utilizar la configuración de RAM más alta disponible, por lo que se asignaran 8Gb a esta tarjeta. Y el sistema operativo que posteriormente se detallará, será cargado sobre una tarjeta microSD SanDisk de 128Gb (aunque no se requiere tanta memoria, esta debería ser, como mínimo, de 32Gb) de tercera generación lo que permite una velocidad de lectura de 190Mb/s, una característica muy importante para poder disponer de mayor fluidez al utilizar el sistema operativo cargado en la misma.

Como sensor se selecciona inicialmente una cámara Sony IMX708 desarrollada expresamente para su uso en Raspberry Pi utilizando el canal CSI dadas sus prestaciones: 12Mpx, ángulo de visión de 120º, soporte HDR y enfoque automático configurable entre otros. Finalmente, se utilizará un sensor Sony Starvis CMOS montado en una cámara de la marca “Elgato”, modelo “Facecam”, como se verá más adelante. Este dispositivo cuenta con características similares al anterior, pero es significativamente más grande, con una resolución de 1920x1080px configurable, velocidad de captura de 60hz y un ángulo de apertura de 82º (significativamente



Figura 1.3.1.3 - Elgato Facecam

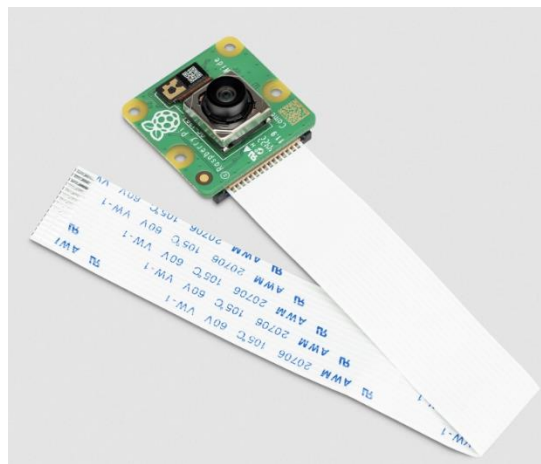


Figura 1.3.1.4 - RaspiCam V3 Wide

menor al anterior). En este caso, la comunicación se realiza por puerto USB necesariamente compatible con tecnología 3.0, lo cual no es un problema dado que la cuarta generación de tarjetas Raspberry Pi cuenta con dos puertos de estas características.

En el caso del soporte del sistema, se decide realizar un diseño propio que cumpla con las restricciones marcadas anteriormente, debido a que no es posible encontrar ningún trípode o carcasa que se adapte a todas las necesidades del proyecto.

El soporte se diseñará mediante el uso de la herramienta Autodesk Inventor y la fabricación se realizará como se explicará en más detalle posteriormente combinando componentes comerciales (tornillería, tubos de acero/aluminio...) con piezas de fabricación propia mediante el uso de una impresora 3D de tecnología FDM.

- **Otras herramientas y componentes:**

A parte de los principales materiales con los que se lleva a cabo el proyecto, se han utilizado, de manera adicional una serie de herramientas con las que realizar el desarrollo del trabajo, componentes cuyas características se resumen a continuación, aunque podrían utilizarse otros diferentes:

- Ordenador de sobremesa Intel Core i9 – 10900K 3.70 GHz

Dispositivo desde el cual se realiza la mayor parte del proceso de redacción de la memoria del presente documento e investigación. Así como su utilización en el diseño de los componentes para soporte del sistema solución del proyecto.

- Ordenador portátil HP Intel Core i5 – 6200U 2.3 GHz

Dispositivo utilizado esencialmente para modificaciones del código de pruebas y durante la fase de testeo de la solución final.

- Impresora 3D Artillery X2.

La máquina de fabricación aditiva utilizada a lo largo del proyecto para la producción en filamento plástico (PLA) de soportes, carcasas de protección, plantillas de prueba...

- Cables 3.1 compatibles

Cables USB especiales para la comunicación entre el procesador y el sensor de diferentes tamaños.

- Herramientas de propósito general

Destornilladores, llaves, sierra, baterías, cables y otros elementos para el montaje del sistema de soporte.

1.3.2 Selección de Software

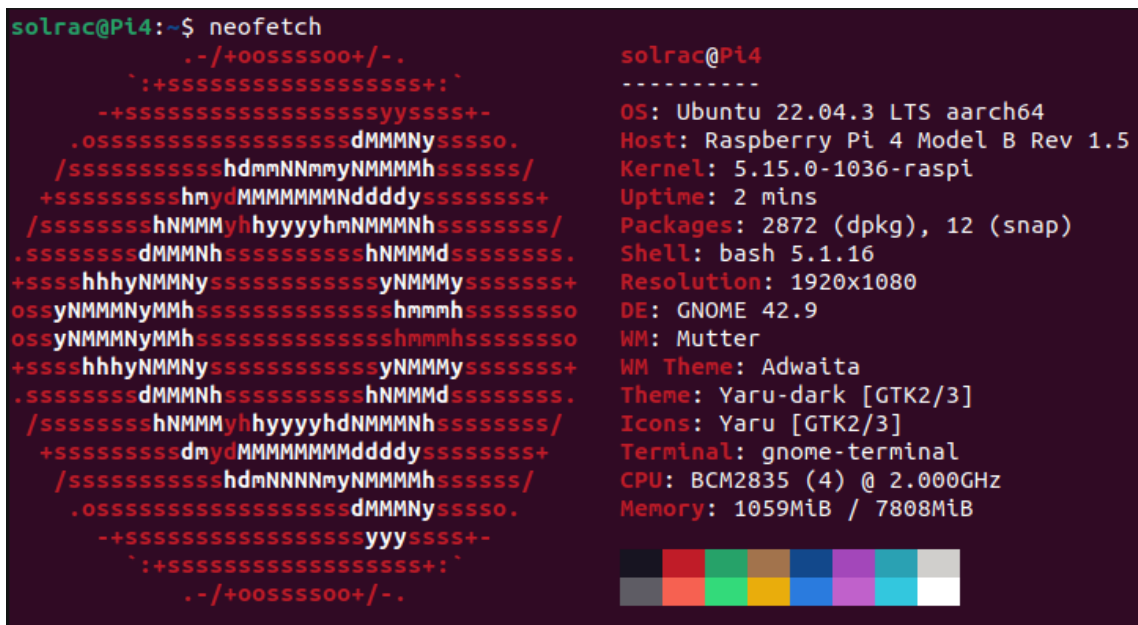
Para llevar a cabo el proyecto ha sido necesaria la selección de diversos programas y lenguajes con los que controlar el procesador y desarrollar los códigos del programa de visión de la solución propuesta.

Principalmente podemos destacar:

- **Ubuntu Mate 22.04.3 LTS**

Un sistema operativo Linux de escritorio de código abierto con una interfaz de usuario amigable y soporte hasta 2027, lo que lo hace especialmente apropiado para proyectos de larga duración. Además, es una elección aconsejable para Raspberry Pi 4 debido a su diseño ligero y eficiente que se ejecuta de manera fluida.

Además, aunque Raspberry Pi tiene su propio ecosistema de software, Ubuntu Mate proporciona acceso a una amplia gama de aplicaciones y herramientas de software de Linux, lo que amplía significativamente las posibilidades de uso de la Raspberry Pi.



```
solrac@Pi4:~$ neofetch
.-/+00SSSS00+/- .
`:+SSSSSSSSSSSSSSSS+:`
-+SSSSSSSSSSSSSSSSyySSSS+-
.oSSSSSSSSSSSSSSSSdMMMNYssSSo.
/SSSSSSSSSSShdmmNNmyNMMMMhSSSSS/
+SSSSSSSSShmydMMMMMMNdddySSSSSSS+
/SSSSSSSSShNMMMyhhyyyhmNMMMNhSSSSSSS/
.SSSSSSSdMMMNHSSSSSSSSShNMMMdSSSSSSS.
+SSShhhyNMMNYSSSSSSSSSSyNMMMySSSSSSS+
oSSyNMMMNyMMhSSSSSSSSSSShmmhSSSSSSSso
oSSyNMMMNyMMhSSSSSSSSSSShmmhSSSSSSSso
+SSShhhyNMMNYSSSSSSSSSSyNMMMySSSSSSS+
.SSSSSSSdMMMNHSSSSSSSSShNMMMdSSSSSSS.
/SSSSSSSSShNMMMyhhyyyhdNMMMNhSSSSSSS/
+SSSSSSSSdnydMMMMMMNdddySSSSSSS+
/SSSSSSSSShdMNNNNnyNMMMMhSSSSSSS/
.oSSSSSSSSSSSSSSSSdMMMNYssSSo.
-+SSSSSSSSSSSSSSSSyyySSSS+-
`:+SSSSSSSSSSSSSSSS+:`
.-/+00SSSS00+/- .

solrac@Pi4
-----
OS: Ubuntu 22.04.3 LTS aarch64
Host: Raspberry Pi 4 Model B Rev 1.5
Kernel: 5.15.0-1036-raspi
Uptime: 2 mins
Packages: 2872 (dpkg), 12 (snap)
Shell: bash 5.1.16
Resolution: 1920x1080
DE: GNOME 42.9
WM: Mutter
WM Theme: Adwaita
Theme: Yaru-dark [GTK2/3]
Icons: Yaru [GTK2/3]
Terminal: gnome-terminal
CPU: BCM2835 (4) @ 2.000GHz
Memory: 1059MiB / 7808MiB
```

Figura 1.3.2.1 - Tabla de características del OS cargado (Ubuntu 22.04.3 LTS)

- **Python 3 y OpenCV**

Uno de los lenguajes de programación más famosos de hoy día por su facilidad de uso, soporte con comunidad activa y en crecimiento, además de ser las principales herramientas con las que se ha trabajado anteriormente durante los estudios de máster en la asignatura “Sistemas de percepción”, por lo que esta herramienta de programación junto con sus librerías de OpenCV son una buena combinación para realizar el proyecto.

- **ROS2 (Robot Operating System)**

Un conjunto de bibliotecas de código abierto diseñado que conforman un entorno de desarrollo que se ejecuta en sistemas operativos basados en Linux, como es en este caso Ubuntu Mate, y se utiliza para gestionar robots o sus entornos de tal forma que ayuda a desarrollar softwares para robots.

Como se explicaba en la introducción, en 2017 se lanzó su versión dos que, a partir de ahora, será la que recibirá soporte y actualizaciones, por lo que, amén de realizar un proyecto fácilmente actualizable, se decide utilizar esta versión además de ser la más compatible con la versión de Ubuntu 22.03.

1.4. Desarrollo del Trabajo/Resultados y Discusión

En este cuarto apartado de la memoria del proyecto se realiza una descripción detallada del desarrollo de los diferentes programas que conforman la solución final. Se documentan los pasos seguidos a lo largo de todo el proyecto para implementar el sistema en el controlador Raspberry Pi 4B+ con el sistema operativo Linux Ubuntu Mate 22.04.3 LTS. Se incluye la instalación de los intérpretes de Python y sus librerías necesarias, así como la implementación en ROS2 para establecer la comunicación con los otros sistemas que forman parte del equipo de competición Red Wire.

Este proyecto de fin de máster establece una serie de metodologías para desarrollar aplicaciones en ROS2 utilizando Python3. Se ha dado especial importancia a la recopilación de documentación durante los pasos realizados en el proyecto para fijar un procedimiento eficaz que minimiza la aparición de errores y se proporcionan soluciones a los problemas que puedan surgir.

Además, se destaca la relevancia de la ingeniería de sistemas de visión por computador en este proyecto. Se detallan todos los procesos relacionados con la calibración, el cálculo de resultados, las consideraciones ambientales y otros aspectos relevantes en este campo.

Por último, se hace una descripción del diseño de los mecanismos de soporte que conforman la estructura física de la solución propuesta, la cual, como se verá más adelante, gana una gran importancia a la hora de poder homologar el proyecto en el entorno de la competición Eurobot.

En resumen, este apartado de la memoria del proyecto se centra en proporcionar una descripción completa y detallada del desarrollo e implementación de la solución final y establecer las bases para el desarrollo de proyectos similares en el equipo.

1.4.1 Instalación del Sistema Operativo y Elementos de Desarrollo

El primer paso en el desarrollo del proyecto es instalar un OS (Operating System) el controlador seleccionado, un entorno compatible con ROS y que permita escribir y administrar los programas de prueba. Por ello se opta por la última versión, a fecha de este proyecto (2023), del sistema desarrollado por Canonical Ltd, Ubuntu Mate 22.04 LTS (Jammy Jellyfish) un sistema operativo basado en Debian GNU/Linux que ha ganado especial fama por su facilidad de uso y agradable experiencia del usuario. Este sistema es descrito por el director de ingeniería de Ubuntu como “la culminación de 2 años de mejora continua para Ubuntu y MATE Desktop” (2022-03-29 Martin Wimpress)

- **Preparación del OS**

Para instalar cualquier OS en una Raspberry Pi hay que hacer uso de una tarjeta microSD que funcionará como disco duro. Al alimentar el controlador busca en la memoria interna el lanzador del sistema durante el proceso de arranque, por lo que esta memoria debe tener “flasheada” una ISO con el sistema operativo que se haya escogido.

Para “flashear” la tarjeta microSD con un sistema específico para su uso en una Raspberry Pi la mejor opción es usar la herramienta “Raspberry Pi Imager”, en concreto para este proyecto, la versión v1.7.5 tras haber descargado previamente el archivo .ISO del sistema operativo directamente desde el apartado de descargas de la página de Ubuntu Mate, véase la siguiente figura:



Figura 1.4.1.1 - Pasos para la descarga del OS (<https://ubuntu-mate.org/download/>)

- **Instalación del OS**

Una vez descargada la ISO de Jammy Jellyfish se accede a Raspberry Pi Imager y se selecciona “Operating System Choose OS” y agregamos en la opción “Use Custom” el archivo descargado.

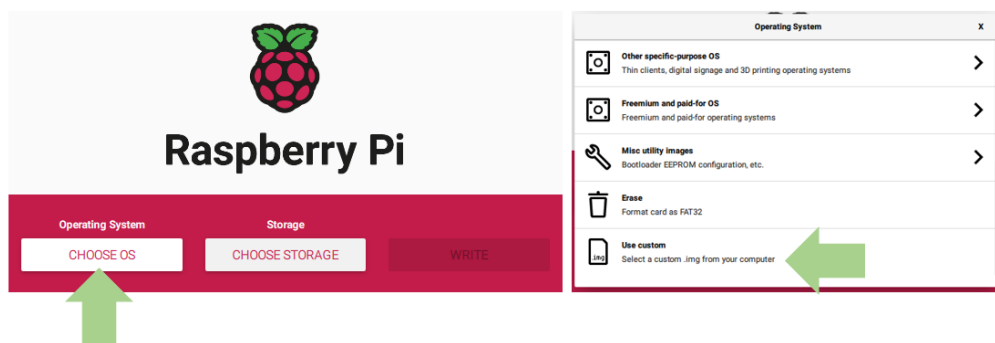


Figura 1.4.1.2 - Pasos para cargar el Ubuntu en la MicroSD

Tras esto sólo es necesario insertar la SD en el ordenador usando, en este caso, un adaptador que permite transferencias de alta velocidad lo que permite ahorrar un tiempo considerable en la escritura del sistema y, como se mostrará más adelante, para clonar la microSD haciendo así “BackUps” de seguridad.

Una vez realizados los pasos anteriores tan sólo hay que introducir la “SDcard” en el puerto específico para ese uso de la Raspberry Pi y alimentar el micro. El sistema operativo se ejecuta automáticamente y es necesario hacer la configuración inicial para poder empezar a trabajar en él.

- **Instalación de Python**

Para continuar con los preparativos del sistema es preciso instalar la última versión de Python y las librerías necesarias para el programa de visión. Para ello se recurre a la herramienta APT directamente desde el terminal de Linux, abriéndolo con el atajo de teclas “ctrl+alt+t” desde el escritorio.

Si nunca se han actualizado las librerías de APT es conveniente realizar la actualización de forma manual:

```
sudo apt update
```

```
sudo apt upgrade
```

Tras actualizar puede accederse al repositorio de Python e instalar la versión más reciente, para este proyecto será la versión 3.11.

```
sudo add-apt-repository ppa:deadsnakes/ppa
```

```
sudo apt install python3.11
```

Para comprobar si la instalación se ha realizado de manera satisfactoria se puede pedir al terminal que devuelva la versión de Python:

```
python3 --version
```

Aun mostrándose de manera correcta la versión de Python instalada, existe la posibilidad de que la IDE de programación que se instale no encuentre el intérprete de Python, un error que ha surgido en Visual Studio Code y que tiene por solución instalar un paquete del sistema:

```
sudo apt install python-is-python3
```

Básicamente se trata de un “Alias” que permite al IDE reconocer Python3 como el intérprete de lenguaje Python.

Se decide utilizar la IDE de Visual Studio Code, por la parte de programación, debido principalmente a la costumbre. VS Code es una herramienta versátil con gran cantidad de “Skills” que permiten acelerar el trabajo o aumentar su calidad, aparte de tener acceso directo al terminal de Github, por lo que se puede llevar a cabo un historial de versiones mucho más rápido que si no estuviera integrado. Pero realmente para este proyecto podría utilizarse cualquier otra IDE que permita trabajar con Python.

- **Instalación de bibliotecas**

Los programas realizados para cumplir con los objetivos del proyecto se basan en Python y, como parte fundamental de la implementación, es necesario instalar el conjunto de bibliotecas en el entorno de ejecución. Estas librerías desempeñan un papel crucial en el funcionamiento del programa ya que contienen las funciones de detección y tratamientos de las imágenes que se tomarán de la cámara.

Por ello son requeridas las siguientes librerías:

- OpenCV. Biblioteca fundamental de la que se extraerán todas las funciones de visión de la cámara y su pos-procesamiento a nivel imagen. Para instalarlo ha de escribirse el siguiente comando en el terminal:

```
sudo apt install libopencv-dev python3-opencv
```

- Numpy. Otra biblioteca muy importante para trabajar con las cadenas de valores que se obtienen de las funciones de OpenCV
- Si el IDE lo permite, la instalación de este y el resto de las bibliotecas puede realizarse directamente desde el terminal interno con la función PIP:

```
pip install numpy
```

- Time. De la que se extraen las funciones que permiten contabilizar el tiempo transcurrido a lo largo del programa para establecer la frecuencia de envío de datos.

```
pip install time
```

- Threading. Biblioteca que permite la gestión de hilos, en este caso para el envío de datos, lo que facilita la ejecución en paralelo del resto de tareas, mejorando así la eficiencia del programa.

```
pip install Threading
```

- **Preparación de la cámara**

Al comienzo del desarrollo del proyecto, se realizó la instalación de una cámara a través del puerto CSI y para ello fue necesaria la realización de una serie de ajustes en la Raspberry Pi para poder habilitar esos puertos.

Para ello es necesario acceder a la configuración del microcontrolador desde el terminal de Ubuntu escribiendo el comando:

```
sudo raspi-config
```

Y accediendo al menú que se muestra a continuación (figura 6.4.1.3) es posible activar el puerto CSI de la Raspberry Pi navegando hasta el apartado “Interface Options” y activando ahí la opción “Legacy Camera”:

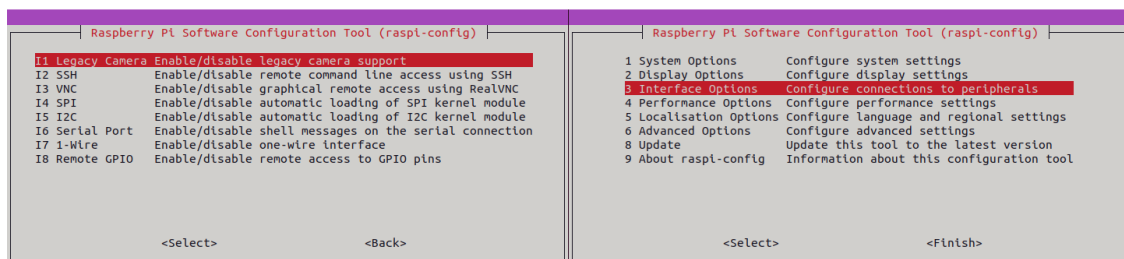


Figura 1.4.1.3 - Menú Raspi-config

Tras esto, es necesario reiniciar el controlador. Para ello resulta rápido lanzar por el terminal el siguiente comando:

```
Sudo shutdown -r now
```

Lamentablemente, y tras realizar numerosas pruebas, desde reconfigurar de manera específica el archivo “config.txt”, agregar herramientas para el control de cámaras en Raspberry Pi, e incluso cambiar al sistema operativo “Raspi OS”, específicamente diseñado para el funcionamiento más sencillo de las Raspberry Pi, no fue posible hacer funcionar el modelo de cámara adquirido. Se determinó que el fallo pudo venir por un mal uso o incluso el defecto de fábrica del propio puerto CSI del controlador, ya que se probaron otros modelos de cámara y se obtuvo el mismo resultado.

Finalmente se decide hacer uso de una cámara con transmisión por USB 3.1, el modelo comentado en anteriores apartados de “Elgato Facecam”.

Si la cámara está correctamente conectada y habilitada se puede revisar si la Raspberry Pi la está identificando correctamente. Para hacerlo, se ejecuta un comando que muestra una lista de los dispositivos de vídeo conectados escribiendo:

```
v4l2-ctl --list-devices
```

Presumiblemente, la cámara estaría alijada en el canal de vídeo 0 al ser la única conectada. Con el comando siguiente pueden comprobarse las características del dispositivo:

```
v4l2-ctl --all -d /dev/video0
```

- **Realización de copias de seguridad**

Una práctica recomendable a la hora de realizar proyectos complejos en Raspberry Pi que impliquen una configuración específica del sistema operativo cargado o muchos componentes organizados en distintos directorios es la de realizar copias de seguridad, “backups”, regularmente cuando se realizan avances significativos en el proyecto. Incluso dentro del equipo de robótica es habitual contar con múltiples tarjetas SD clonadas de los robots de competición ya que de producirse algún fallo y perder la información de la tarjeta original, es rápida y robusta la solución de insertar una copia exacta que contendrá la misma información y, por lo tanto, dará el mismo resultado que la original.

Para esta tarea se recurre a un programa especial de lectura y escritura de ISOs en Windows “Win32 Disk Imager”, un sencillo software, gratuito, con el cual se descarga y genera un archivo comprimido “.img” del sistema cargado en una memoria extraíble y permite volcarla en otra con muy pocos pasos.

Primero se selecciona el disco (MicroSD original) y se le pone un nombre y ubicación dentro del pc que se esté utilizando y se pulsa la opción “Read”. Una vez finalizado el proceso de lectura, puede conectarse la nueva memoria (MicroSD clon) seleccionarse el archivo “.img” creado y pulsar la opción “Write”.

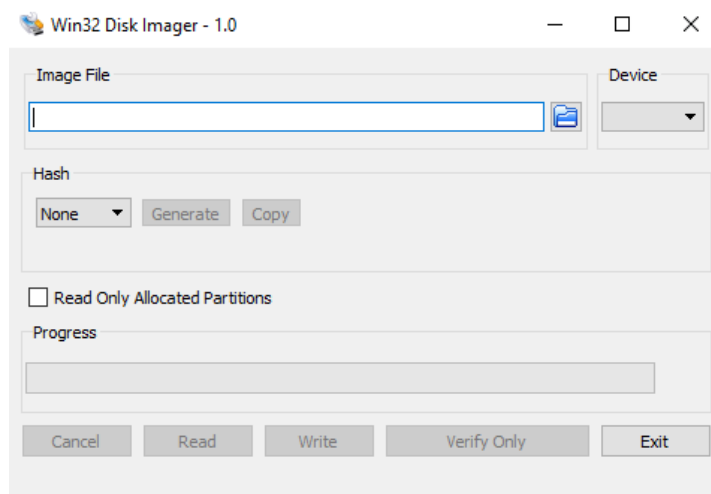


Figura 1.4.1.4 - Win32 Disk Imager

Al finalizar el proceso se obtiene una copia de seguridad del proyecto completo.

Para estos casos, y en función del peso de los archivos, es cuanto menos recomendable el uso de un lector de tarjetas de alta velocidad, como se comentó anteriormente, ya que estos archivos suelen pesar mucho y la carga y descarga puede demorarse incluso horas.

- **Overclocking**

Dependiendo de la complejidad del proyecto o consumo de recursos de este puede ser conveniente realizar un “Overclocking” al microcontrolador para mejorar su rendimiento, y teniendo en cuenta que este proyecto trabaja con tratamientos de imágenes, este es el caso más indicado.

Un “Overclocking”, como su nombre indica, consiste modificar, más concretamente aumentar, la frecuencia de reloj interno de la CPU y GPU de la Raspberry Pi aumentando de esta forma su rendimiento, tiempo de respuesta, etc. No obstante, esta práctica conlleva ciertos riesgos. Si bien es cierto que el microcontrolador no viene configurado de fábrica para explotar todo su potencial, forzar el procesamiento puede provocar el aumento de temperatura de los componentes internos de la Raspberry Pi lo que puede provocar un desgaste prematuro del dispositivo o incluso el fallo total del mismo. Por ello es recomendable hacer uso de un buen disipador de calor, con ventilador para extraer el calor por convección forzada. Además, no todas las frecuencias dan buenos resultados y es preciso realizar un proceso iterativo para buscar una frecuencia apropiada para la aplicación, aunque algunas frecuencias características son comúnmente válidas.

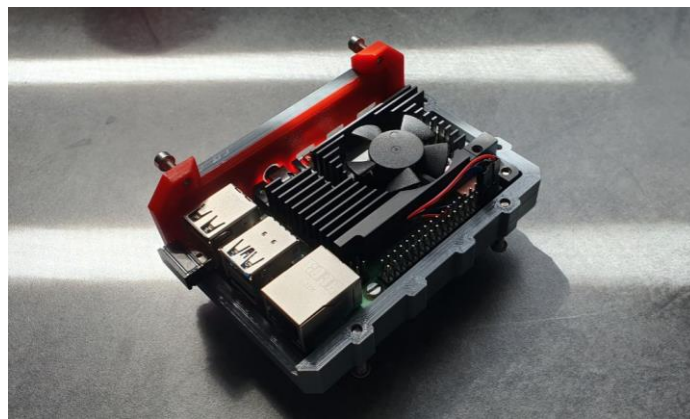


Figura 1.4.1.5 - Raspberry Pi con disipador y ventilador

De esta forma, para aumentar la frecuencia de reloj de la Raspberry Pi es necesario editar el archivo de configuración interno “config.txt” desde un editor de texto dentro de la memoria de la tarjeta SD como se explicaba en apartados anteriores.

Como resumen, una frecuencia que ha dado buenos resultados ha sido la de 2000Mhz para el cual se deben insertar las siguientes líneas al final del archivo “config.txt”:

```
#CPU Overclock
over_voltage=4
arm_freq=2000
#GPU Overclock
over_voltage=4
gpu_freq=620
```

1.4.2 Instalación de ROS2

- **Preparación**

Para poder acceder al repositorio de ROS2 Humble se debe realizar primero una serie de pasos preparativos.

Primero se instalarán los siguientes paquetes esenciales:

```
sudo apt install software-properties-common curl gnupg lsb-release
```

También es necesario instalar un gestor de paquetes software para administrar la instalación de ROS2 por lo que se hace uso de "Ubuntu Universe" en la Raspberry Pi, un software gratuito y de código abierto:

```
sudo add-apt-repository universe
```

Por último, antes de empezar con la instalación de ROS, es necesario descargar las claves GPG (GNU Privacy Guard) de ROS, unas licencias que permitirán la utilización de los paquetes de ROS2, y así poder agregar el repositorio ROS2 al directorio `/etc/apt/sources.list.d/` usando los siguientes comandos:

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o  
/usr/share/keyrings/ros-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-  
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(source /etc/os-release && echo  
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Una vez realizados los preparativos, es conveniente hacer una actualización del APT.

```
sudo apt update
```

```
sudo apt upgrade
```

- **Instalación**

Ahora sí, se procede a la instalación de ROS2 HUMBLE con el administrador APT.

La instalación puede demorarse varios minutos y solicitar confirmaciones ya que descargarán todos los componentes de ROS como son: bibliotecas ROS, Python y sus dependencias, herramientas de desarrollo...

```
sudo apt install ros-humble-desktop
```

```
sudo apt install ros-humble-ros-core
```

Al finalizar la instalación de ROS2 se procede a inicializarlo mediante el comando:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

- **Comprobación**

Si la instalación y la inicialización se han realizado de manera satisfactoria, es posible probar su funcionamiento ejecutando dos programas simultáneamente, uno publicará un mensaje de manera periódica y el otro lo recibirá por lo que es preciso abrir dos terminales

En el primer terminal se accede directorio donde está guardado el publicador y se ejecuta mediante los comandos:

```
cd /opt/ros/humble/lib/demo_nodes_cpp/
```

```
ros2 run demo_nodes_cpp talker
```

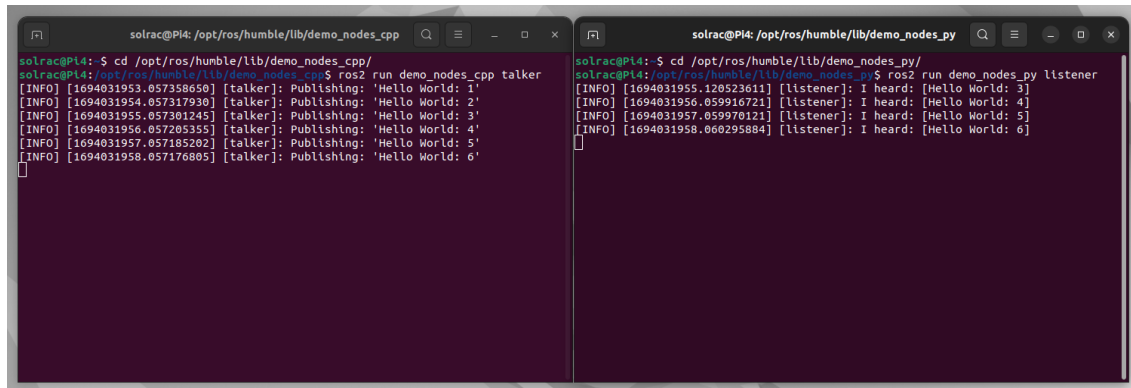
Después, en el segundo terminal se accede y ejecuta el programa receptor:

```
cd /opt/ros/humble/lib/demo_nodes_py/
```

```
ros2 run demo_nodes_py listener
```

Si todo funciona correctamente, el receptor muestra el mismo mensaje que el publicador está escribiendo en el nodo de prueba.

Para salir del programa presionamos "ctrl+c" en cada una de las terminales. Si se finaliza primero el programa del publicador vemos como el receptor deja de leer nuevos mensajes.



```
solrac@Pi4: /opt/ros/humble/lib/demo_nodes_cpp
solrac@Pi4: $ cd /opt/ros/humble/lib/demo_nodes_cpp/
solrac@Pi4: /opt/ros/humble/lib/demo_nodes_cpp $ ros2 run demo_nodes_cpp talker
[INFO] [1694031953.057358659] [talker]: Publishing: 'Hello World: 1'
[INFO] [1694031954.057317930] [talker]: Publishing: 'Hello World: 2'
[INFO] [1694031955.057301245] [talker]: Publishing: 'Hello World: 3'
[INFO] [1694031956.057205355] [talker]: Publishing: 'Hello World: 4'
[INFO] [1694031957.057185202] [talker]: Publishing: 'Hello World: 5'
[INFO] [1694031958.057176805] [talker]: Publishing: 'Hello World: 6'

solrac@Pi4: /opt/ros/humble/lib/demo_nodes_py
solrac@Pi4: $ cd /opt/ros/humble/lib/demo_nodes_py/
solrac@Pi4: /opt/ros/humble/lib/demo_nodes_py $ ros2 run demo_nodes_py listener
[INFO] [1694031955.120523611] [listener]: I heard: [Hello World: 3]
[INFO] [1694031956.059916721] [listener]: I heard: [Hello World: 4]
[INFO] [1694031957.059970121] [listener]: I heard: [Hello World: 5]
[INFO] [1694031958.060295884] [listener]: I heard: [Hello World: 6]
```

Figura 1.4.2.1 - Prueba de funcionamiento de ROS2

1.4.3 Desarrollo del Programa de Visión

Una vez configurado el entorno se puede comenzar a desarrollar la solución del sistema. Para ello, hay que tener en cuenta que existen diversas formas de afrontar el mismo problema desde el punto de vista de la computación, por lo que se busca desarrollar la solución que mejores resultados pueda proveer atendiendo a las necesidades transversales del proyecto como son la depuración, las fases de prueba, el desarrollo de nuevos elementos dado que la normativa de la competición varía cada año. Es decir, si bien el código escrito en Python debe ser lo más limpio posible de cara a mejorar el rendimiento del conjunto, también debe permitir cierto grado de modificabilidad para futuras competiciones y sencillez de cara a que otros miembros del equipo o externos puedan tomar el proyecto y mejorarlo, adaptarlo a otras soluciones o continuar con el proyecto planteado.

Por ello se va a elaborar esta parte de la memoria de forma que, como en otros apartados, pueda servir de guía para la comprensión y la replicación del proyecto actual.

Expuesto este punto, hay que tener en cuenta que hacen falta más preparativos para comenzar realmente con el desarrollo de la aplicación, pero en este punto, el proceso puede cambiar o devolver resultados diferentes para cada solución específica del proyecto.

- **Proceso de calibración del sensor.**

El proceso de calibración del sensor utilizado es un paso crítico para este proyecto y otros que busquen trabajar con sistemas de visión por computador, incluso si estos utilizan elementos controlados como son los marcadores ArUco.

El objetivo de la calibración es obtener los parámetros intrínsecos y extrínsecos del sensor, así como sus posibles distorsiones. Más concretamente, realizar una calibración controlada, como es el proceso de calibración por tablero de ajedrez, permite obtener características de la cámara como, por ejemplo, la distancia focal. De esta forma se pueden determinar las coordenadas en píxeles de un punto dentro de una imagen y relacionarlas con su verdadera posición en el mundo real por medio de la expresión:

$$s \begin{pmatrix} p' \\ l \end{pmatrix} = A \begin{pmatrix} Ry & T \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p \\ l \end{pmatrix}$$

Donde **A** es una matriz que contiene los parámetros intrínsecos, **Ry** y **T** contienen los parámetros extrínsecos (matriz de rotaciones y vector de translación respectivamente) y de esta forma el punto **p** dentro del sistema interno de la cámara se transforma en el punto **p'** del sistema global.

Para obtener estos parámetros por medio de la calibración existen herramientas específicas facilitadas por las librerías de OpenCV donde, a partir de una serie de imágenes tomadas con el sensor a calibrar que contengan un tablero de ajedrez de unas dimensiones determinadas y conocidas, se detecta la posición de los vértices del ajedrezado dentro de la imagen y se relacionan esos puntos con los correspondientes en el espacio real para conseguir los parámetros mencionados.

Para llevar a cabo esta tarea, se utiliza un programa escrito en Python que incorpora la biblioteca de OpenCV y cuya función es la de acceder a la cámara previamente montada en un soporte y conectada a la Raspberry Pi para, al pulsarse una tecla (en este caso la letra “c”), tomar una captura y extraer de ella los vértices de los cuadrados del ajedrezado mencionado.

Es importante recalcar que ha de hacerse uso de un patrón de calibración (el tablero de ajedrez) con dimensiones conocidas, e informar al programa de estas dimensiones para que la calibración sea correcta. Para evitar errores, la superficie del ajedrez debe ser completamente plana, por lo que se utilizó un tablero de madera para asegurar esta condición.

Dimensiones del patrón de calibración:

Ajedrez_DIM = (9, 6) # El número de vértices internos de ancho y largo

Cuadrado_Tamaño = 38.0 # Tamaño de cada cuadro en milímetros

#Nota: Vertices internos = Nº Cuadrados - 1

Para asegurar una alta precisión en la calibración se deberían tomar al menos ocho imágenes desde diferentes ángulos y posiciones como se muestra en la siguiente figura:



Figura 1.4.3.1 - Composición de las imágenes de calibración

Como se aprecia en la figura anterior, el programa devuelve las imágenes capturadas añadiendo la visualización de los vértices detectados, por lo que, si la calibración ha sido satisfactoria, los puntos dibujados deben corresponder con los vértices de los cuadrados.

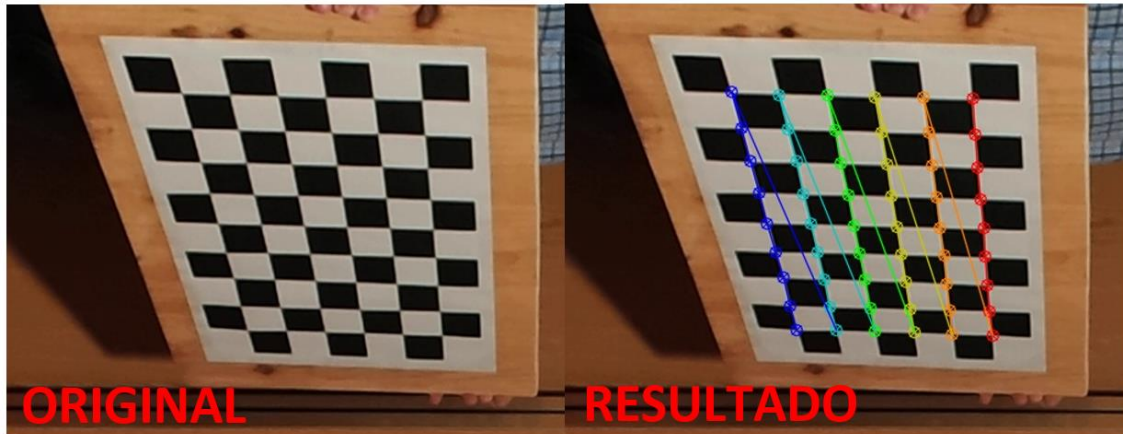


Figura 1.4.3.2 - Comprobación de la calibración

Una vez realizada la calibración, el programa devolverá un archivo “.npz”, un formato especial para el tratamiento de datos desde Python con el nombre “calibration_data” que será el utilizado a partir de ahora para el desarrollo del resto del proyecto.

Puede revisarse el código empleado y el patrón de calibración en el apartado de anexos.

- **Consideraciones del programa.**

Como se ha ido explicando anteriormente, este proyecto busca encontrar un punto intermedio entre diversas características como son: sencillez para su futura comprensión, simplicidad y depuración para mejorar la eficiencia, precisión para la correcta detección y uso, etc.

Por ello, hay que tener en cuenta que al tratarse de un complejo sistema de posicionamiento de los marcadores no siempre puede hacerse uso de complicadas herramientas de procesamiento de la imagen, puesto que afectarían al rendimiento del controlador, incluso después de haber realizado un “Overclocking” sobre este. Es necesario tomar ciertas consideraciones del entorno que se está analizando y concretamente en este proyecto se tiene la suerte de contar con un mapa cerrado y conocido.

Esto quiere decir que, los marcadores siempre se van a encontrar en un área marcada por los límites del campo y siempre se van a encontrar los mismos códigos para los distintos componentes que forman el partido, ya sean robots, de un equipo u otro, puntos de referencia fijos u objetos a capturar tal y como especifica la normativa de Eurobot.

Como se ha mencionado anteriormente, la competición pone a disposición diversos recursos útiles para este tipo de proyectos de detección y posicionamiento por códigos ArUco, como son para la normativa de este último año, la distribución de cuatro puntos de referencia fijos marcados con los códigos del 20 al 23 incluidos. Como estos últimos, todos los ArUcos entre el valor 1 y 100 son conocidos, tanto en designación como en tamaño, siendo todos marcadores con patrón 4x4. Esto simplifica significativamente el programa ya que solo se van a buscar unos determinados códigos y se trabajará con cada uno de ellos de una forma específica.

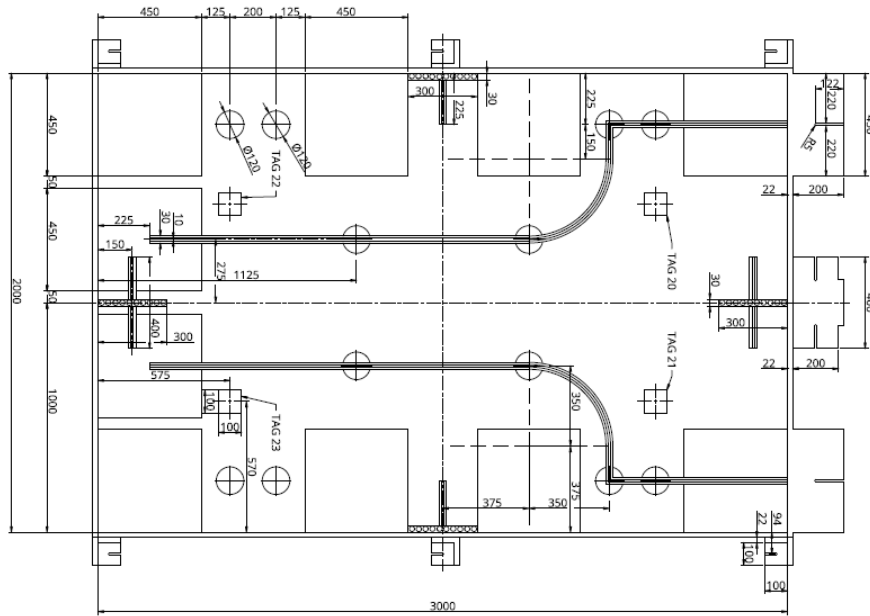


Figura 1.4.3.3 - Posiciones de las referencias del campo (Normativa Eurobot 2023)

También son conocidas las cotas de altura a las que podemos encontrar los distintos marcadores en el mapa, siendo por ejemplo la cota 510mm aquella reservada a contener los ArUcos montados sobre los robots.

Además, es posible asumir la posición inicial de los robots del equipo propio, ya que, dentro de unas áreas expresas para la colocación en el inicio del partido, cada robot debe colocarse con la mayor precisión posible en una determinada posición y orientación. Esto será de gran ayuda a la hora de eliminar errores procedentes de la precisión de las dimensiones del campo, posicionamiento de la cámara, etc.

Como se verá en próximos apartados, el diseño de la estructura de soporte se ha realizado teniendo en cuenta las implicaciones que conlleva una variación en la inclinación o posición de la cámara, por lo que se ha procurado una estructura que permita asegurar (excluyendo leves variaciones) que el sensor de la cámara se encuentre siempre en la misma posición a 525mm con respecto al suelo del campo y una inclinación de 43° con respecto al vector normal del campo. Estos valores son algo distintos a los previstos en el anteproyecto y esto es debido a la necesidad de cambiar el sensor durante la fase de desarrollo como se explicaba en anteriores capítulos.

- **Programa Python y OpenCV**

A lo largo de este punto se van a explicar las secciones del código desarrollado y cómo se ha ido adaptando la solución pasando por diversas fases de prueba o líneas de investigación añadiendo nuevas funcionalidades a las ya enumeradas o descartando algunos objetivos según se ha ido realizando el estudio.

El programa final, se divide fundamentalmente en tres secciones:

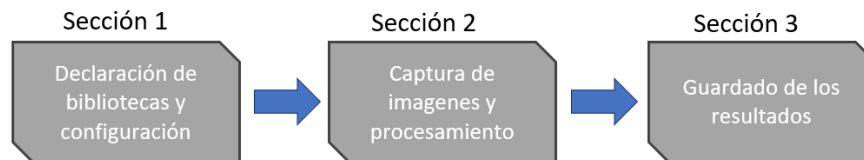


Figura 1.4.3.4 - Secciones del programa

- **Sección 1**

Esta primera sección contiene la “configuración” y declaración de variables del programa, así como la importación de las bibliotecas necesarias para el resto de las secciones del código. A parte de estos puntos, en esta sección se ha prestado especial interés en establecer variables de configuración que permitan bloquear o habilitar ciertas funcionalidades del código. Como se ha explicado, esta aplicación puede utilizarse en distintas situaciones, siendo posible que, durante las fases de testeo de los robots del equipo, pueda ser de interés añadir más información para la depuración de trayectorias, contar con datos sobre la orientación del robot, las distancias de seguridad establecidas, el tiempo de partido... y mostrar en una pantalla estas características en directo. Sin embargo, durante el partido no está permitida la depuración o el control externo de los sistemas que estén en el área de juego, por lo que hay muchas utilidades que consumen cierta cantidad de recursos que pueden ser deshabilitadas para sólo extraer la información necesaria.

- **Sección 2**

En esta sección se realiza la obtención y procesamiento de las imágenes, dedicada a la obtención de los datos marcados anteriormente entre los que destacan principalmente la posición en el plano de todos los robots del partido y su orientación, ya que este valor puede resultar interesante en robots con morfologías como la de tracción diferencial, donde no es posible que el robot avance de manera rectilínea fuera de su eje de tracción. Además, la descripción de curvas no es una práctica que abunde en la competición, ya que estos movimientos son complicados de realizar y pueden introducir errores en odometría basada en encoders sin suponer una ventaja sustancial en tiempo de operación, por lo que siempre se suele preferir realizar movimientos rectos o giros sobre el propio centro de giro del robot.

Volviendo al desarrollo de la sección dos del código, esta parte se encuentra dentro de un bucle de lectura, el cual engloba todo lo marcado en rojo en el diagrama mostrado más adelante, continua hasta que el usuario finaliza el programa, ya que no hay restricciones con respecto a este punto en la normativa, el sistema puede estar encendido incluso desde antes del comienzo del tiempo de preparación del partido y simplemente no considerar las respuestas del programa en los tiempos externos a la duración del partido, algo que también sucede en otras funciones como la de los “LIDAR”, unos sistemas que tardan cierto tiempo

en inicializarse y por ello la estrategia suele ser la de entrar al campo con estos accesorios ya encendidos.

A lo largo de esta sección se realizan los siguientes cálculos importantes:

1. Se solicita la detección de todos los ArUcos dentro del último frame capturado por la cámara.
2. Se recorre la lista de ArUcos detectados extrayendo una primera estimación de la "pose".
3. En caso de estar habilitado, se trazan sobre el ArUco los ejes de posición en función de su orientación y su ID.
4. Se extrae la información correspondiente al ángulo de orientación del marcador con respecto a su eje normal "z" de tal forma que, si el marcador está correctamente alineado con el robot, es posible conocer su dirección.
5. Se realiza una corrección de la posición del marcado dado el cambio de perspectiva de la cámara con respecto al campo por la inclinación de 43º descrita anteriormente.
6. Si está habilitada, se dibuja sobre la imagen unas "áreas de seguridad" que permiten identificar cuando dos robots se encuentran muy próximos entre sí.
7. Se calcula la distancia entre cada uno de los ArUcos detectados si estos pertenecen a un robot, de tal forma que se conoce la distancia entre cada uno de los robots del campo. Si esta distancia es inferior a un valor, se genera una alarma y las "áreas de seguridad" cambian de color verde a rojo.

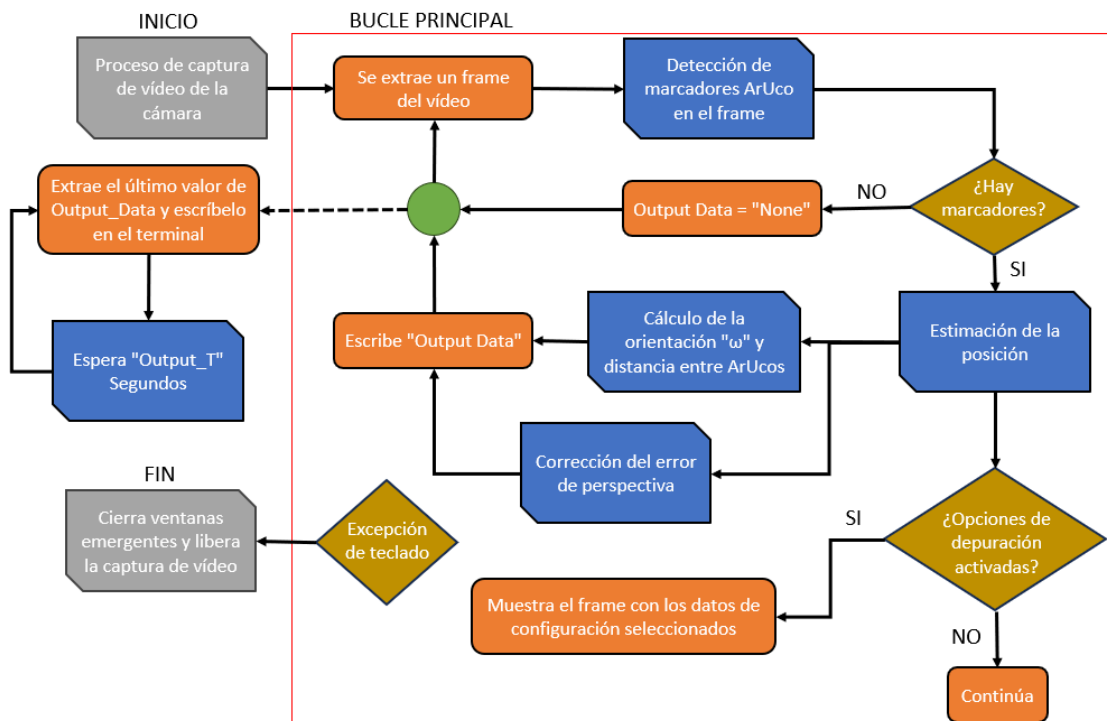


Figura 1.4.3.5 - Flujograma de procesos

- Sección 3

Finalmente, en la tercera sección se genera un hilo secundario al programa principal que se encarga periódicamente de extraer los datos calculados a lo largo del programa para prepararlos para su envío a través de ROS como un publicador. Es importante utilizar esta función, ya que de esta forma se consigue siempre una transmisión periódica de los datos de salida ya que de realizarse en mitad del hilo principal podrían generarse tiempos muertos y no asegurar la correcta frecuencia de transmisión de los datos.

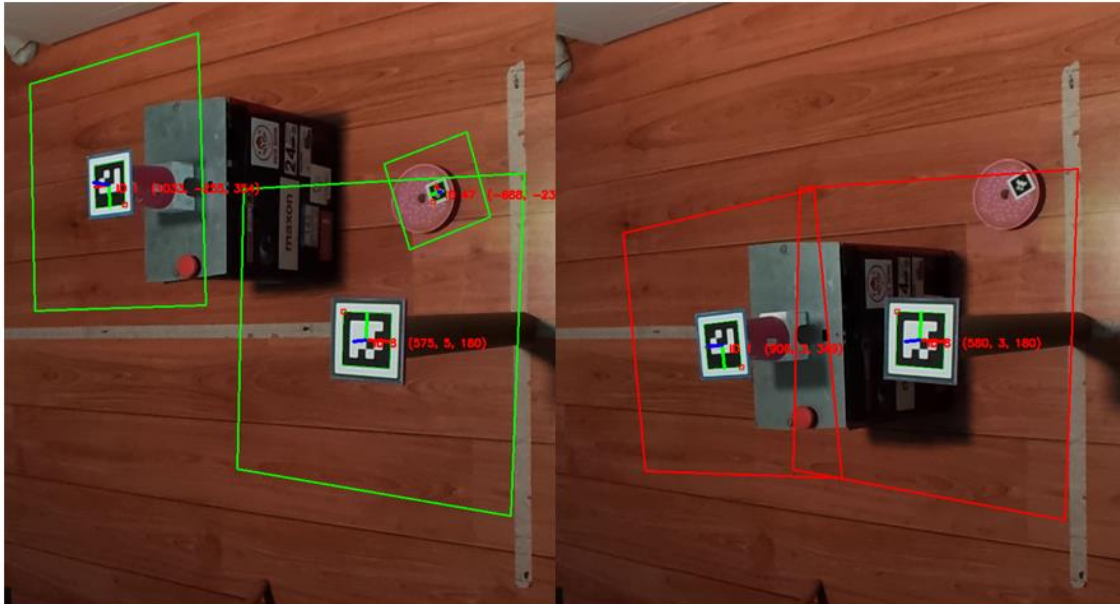


Figura 1.4.3.6 - Ejemplo de funcionamiento de las "áreas de seguridad"

- **Cálculo de la posición y la orientación**

Para realizar el cálculo de la posición de los ArUcos no basta con las funciones de OpenCV que detectan el centro del marcador dentro del plano de la imagen y hacen la transformación al mundo real por medio de la matriz de parámetros intrínsecos y extrínsecos vista anteriormente, ya que esta metodología presupone que los códigos se encuentran paralelos al plano de imagen, un supuesto que aquí no se cumple debido a que la cámara se encuentra en uno de los laterales del campo y no justo encima de él.

Para mejorar la estimación de la posición es necesario hacer una corrección del error debido al ángulo de inclinación de la cámara con respecto al plano del campo, para lo cual puede hacerse uso de estrategias de corrección de perspectiva que utilizan matrices de transformación, pero regresando a los supuestos mencionados anteriormente. Se conoce perfectamente la posición de la cámara en el mapa y su orientación, por lo que puede simplificarse notablemente el cálculo y por lo tanto también así las exigencias de rendimiento del microprocesador si compensamos el error de forma lineal, ya que la inclinación se da tan solo en uno de los ejes, por lo que se procede a ajustar los valores de compensación hasta que el error de posicionamiento disminuye.

En cuanto al cálculo de la orientación de los ArUcos entorno al eje vertical, es posible calcular su valor a partir del vector de rotaciones “rvec[]” que devuelve la función de OpenCV “estimatePoseSingleMarkers()” extrayendo la matriz de rotaciones con la función “Rodrigues()” y aplicando la siguiente ecuación:

$$R_z = \begin{pmatrix} \cos \omega & -\sin \omega & 0 \\ \sin \omega & \cos \omega & 0 \\ 0 & 0 & 1 \end{pmatrix} ; \omega = \tan^{-1} \left(\frac{R_z[1,0]}{R_z[0,0]} \right)$$

Una vez recopilados todos los datos se envía un mensaje a través del terminal que sigue la siguiente estructura: “msg: (Aruco_ID, Pose_X, Pose_Y, orientación_W)”

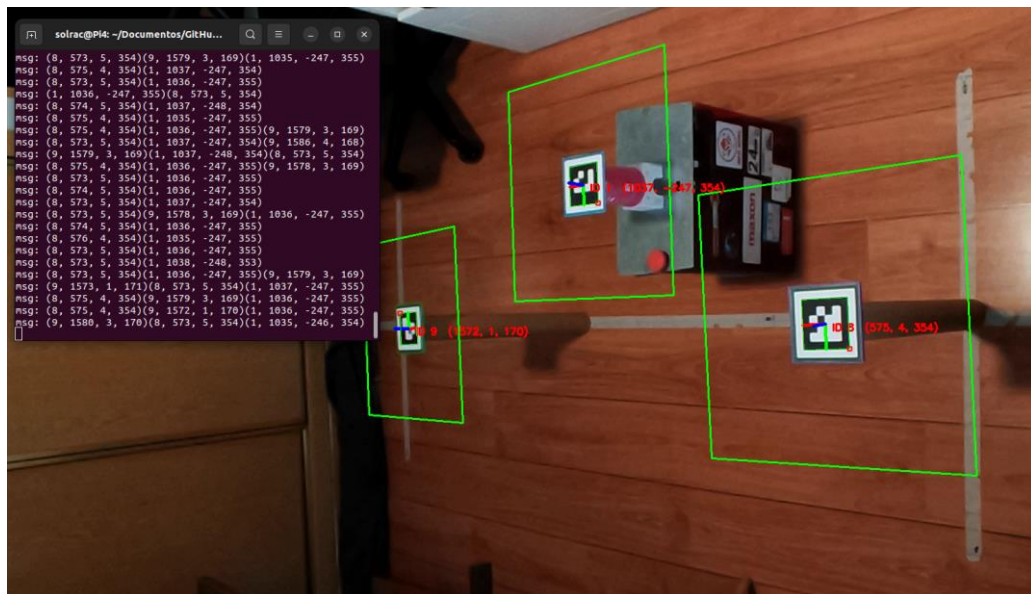


Figura 1.4.3.7 - Ejemplo de funcionamiento del programa de visión

1.4.4 Desarrollo del nodo de ROS

En este capítulo se procede a explicar el proceso de configuración de ROS2 para la creación de un espacio de trabajo con sus dependencias y un nodo publicador capaz de transmitir los datos procesados por el programa de visión de la cámara a un topic de ROS2 de tal forma que otra máquina con un nodo receptor pueda leer el dato y utilizarlo cuando lo necesite. En este caso el receptor sería el robot del equipo que precisa de comprobación de su posición desde un sistema externo ya que, hasta ahora, el dispositivo se basaba en odometría y tomaba datos del exterior por medio de un LIDAR para exclusivamente detectar objetos en las inmediaciones de este y realizar paradas de emergencia para evitar una posible colisión.

El primer paso para escribir el nodo de ROS2 es la creación del espacio de trabajo en la Raspberry Pi, para ello es necesario instalar primero la herramienta “colcon” con la que se compilan los programas de ROS:

```
sudo apt install python3-colcon-common-extensions
```

Se genera el directorio de trabajo, preferiblemente desde la raíz de Ubuntu con un nombre que no incluya espacios ni mayúsculas, en este caso “ros2_ws” y dentro de este el directorio fuente “src”, que contendrá los paquetes que creados.

```
mkdir ros2_ws
```

```
cd ros2_ws
```

```
mkdir src
```

Ahora con la herramienta instalada anteriormente se genera la estructura de ROS2

```
colcon build
```

Se generarán varios directorios y archivos. Para configurar las funcionalidades de la estructura de ROS, como autocompletado, se puede editar el archivo “.bashrc” que se encuentra dentro de la carpeta “install”, para editarlo directamente desde “gedit”

```
Gedit ~/.bashrc
```

Y se añaden las siguientes sentencias al final:

```
source /opt/ros/humble/setup.bash
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash
source ~/ros2_ws/install/setup.bash
```

Una vez editado se guarda y cierra el archivo “.bashrc” y para confirmar los cambios en el terminal hay que introducir el comando:

```
source .bashrc
```

Ahora el espacio de trabajo esta correctamente creado y configurado. Ahora se debe crear un paquete que puede contener múltiples nodos con distintas funcionalidades. Para crear el paquete hay que ir al directorio “src” creado anteriormente y crear el paquete, de nuevo con un nombre sin espacios y en minúsculas. Para este proyecto se crea el paquete “arucocam_pkg”:

```
cd src/
```

```
ros2 pkg create arucocam_pkg --build-type ament_python --dependencies rclpy
```

Se compila el paquete desde la raíz del workspace:

```
colcon build
```

Es probable que, aunque se complete la compilación, aparezca un “warning” producido por el uso de una versión demasiado reciente de “setuptools”. Para solucionar este error hay que bajar esta herramienta de la versión 59.6.0 a 58.2.0

```
pip3 install setuptools==58.2.0
```

Una vez creado el paquete “arucocam” es posible escribir el nodo dentro de este. Para ello, dentro del directorio del paquete creamos el archivo “.py” del nodo.

```
cd /ros2_ws/src/arucocam_pkg/arucocam_pkg
```

```
touch arucocam_node.py
```

Es necesario habilitar este archivo como ejecutable:

```
chmod +x arucocam_node.py
```

Y con el archivo generado se comienza a escribir el código del nodo.

El nodo escrito consta esencialmente de tres secciones:

La primera sección sería la declaración de bibliotecas junto con la selección del entorno que se va a utilizar, en este caso el de "Python3", por ello el programa lleva como encabezado la sentencia: `#!/usr/bin/env python3` de esta forma ROS interpreta como debe utilizar este archivo.

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
```

Todas las bibliotecas que son dependencias de ROS deben tener una sección en el archivo "package.xml" por lo que aquí siempre se encuentra, mínimo, la biblioteca "rclpy" (ROS Client Library for Python) pero, además, para escribir los mensajes en el topic que provienen del programa de Python es necesario establecer el tipo de mensaje que se va a utilizar, y como en este caso el resultado se ha guardado en formato "string", se carga la biblioteca "std_msgs.msg".

En el segundo apartado tendríamos la clase que se va a definir, ya que ROS funciona con referencias a objetos y por ello se utilizó para la clase el nombre del nodo de tal forma que dentro existirán dos funciones, una función de iniciación que solo se ejecuta al principio del programa y sirve para definir las variables que se van a utilizar y establecer la condición de "Publisher" al nodo. Y también tendríamos otra función periódica, definida en la inicialización, que es la encargada de extraer el dato "Output_Data" del archivo ".dat" donde el programa de visión lo ha almacenado y enviarlo al topic establecido en la inicialización.

```
class ArucoCamNode(Node):

    def __init__(self):
        super().__init__("arucocam_node")
        self.create_timer(0.2, self.send_Output_Data)
        self.counter_ = 0
        self.Output_Data = "None"
        self.Output_Data_link =
        '/home/solrac/Documentos/GitHubRepositories/MUII/TFM/pipe.dat'
        self.Output_Data_Pub = self.create_publisher(String, "/outputdata_topic", 10)
        self.get_logger().info("ArucoCam_Node Activado")
```

Por último, estaría el "main" donde se lanza primero la función de iniciación, después establece la función nodo como repetitiva de tal forma que cuando acabe las sentencias de las funciones no acabara el programa, sino que repetirá el proceso, y por último estaría la finalización del programa donde cuando se sale del bucle de funciones se cierra el nodo de manera controlada.

```
def main(args=None):
    rclpy.init(args=args)
    node = ArucoCamNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

El código completo puede encontrarse en los Anexos del proyecto.

Tras esto ya está generado el nodo pero falta declararlo en el archivo “.setup” del paquete donde se declaran todos los nodos que este contenga y compilar el paquete.

```
entry_points={
    'console_scripts': [
        "arucocam_node = arucocam_pkg.arucocam_node:main"
    ],
}
```

Finalmente se ha generado en espacio de trabajo “ros2_ws” que contiene el paquete “arucocam_pkg” y a su vez en este último se encuentra el nodo “arucocam_node” que toma los datos resultantes del programa de visión desarrollado y los publica en el topic declarado “outputdata_topic” como se muestra en el esquema siguiente:

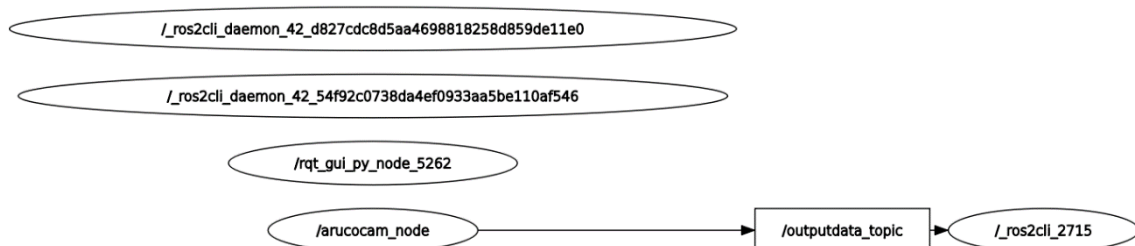


Figura 1.4.4.1 - Gráfico “rqt_graph” del proyecto

1.4.5 Diseño de la estructura de soporte

Como se ha venido explicando, no se le puede restar importancia al diseño de las estructuras que soportan los componentes del sistema, ya que la normativa de la competición exige unas determinadas características, tanto dimensionales como de seguridad, que deben respetarse para la correcta homologación del proyecto.

El diseño de estos soportes se ha realizado por medio del software de diseño 3D Autodesk Inventor, cuya licencia es aportada por la Universidad de Alcalá. Es con este software con el que se realizan las simulaciones del campo y los robots para su diseño.

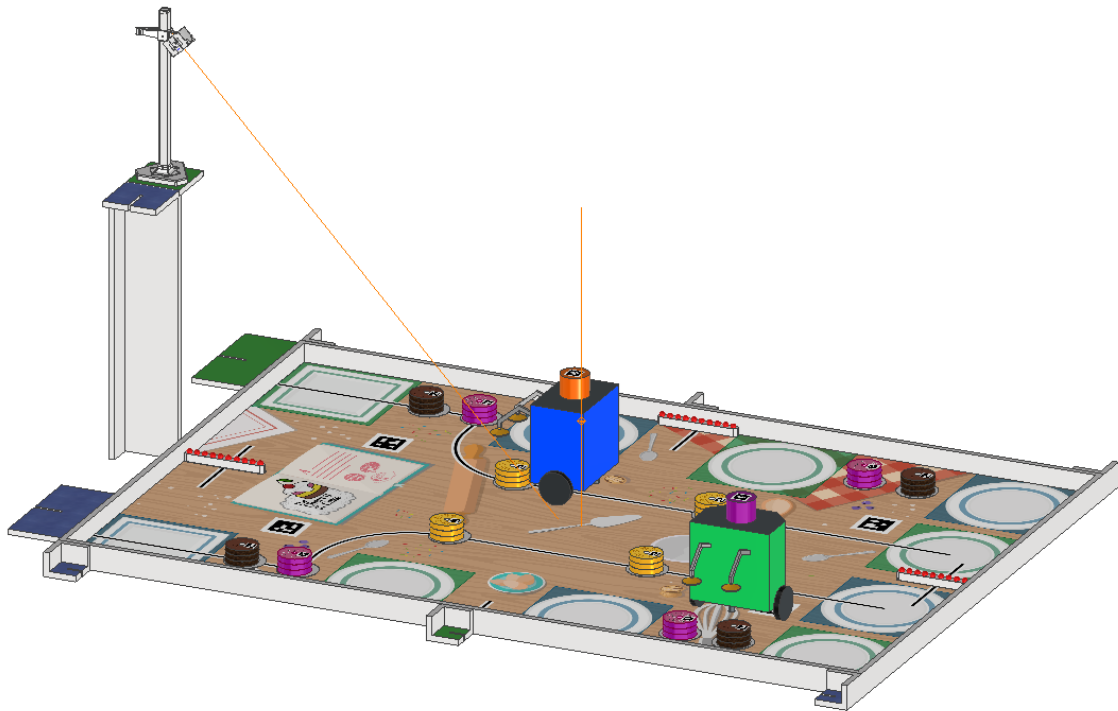


Figura 1.4.5.1 - Simulación del mapa de competición

Esta herramienta resulta de gran utilidad a la hora de realizar mediciones y como se verá a continuación, ha permitido la obtención de muchas figuras explicativas.

Para la fabricación del soporte del sistema se han empleado piezas impresas en 3D previamente diseñadas con la herramienta Inventor y componentes comerciales como son tubos de acero o tornillerías.

El objetivo es conseguir una superficie rígida sobre la que montar todos los componentes del sistema: sensor, controlador, baterías, pantallas...; de tal forma que puedan ser transportados y montados en el campo de la manera más rápida y cómoda posible, ya que el tiempo de preparación previo al partido es escaso y se tienen que realizar muchas tareas. Cuanto mayor sea la complicación del sistema mayores son las posibilidades de error durante el montaje.

El conjunto soporte se compone esencialmente de doce componentes un total de dieciséis piezas excluyendo fijaciones. Un total de veintiuna referencias repartidas entre componentes comerciales como la tornillería o la columna central y componentes impresos en 3D como el soporte de la Raspberry Pi entre otros (véase el BOM del proyecto en el apartado de anexos).

Para el diseño de este conjunto se ha tenido principalmente en cuenta la disposición geométrica de la “torre” del campo donde se debe anclar el soporte y la normativa que restringe los límites de tamaño y peso de todo el sistema. Es por ello por lo que lo primero que se realiza es un estudio de la posición de los componentes y las especificaciones de la cámara para determinar la mejor posición y orientación permitidas para el sensor de visión.

Conocidas las características de la cámara es posible realizar la siguiente representación:

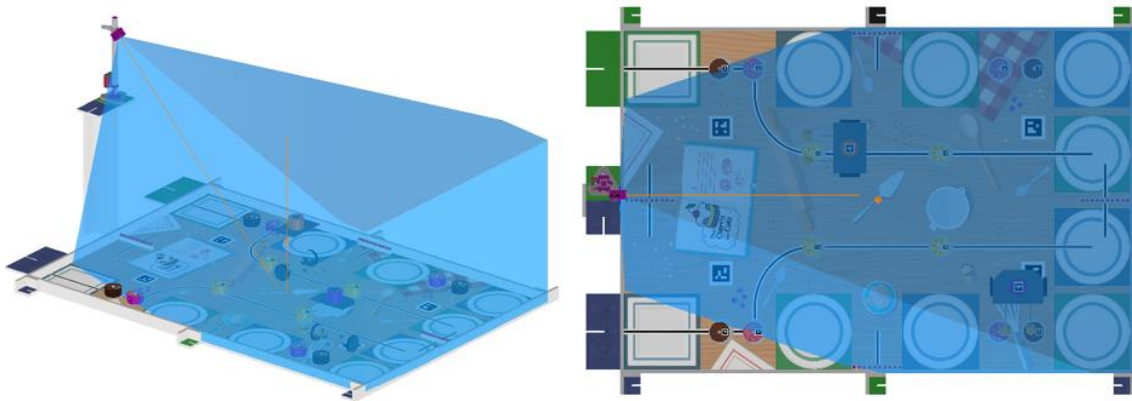


Figura 1.4.5.2 - Representación de los límites de visión del sistema

En la figura se puede apreciar en naranja los ejes que representan el centro del campo y el vector de visión de la cámara. La cámara cuenta con un mayor rango de visión horizontal que vertical, sin embargo, la “torre” se encuentra en uno de los lados menores del campo cuyas dimensiones es 3000x2000mm por lo que, para captar la mayor cantidad de campo posible se coloca la cámara de lado permitiendo que el rango de visión horizontal (el mayor) coincida con el lado mayor del campo.

Aun contando con esta modificación vemos que existen zonas del campo cercanas a la “torre” donde se pierde la visión debido a este rango de visión vertical mucho menor de la cámara, pero se pueden ganar unos centímetros más de visión si se apunta con la cámara un poco más hacia abajo, un ángulo que no aleje mucho el centro de visión del centro del campo pero que permita ganar ese espacio extra. Hay que tener en cuenta de que el plano de visión no se encuentra a la altura del campo, sino a la altura de las balizas montadas en los robots por lo que no es posible bajar la cámara demasiado ya que se perdería la referencia con el lado opuesto del campo.

Para conseguir la mayor cantidad de espacio de visión y minimizar errores de posición debidos a desajustes entre la posición de la cámara y el resto del campo se ha de colocar la cámara a la altura máxima permitida, es decir, a 600mm de altura con respecto a la azotea de la “torre” (1600mm con respecto a la base del campo), pero unos milímetros más abajo contando con que la parte de la estructura que queda por encima del centro del sensor tiene que estar acotada dentro de esos 600mm máximo. De esta forma, el centro del sensor se encontraría exactamente a 1525mm con respecto al campo.

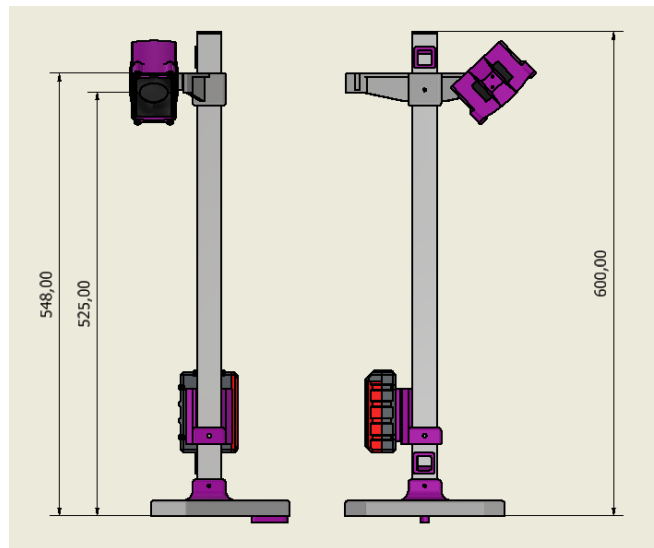


Figura 1.4.5.3 - Plano de dimensiones generales

Algunos aspectos interesantes del diseño dignos de mención podrían ser:

- el doble brazo superior permite el cambio de la posición de la cámara en función del color del equipo seleccionado sin necesidad de realizar ajustes a la estructura
- para que el sistema sea más fácil de montar o desmontar incorpora carriles de cola de milano que permiten la inserción de los componentes más sensibles sin necesidad de tornillos.

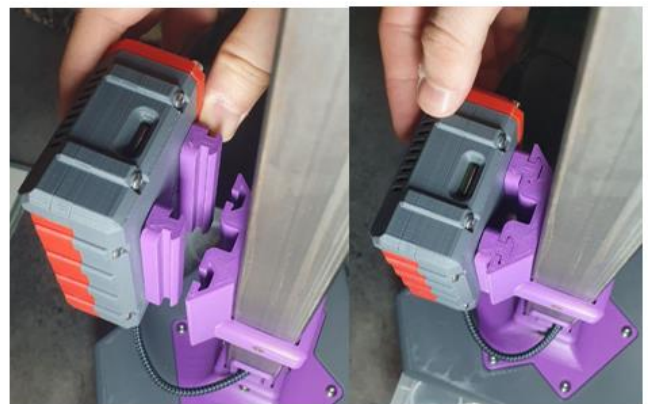
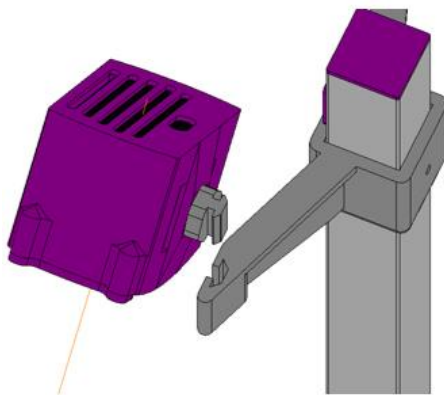


Figura 1.4.5.4 - Diseño de conexiones por carril de cola de milano

- Como se puede apreciar en la imagen anterior, el tubo estructural de acero hace tanto de columna principal como de canaleta pasacables para la cámara.
- Tras fabricar el prototipo se comprueba que su peso total es de 1.36kg por lo que no se supera la restricción de 2kg de la normativa.

Los planos de cada una de las piezas diseñada pueden encontrarse en la sección de planos del apartado de anexos.

1.4.6 Fase de Pruebas

A lo largo del desarrollo del proyecto ha sido precisa la realización de diversas pruebas, tanto de la solución final como de cada uno de los diferentes subsistemas por separado. Para ello ha sido necesaria la preparación del espacio de pruebas y los diferentes objetos de detección que conforman el grueso del proyecto:

Dado que el campo oficial no siempre ha estado disponible para su uso, ha sido necesario habilitar un espacio para las primeras pruebas que cuenta con las dimensiones y principales características de éste, marcando sobre el suelo los diferentes puntos de interés mediante cintas adhesivas.

Tras estudiar los casos de códigos disponibles que marca la normativa de la competición se han fabricado unas tarjetas que incluyen el propio código del marcador Aruco y el contorno de éste en función del objeto en el que va impreso respetando cuidadosamente los tamaños originales.

USO	Códigos	Tamaño Código	Contorno Blanco	Contorno Exterior	Color contorno exterior
Robot equipo Azul	1 a 5	70x70	90x90	100x100	Azul
Robot equipo Verde	6 a 10	70x70	90x90	100x100	Verde
Campo	20 a 23	100x100	140x140	150x150	Gris
Tarta Fresa	47	30x30	40x40	50x50	Rosa
Tarta Vainilla	13	30x30	40x40	50x50	Amarillo
Tarta Chocolate	36	30x30	40x40	50x50	Marrón
Otros usos equipo Azul	51 a 52	70x70	90x90	100x100	Azul
Otros usos equipo Verde	71 a 72	70x70	90x90	100x100	Verde

References	CMYK
RAL 5017 Mat	100% , 60% , 0% , 10%
RAL 6029 Mat	100% , 5% , 90% , 30%
RAL 4010 Mat	15% , 100% , 15% , 10%
RAL 1023 Mat	0% , 25% , 100% , 0%
RAL 8016 Mat	40% , 80% , 70% , 70%
RAL 7032 Mat	15% , 10% , 25% , 20%

Figura 1.4.6.1 - Marcadores y usos

Dado que no siempre se ha contado con robots reales para la realización de las pruebas, se han utilizado balizas con un soporte que permite insertar el código Aruco que se desea probar, con la altura misma altura que tendría en un robot.

Para la realización de las pruebas del sistema se han tenido en cuenta los siguientes parámetros considerados críticos a la hora de trabajar con este tipo de marcadores:

- El brillo y los reflejos afectan de manera significativa al sistema ya que, teniendo en cuenta que los códigos de pruebas fabricados no están hechos con el mismo material usado en la competición (vinilo), bajo un foco de luz muy potente se pierde el contraste entre el blanco y el negro dificultando así el reconocimiento del código.
Para comprobar de manera cualitativa este efecto se utilizará un foco de luz de 10000LM directamente sobre el ArUco.
- La distancia con el Aruco, que afecta en si al tamaño en píxeles del objeto en la imagen, cuanto más lejos o pequeño sea el ArUco a detectar, menos resolución tendrá y por lo tanto es posible que no sea detectado.
Para estudiar este efecto se plantea colocar los ArUcos a diferentes distancias y comprobar los efectos de esta distancia sobre el error de estimación de la posición.

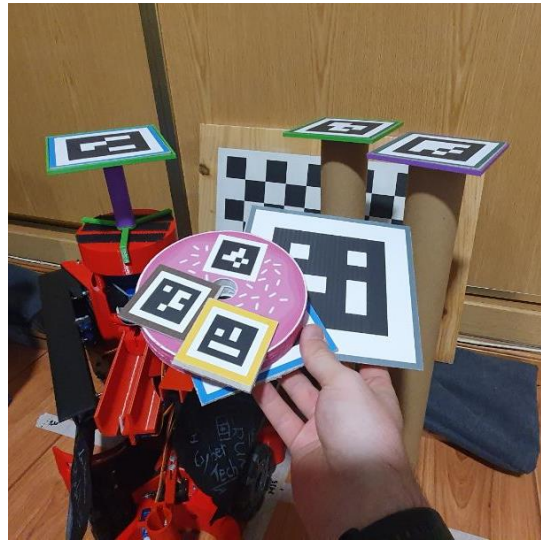


Figura 1.4.6.2 - Marcadores de pruebas

También se han estudiado otros parámetros que resultan de interés a la hora de implementar la solución en un sistema de control de un robot de competición como son la latencia, es decir, el tiempo que tarda el mensaje de la estimación de posición en llegar hasta el receptor, y las oscilaciones de la medida ya que no sólo es importante el valor absoluto del error sino también entre que valores puede variar.

Determinar cuándo un error en la posición es aceptable o no depende de la aplicación ya que, en función de los objetivos y normativa de la competición pueden requerirse sistemas más rápidos, pero menos precisos, como es el caso de la edición 2023 donde los objetos que se deben recoger son discos de 120mm de diámetro, o más lentos pero con menos tolerancia al error como era el caso de la edición 2021 donde los puntos se obtenían por la captura de troncos de cono de 55mm de diámetro, por lo que no se puede valorar de la misma forma un error de 10mm en 120mm que en 55mm. Por lo general, y como criterio tomado dentro del equipo, se puede decir que el error en la posición es aceptable si es menor a 10mm, es decir, se concede una incertidumbre en la estimación de la posición XY de una circunferencia de diámetro 20mm ya que este valor representa un 1% de la longitud menor del campo.

En el caso de la estimación en la orientación se exige mayor precisión, ya que en las traslaciones largas una pequeña desviación de la orientación puede provocar un error de posición muy grande, y aunque el sistema sea capaz de medir este posicionamiento erróneo para corregirlo, siempre es preferible no invertir más tiempo del estrictamente necesario en reajustar la posición. Por lo que en este caso se considerará válida una estimación con una precisión de $\pm 1^\circ$.

Una vez se comienza a realizar las primeras pruebas de detección de los marcadores en la estructura de pruebas se descubre que la detección de los códigos pertenecientes a los objetos repartidos por el espacio de juego no se puede realizar con fiabilidad, dado el tamaño de estos marcadores no permite su identificación con la cámara seleccionada. Se puede deducir que el objetivo de estos marcadores no es el de detección desde la “torre” principal del campo, sino desde un sistema de visión montado en el robot, ya que, a menos que estos marcadores se encuentren a una distancia inferior a 1000mm con respecto a la base del soporte de la cámara, no se obtienen resultados de su posición y, por lo tanto, no es posible desarrollar una estimación de la puntuación dinámica mediante la detección de los ArUcos de esta forma.



Figura 1.4.6.3 - Calidad de detección de objetos

Para añadir más información a este aspecto se han realizado pruebas de detección de los diferentes arucos en distintas posiciones del campo para poder hallar los límites de detección de estos tal y como se muestra en el siguiente gráfico:

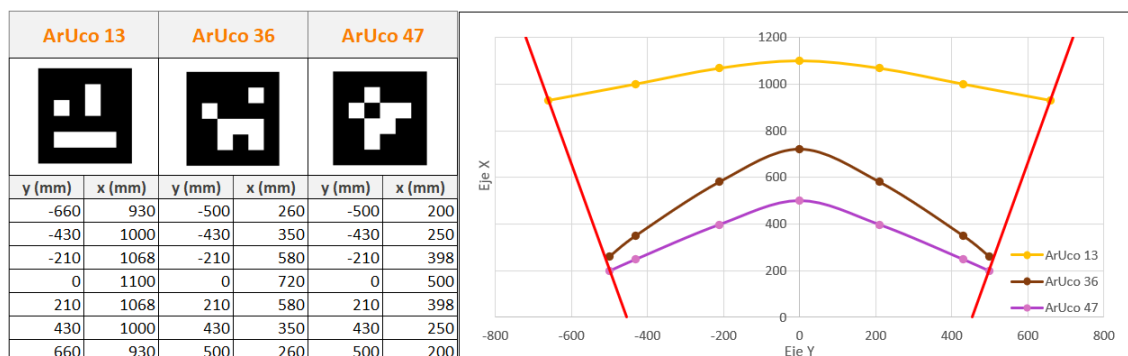


Figura 1.4.6.4 - Rango de detección de los ArUcos

En este gráfico se muestra como el marcador 13 tiene un rango de detección que llega hasta los 1100mm, mucho mayor que los otros dos marcadores, 36 y 47 con 720 y 500mm respectivamente, probablemente esto se deba a que el código 13 tiene un patrón más sencillo que los otros dos, por lo que es más fácil de detectar

Como se puede ver, la detección de estos códigos en las zonas de obtención de puntos no es posible utilizando un sistema de visión desde la “torre”. Sin embargo, para que este objetivo del proyecto no quede del todo incompleto se plantea la siguiente alternativa: Se plantea un sistema de visión muy similar al ya desarrollado para la “torre”, pero en este caso, iría montada dentro de un robot. EL robot es el encargado de transportar los objetos desde

distintas zonas del campo hasta las zonas de obtención de puntos y por ello es capaz de tener una mejor vista de estos una vez que los ha dejado.

De esta forma es posible implementar un programa que tome una foto justo cuando el robot ha realizado la acción “dejar objeto” y analizar esta imagen para detectar si se han conseguido marcar esos puntos.

Para ello hay que tener en cuenta que la manera de marcar puntos es a través de unos objetos cilíndricos (discos) que simulan pisos de un pastel, si el pastel tiene tres pisos de tres colores se marcan más puntos que de solo tener dos. Por lo tanto, una vez detectada en la imagen un ArUco correspondiente a una de estas plantas del pastel se recorta una sección de la imagen que contenga solamente una fracción de cada piso del pastel y se analiza haciendo una segregación por color del número de pisos que tiene el pastel como se muestra en la siguiente figura.



Figura 1.4.6.5 - Programa de cálculo de puntos dinámico

- **Pruebas en el campo real**

Finalmente se realizan las pruebas del objetivo fundamental del proyecto, la estimación de la posición del robot del equipo y los robots enemigos cercanos.

Para ello se hace uso de un campo oficial y uno de los robots diseñados para esta competición de tal forma que se pueda observar el funcionamiento en la situación más realista posible.

Se dispone en el campo el robot del equipo y dos balizas de robots enemigos y se ejecuta el código de visión junto con el nodo de ROS2. Es posible acceder al topic sobre el que se ha escrito el valor de posición del robot a través de otra máquina remota conectada a la misma red mediante el uso de la función “echo” y siempre y cuando la variable de entorno “ID” de las dos máquinas (publisher y receiver) estén configurados al mismo valor.

```
Export ROS_DOMAIN_ID = 50
ros2 topic -echo /outputdata_topic
```

De esta forma se analiza primero de manera cualitativa como el programa de visión rastrea la posición del robot correctamente mediante un “visualizador” escrito con “matplotlib”:

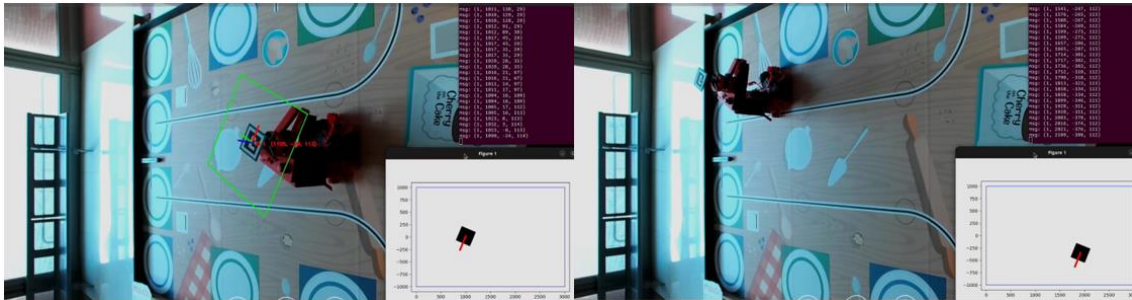


Figura 1.4.6.6 - Prueba sobre el robot en campo

De esta manera se determina que, visualmente el sistema de localización sigue correctamente los movimientos realizados por el robot en esta rutina preprogramada pero cuando el marcado a bordo del robot sale del campo por el frontal de éste, el reflejo de la luz en el suelo provoca la pérdida de la detección de este ArUco, esto reafirma la hipótesis de que el espacio debe estar muy controlado para que el sistema funcione de manera adecuada.

A partir de aquí se realizan las pruebas de posicionamiento mediante el uso de las balizas y el robot colocándolo en nueve puntos del campo y comprobando la posición con ayuda de un flexómetro:

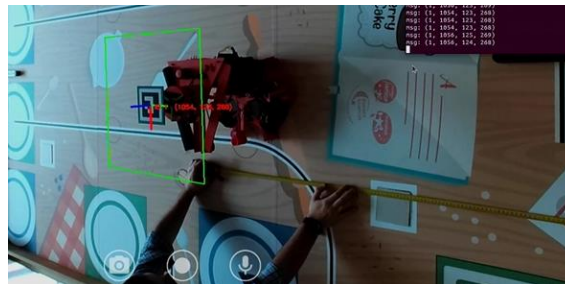


Figura 1.4.6.7 - Comprobación de las medidas del sistema

Es importante señalar que, en el caso de los puntos más alejados, debido a los reflejos del suelo el Aruco dejaba de detectarse y ha sido preciso utilizar una pantalla para minimizar el efecto de la luz y mejorar el contraste, pero aun así hay momentos en los que se pierde la detección. Por ello, los resultados deben estudiarse sabiendo que el efecto de la luz está afectando significativamente a esos puntos en la estimación de la posición, pero realmente durante la competición no está bajo el control de los equipos la distribución de puntos de luz alrededor del campo ni como pueda reflejarse esta en algún objeto.

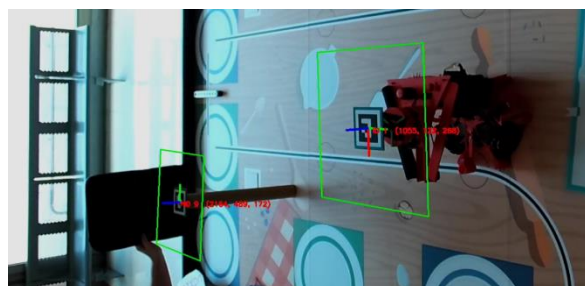
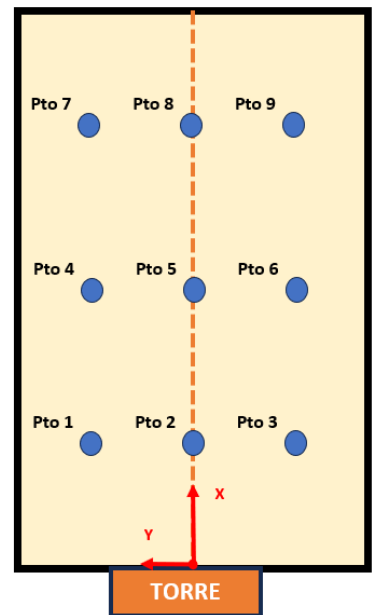


Figura 1.4.6.8 - Errores de detección

	Posición Real			Posición detectada					
	X [mm]	Y [mm]	W [grados]	X Mayor [mm]	X Menor [mm]	Y Mayor [mm]	Y Menor [mm]	W Mayor [grados]	W Menor [grados]
Pto 1	575	430	225	576	575	431	428	225	225
Pto 2	575	0	180	575	574	0	-1	180	180
Pto 3	575	-430	135	579	576	-427	-431	136	135
Pto 4	1500	430	270	1505	1503	433	432	270	270
Pto 5	1500	0	0	1502	1500	-1	-1	0	0
Pto 6	1500	-430	90	1505	1504	-423	-434	90	89
Pto 7	2425	430	315	2433	2429	417	437	318	315
Pto 8	2425	0	0	2430	2427	8	7	3	1
Pto 9	2425	-430	45	2440	2437	-413	-420	49	47

	Error X		Error Y		Error W Mayor	
	Mayor [mm]	Menor [mm]	Mayor [mm]	Menor [mm]	[grados]	[grados]
Pto 1	-1	0	-1	2	0	0
Pto 2	0	1	0	1	0	0
Pto 3	-4	-1	-3	1	-1	0
Pto 4	-5	-3	-3	-2	0	0
Pto 5	-2	0	1	1	0	0
Pto 6	-5	-4	-7	4	0	1
Pto 7	-8	-4	13	-7	-3	0
Pto 8	-5	-2	-8	-7	-3	-1
Pto 9	-15	-12	-17	-10	-4	-2



Como se puede ver en los resultados, los puntos más alejados de la cámara se ven afectados por los reflejos de la luz ambiental en el suelo y provocan una estimación imprecisa e inestable. Sin embargo, el resto de los puntos dentro de la zona de detección recibe buenos resultados no superándose en ningún caso los límites de error establecidos anteriormente.

Se establece así el siguiente mapa donde se puede entender el reparto de áreas del campo:

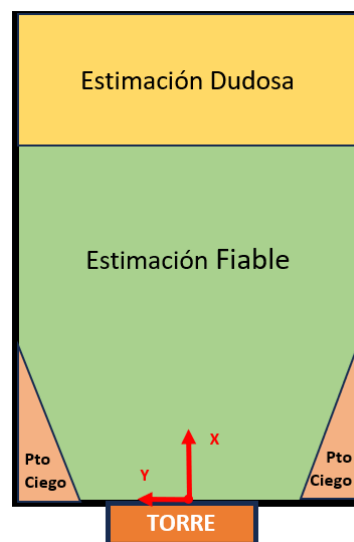


Figura 1.4.6.10 - Mapa de resultados

Se realizan pruebas con el foco de luz externo, pero no se aprecian variaciones significativas de la estimación de la posición en la medida. Al haber ajustado el valor de exposición y brillo de la cámara para que la luz ambiente no afectase tanto a los puntos más alejados la luz emitida por este foco apenas tiene efecto.

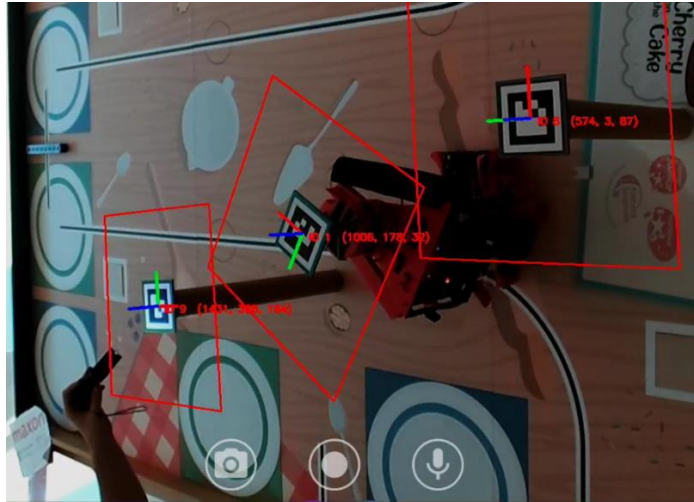


Figura 1.4.6.11 - Pruebas con focos de luz directa

Finalmente, para estimar el lapso de tiempo desde que un dato es estimado hasta que es recibido por la máquina remota se tapa un código y se pone un cronómetro en el momento en el que se destapa, y el mensaje es recibido por la máquina remota a los 1,8 segundos.

Con estos resultados se puede concluir que, siempre y cuando el entorno esté controlado, las estimaciones de posición y orientación van a tener buenos resultados. No obstante, existen puntos ciegos y áreas donde disminuye la precisión del sistema y, cuando el robot se encuentra en estas áreas, es mejor tomar datos de otros sensores.

1.5 Conclusiones

En conclusión formal del proyecto, pese a que no ha sido posible resolver todos los objetivos propuestos de la manera estimada, se han desarrollado alternativas con las que poder cumplir con todos los puntos establecidos en el anteproyecto de este trabajo resolviendo las limitaciones técnicas de la solución inicial, con ello, también se han cumplido con los principales objetivos del proyecto en un marco controlado como es el de la competición Eurobot, esta vez sí, aplicando las soluciones planificadas inicialmente, es decir, el sistema puede fallar bajo ciertas circunstancias o bajo la influencia de factores que se escapan del control como es el visto anteriormente en los resultados del reflejo en el suelo, por ello, es necesario contar con una figura experta que comprenda las limitaciones del sistema y sea capaz de identificar cuando pueden surgir errores durante la competición para poder actuar a tiempo. Se ha demostrado que es posible realizar una estimación precisa de la posición y orientación del robot en el campo por medio de un sistema independiente formado por una cámara y un procesador que son capaces de publicar esa información en un topic de ROS2 para que el propio robot del equipo pueda utilizar esa información como realimentación de los movimientos que realiza.

Es importante destacar que se ha desarrollado una solución homologable ya que cumple con las restricciones impuestas por la normativa de tal forma que podrá introducirse en futuras competiciones a falta de pequeñas modificaciones requeridas por cada edición concreta.

Este proyecto ha estado marcado por la obtención de nuevas habilidades dentro del ámbito de la robótica en la línea del desarrollo de aplicaciones software que se establecen a nivel de grupo como “programación a alto nivel” o “inteligencia”, permitiéndome a día de hoy, haber pasado por todos los grupos de trabajo que componen el equipo de robótica (mecánica, electrónica, control, inteligencia y gestión).

Durante el proyecto se han tenido que resolver diversas complicaciones como la que ha supuesto la escasez de componentes electrónicos, razón por la cual se decide mantener el mismo elemento aun encontrándose defectuoso y buscando alternativas en el sistema de visión para no tener que acceder a reemplazar la pieza más costosa del sistema; o todas las complicaciones que supone la puesta en marcha por primera vez de un sistema operativo en última versión y el entorno de ROS2 entre otros.

Gracias a este proyecto se ha podido demostrar de manera práctica distintos conocimientos obtenidos durante los estudios de máster y sobre todo de la especialización en robótica y percepción como son el tratamiento de imágenes y la utilización de cámaras para desarrollar sistemas de visión o la utilización de herramientas del entorno de ROS.

Aun con todo y con ello, el desarrollo de este proyecto no finaliza aquí dado que todavía faltan muchas acciones de mejora por explorar como pueden ser las siguientes:

- Simplificar la utilización del sistema realizando una calibración automática de los parámetros de corrección del error de posicionamientos utilizando por ejemplo los ArUcos impresos en el campo (aunque no es seguro que estos vayan a estar disponibles en futuras ediciones).
- Mejorar la corrección del error de posicionamiento mediante mecanismos de corrección de la perspectiva de la imagen obtenida dado que el plano de la imagen no coincide con

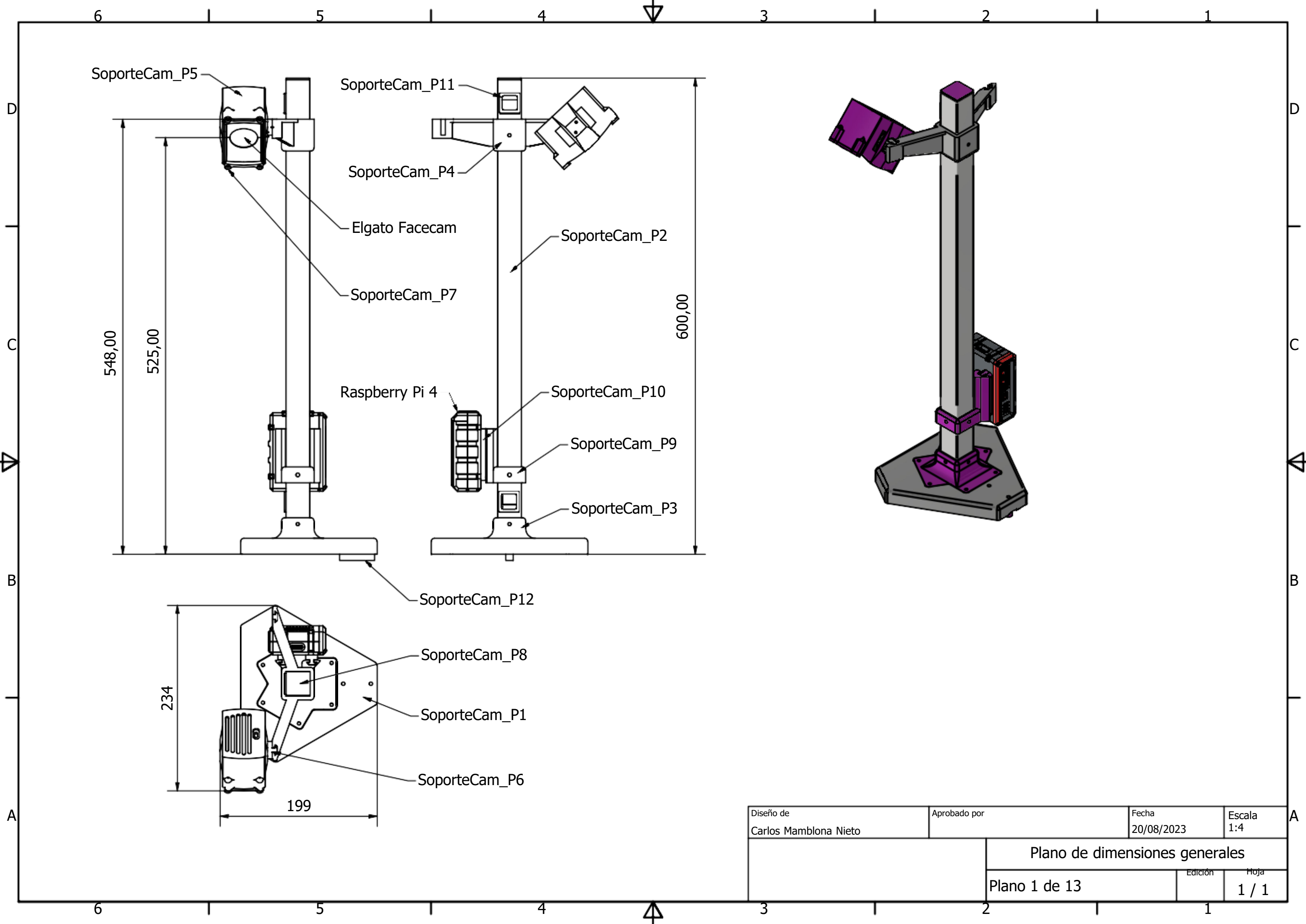
el plano de los ArUcos, teniendo en cuenta que estas mejoras no deben afectar significativamente al rendimiento.

- Desarrollar nuevas aplicaciones que aporten más información al sistema como por ejemplo la velocidad lineal de los robots en el campo ya que no entraña el mismo riesgo un robot orientado hacia otro pero estático que si este se desplaza a una determinada velocidad.

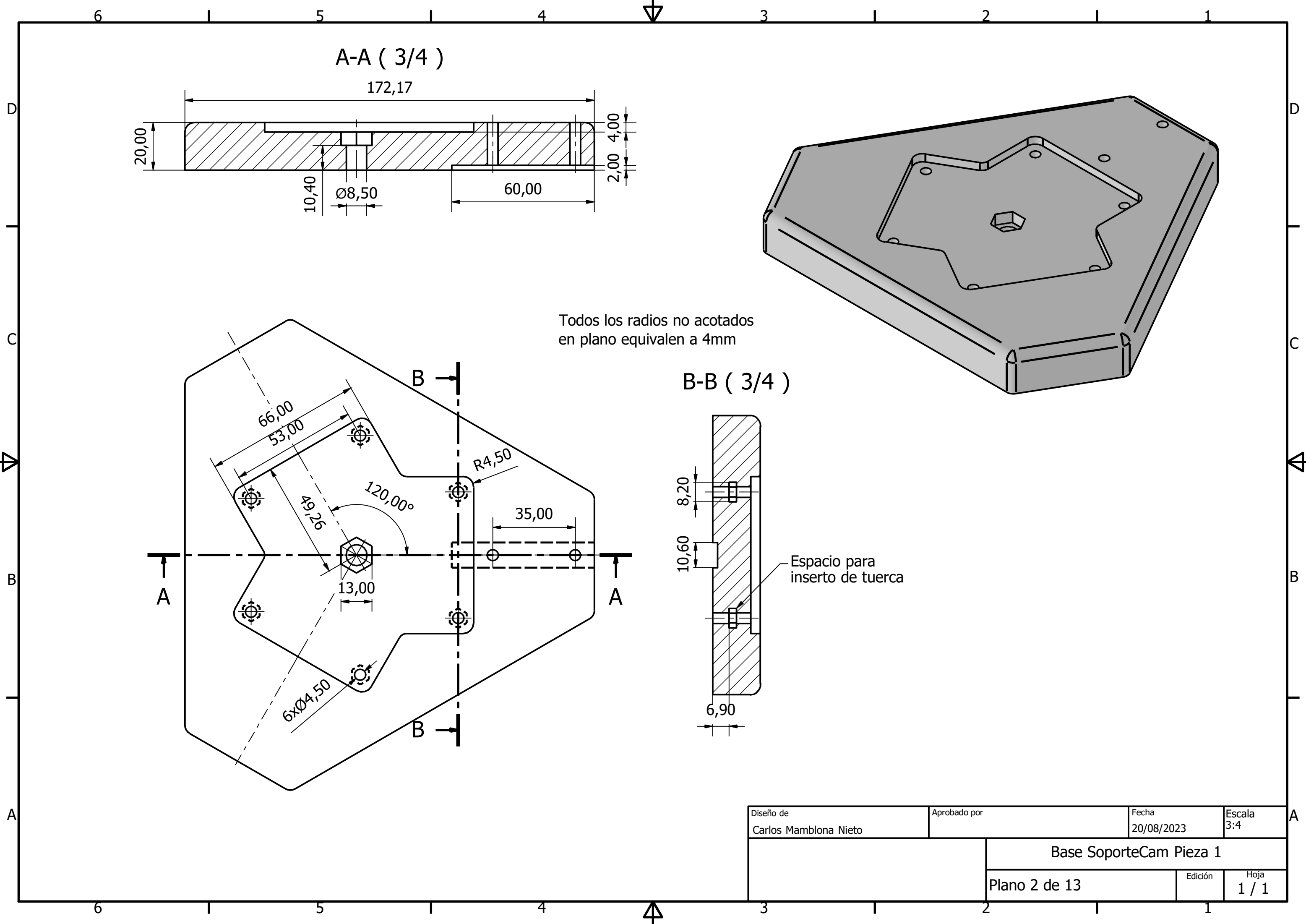
2. Planos y Diagramas

2.1 Planos técnicos del sistema

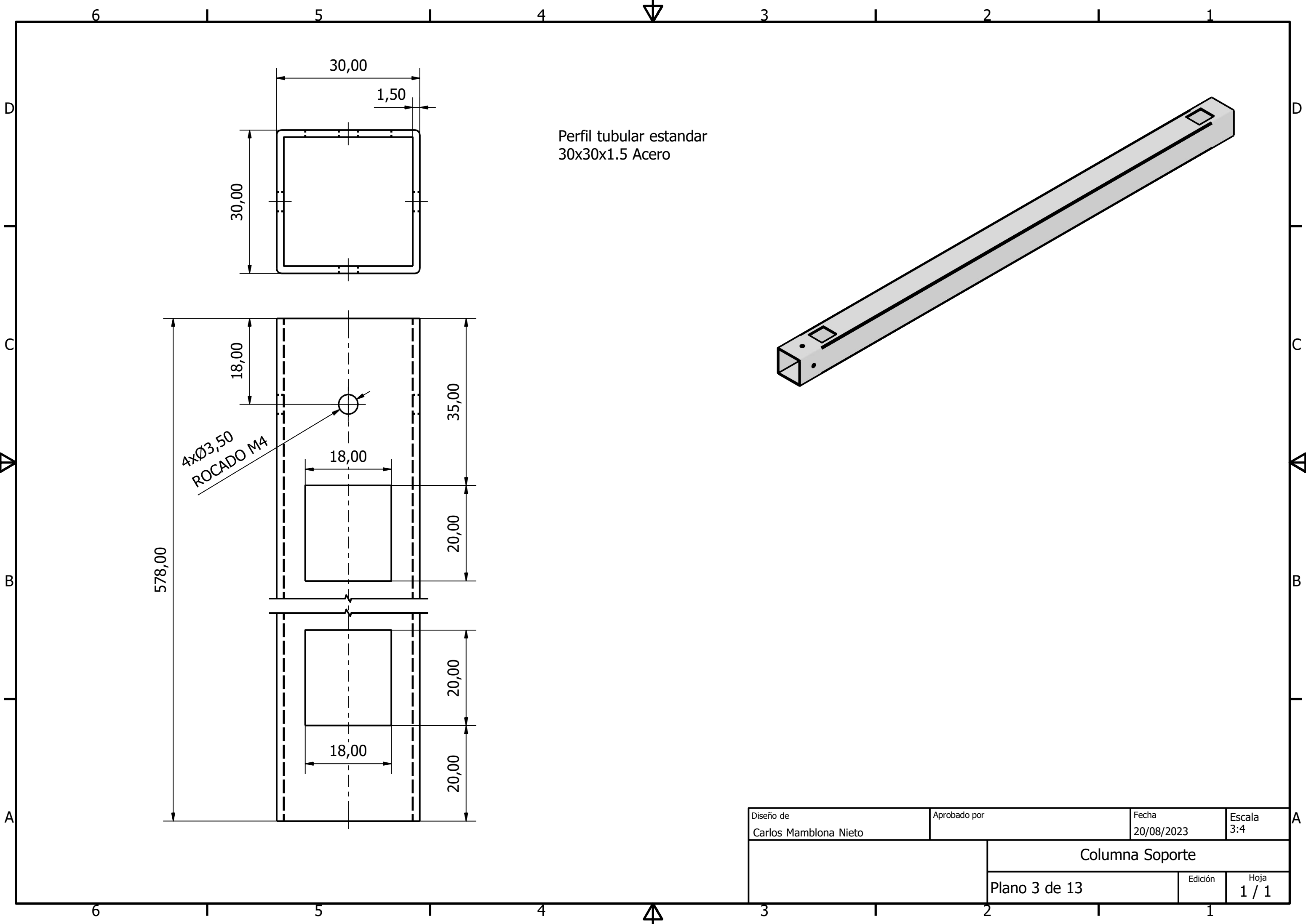
Referencia	Descripción	Tipo	Nº Plano
SoporteCam_P1	Base SoporteCam Pieza 1	Impresión 3D	2
SoporteCam_P2	Columna Soporte	Comercial	3
SoporteCam_P3	Base SoporteCam Pieza 2	Impresión 3D	4
SoporteCam_P4	Barazo SoporteCam	Impresión 3D	5
SoporteCam_P5	Carcasa FaceCam	Impresión 3D	6
SoporteCam_P6	Unión cámara soporte	Impresión 3D	13
SoporteCam_P7	Pestaña de cierre carcasa FaceCam	Impresión 3D	7
SoporteCam_P8	Marco de protección de cantos Tipo 1	Impresión 3D	8
SoporteCam_P9	Soporte Raspberry Pi Pieza 1	Impresión 3D	9
SoporteCam_P10	Soporte Raspberry Pi Pieza 2	Impresión 3D	10
SoporteCam_P11	Marco de protección de cantos Tipo 2	Impresión 3D	11
SoporteCam_P12	Chaveta de alineación	Impresión 3D	12



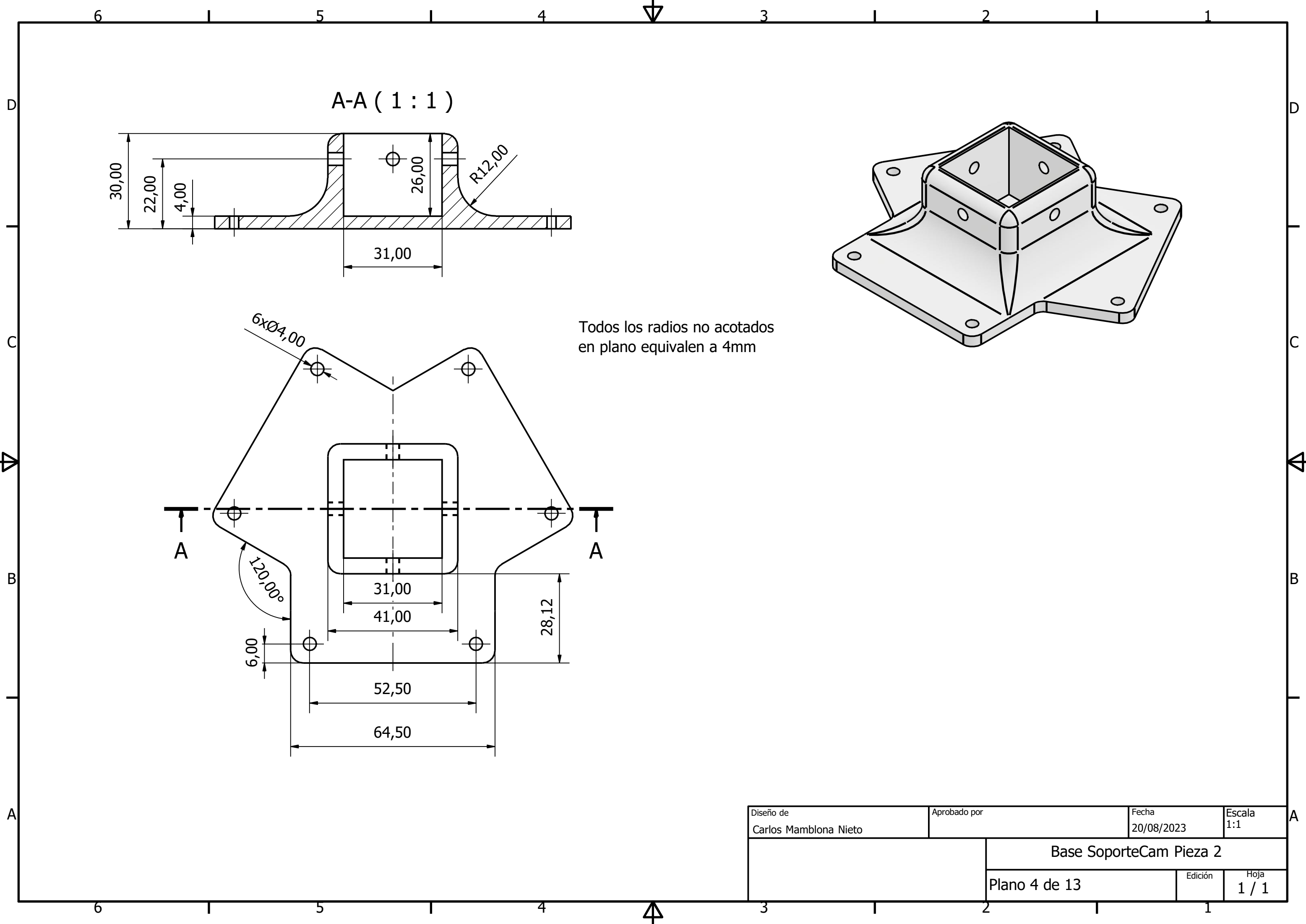
Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 1:4
		Plano de dimensiones generales	
		Edición Plano 1 de 13	Hoja 1 / 1



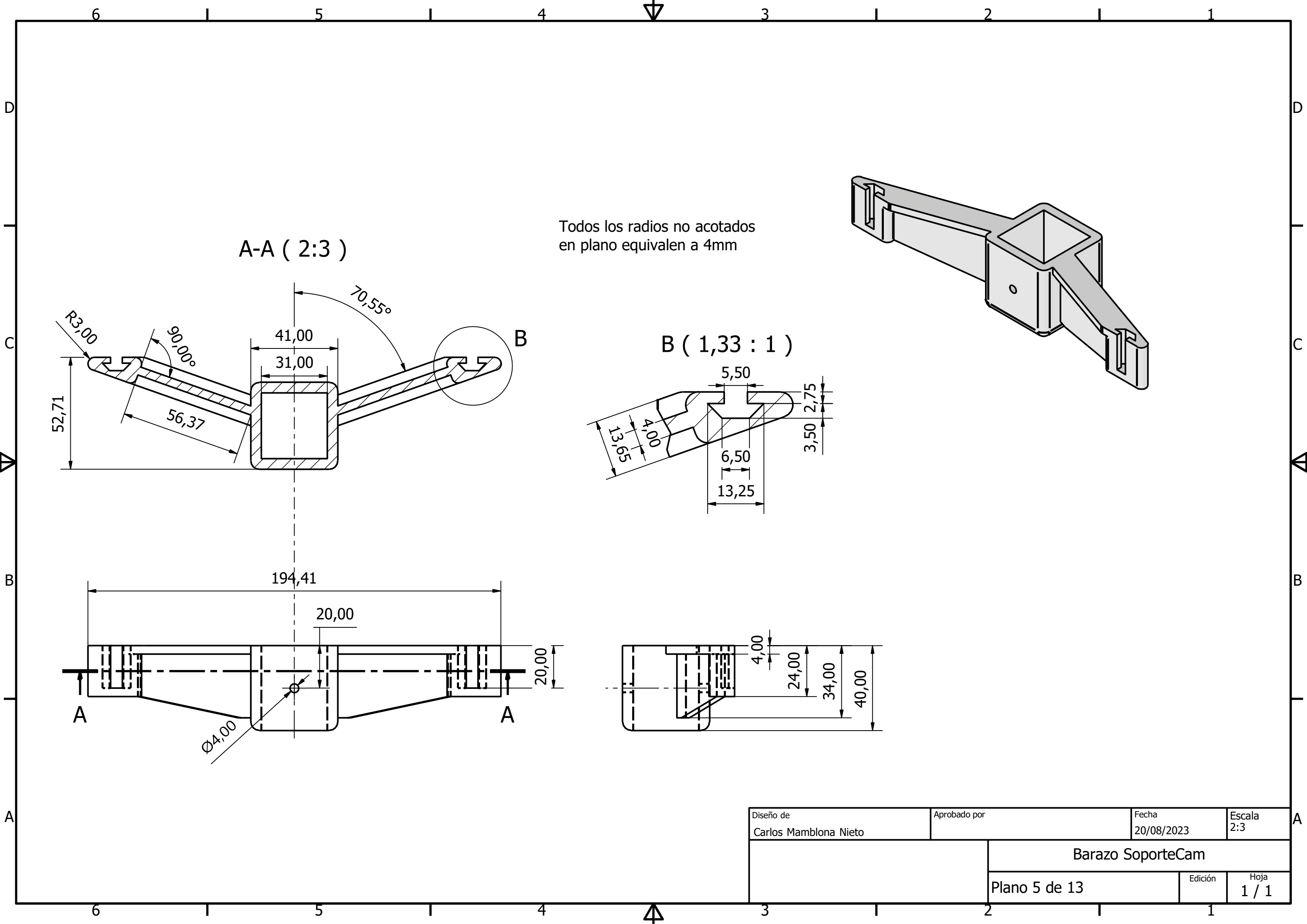
Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 3:4
		Base SoporteCam Pieza 1	
		Plano 2 de 13	Edición Hoja 1 / 1



Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 3:4
		Columna Soporte	
		Plano 3 de 13	Edición Hoja 1 / 1

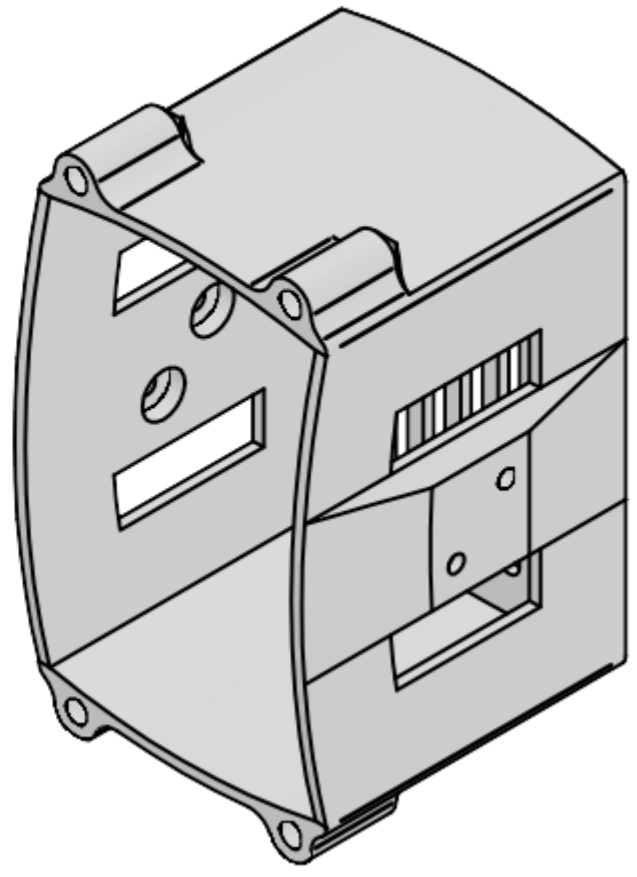
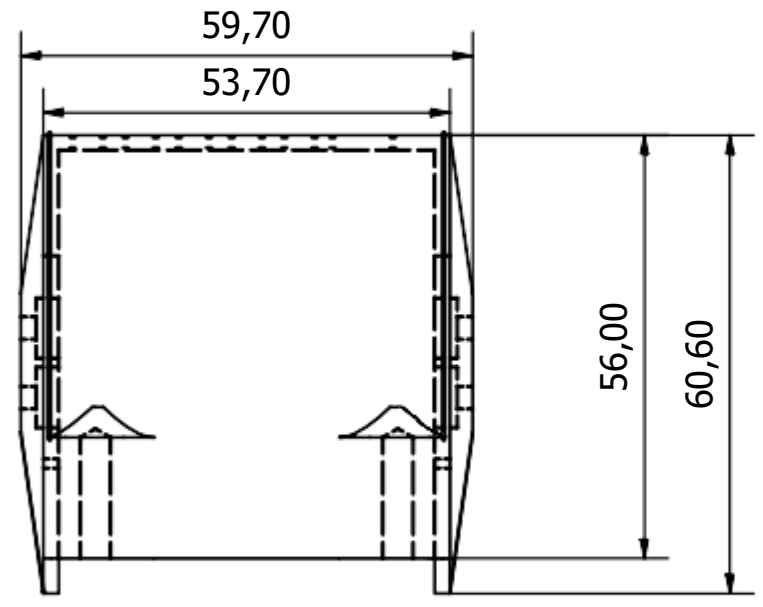
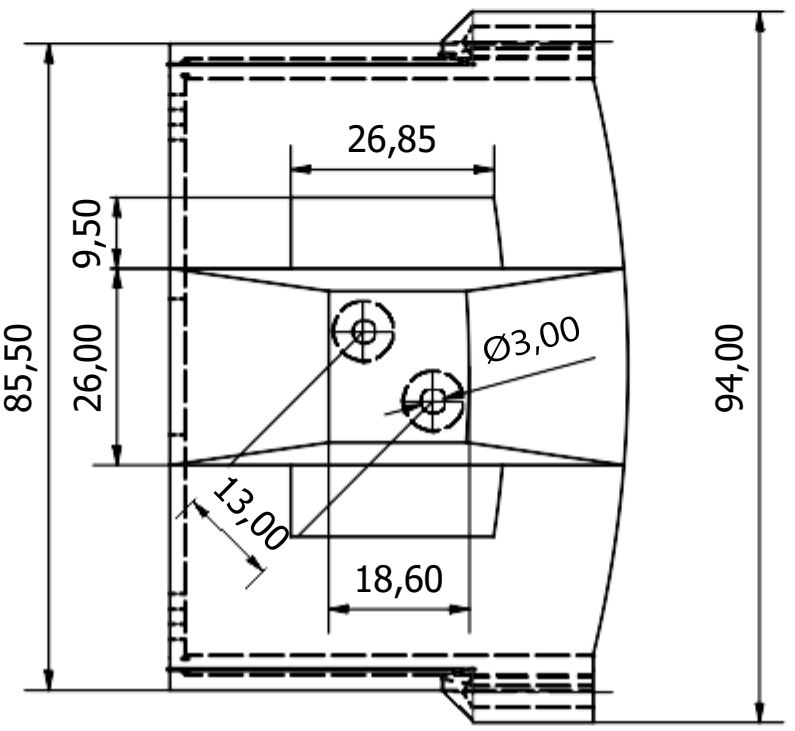
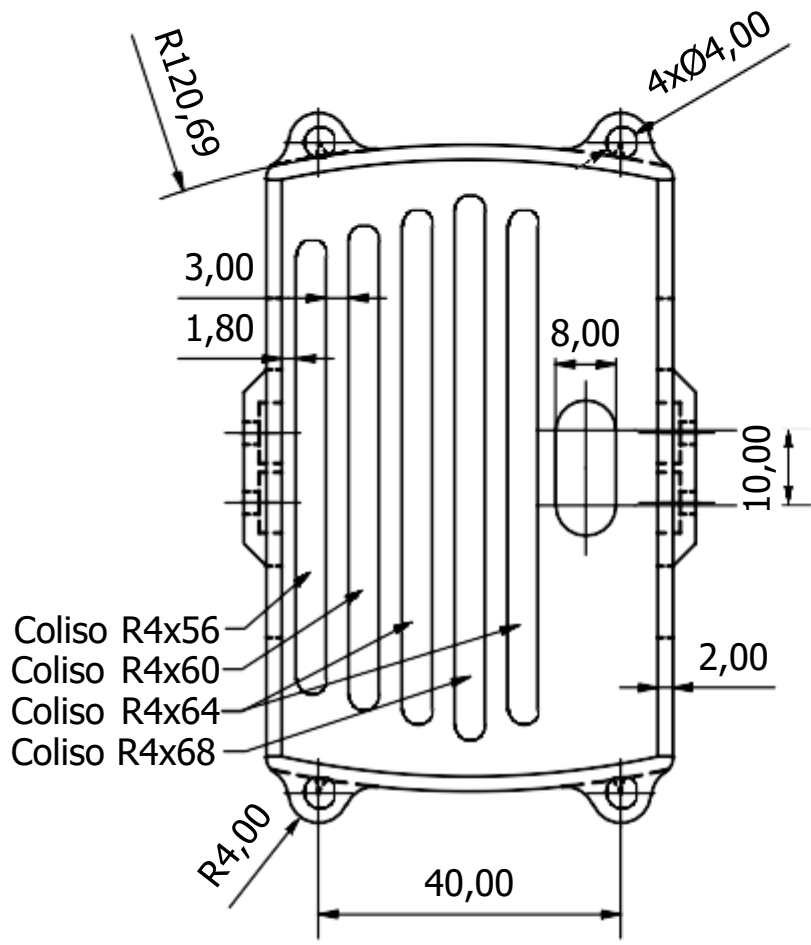


Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 1:1
		Base SoporteCam Pieza 2	
		Plano 4 de 13	Edición Hoja 1 / 1

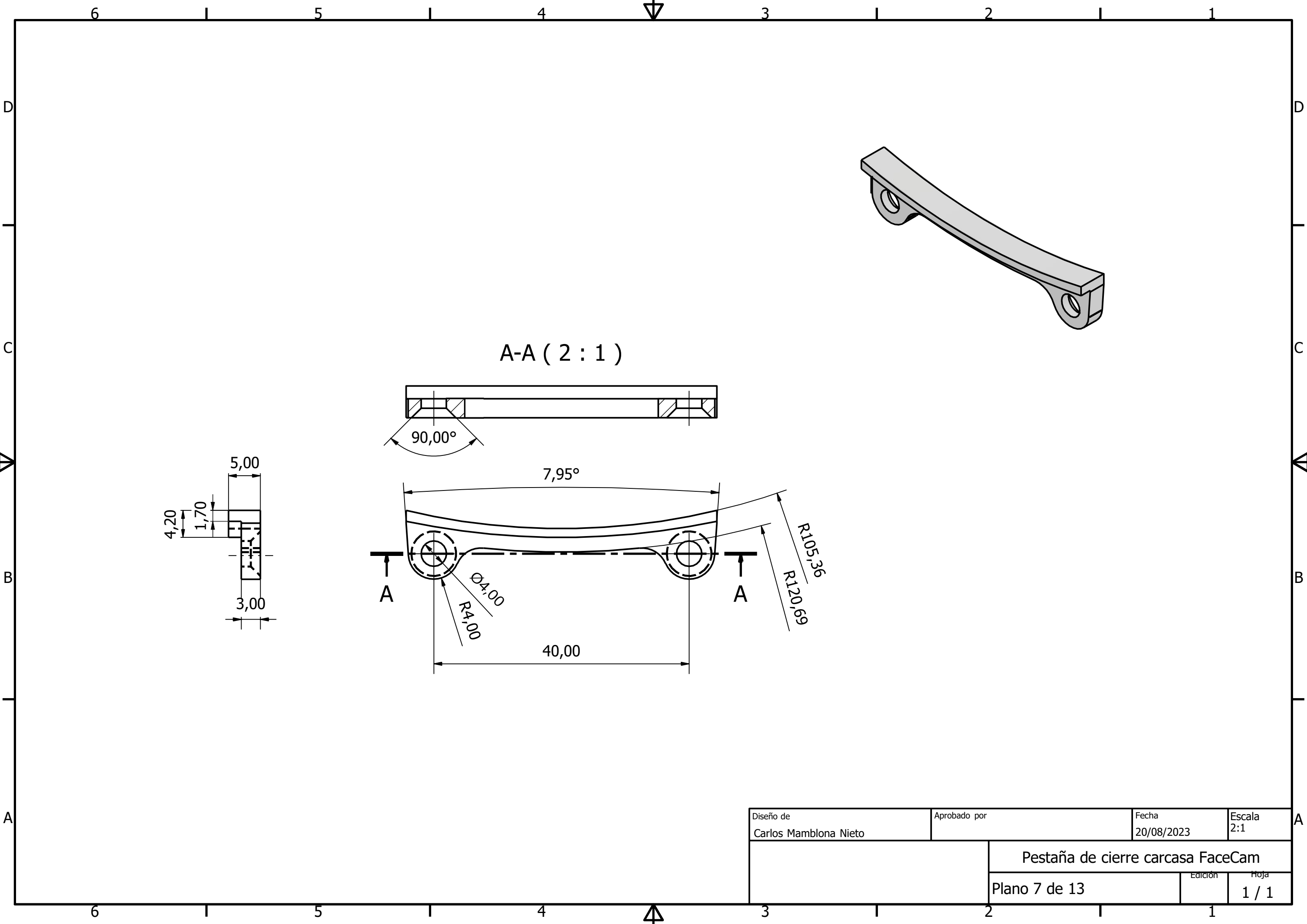


Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 2:3
		Barazo SoporteCam	
		Plano 5 de 13	Edición Hoja 1 / 1

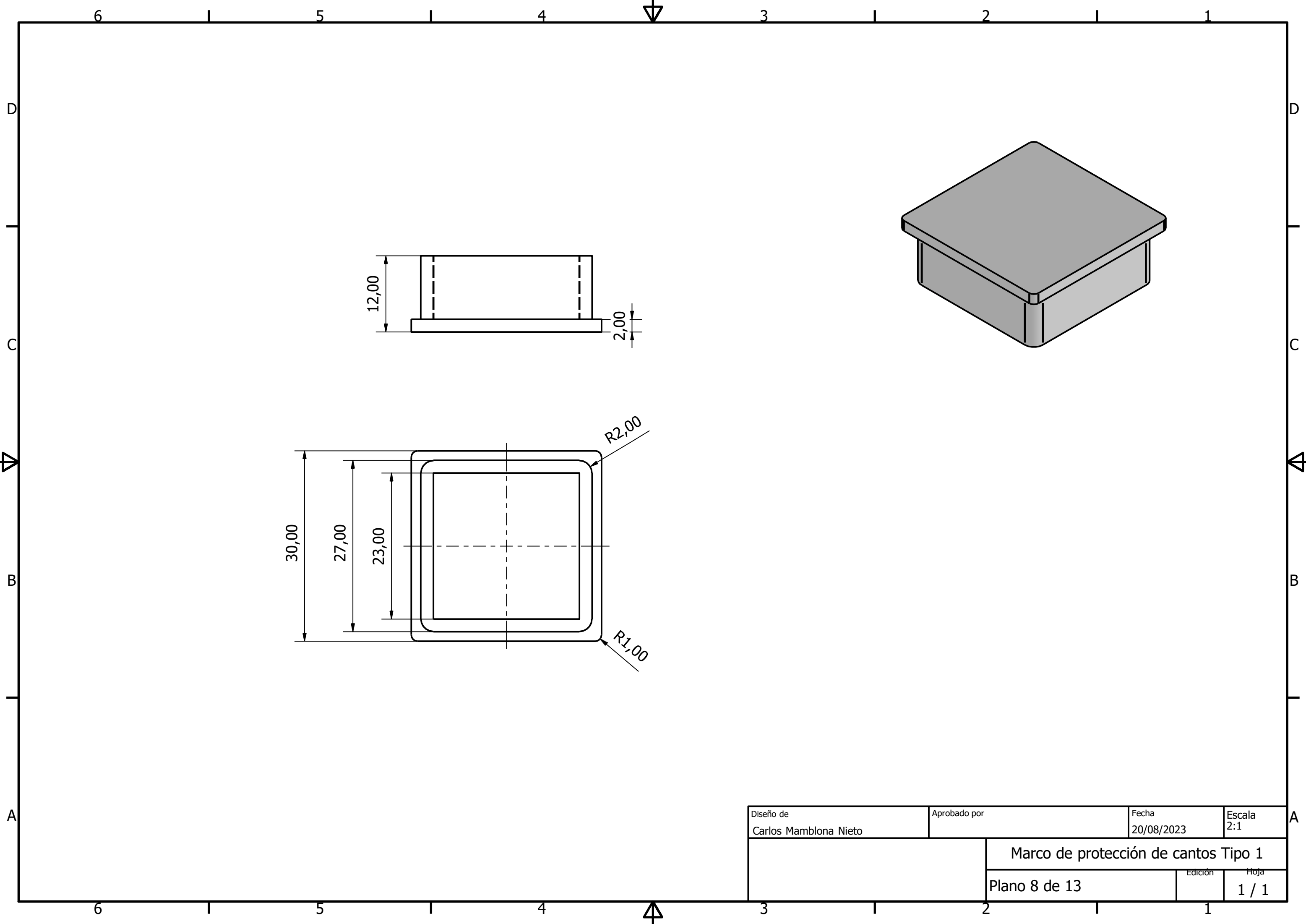
Diseño basado en la geometría del artículo:
Elgato Facecam



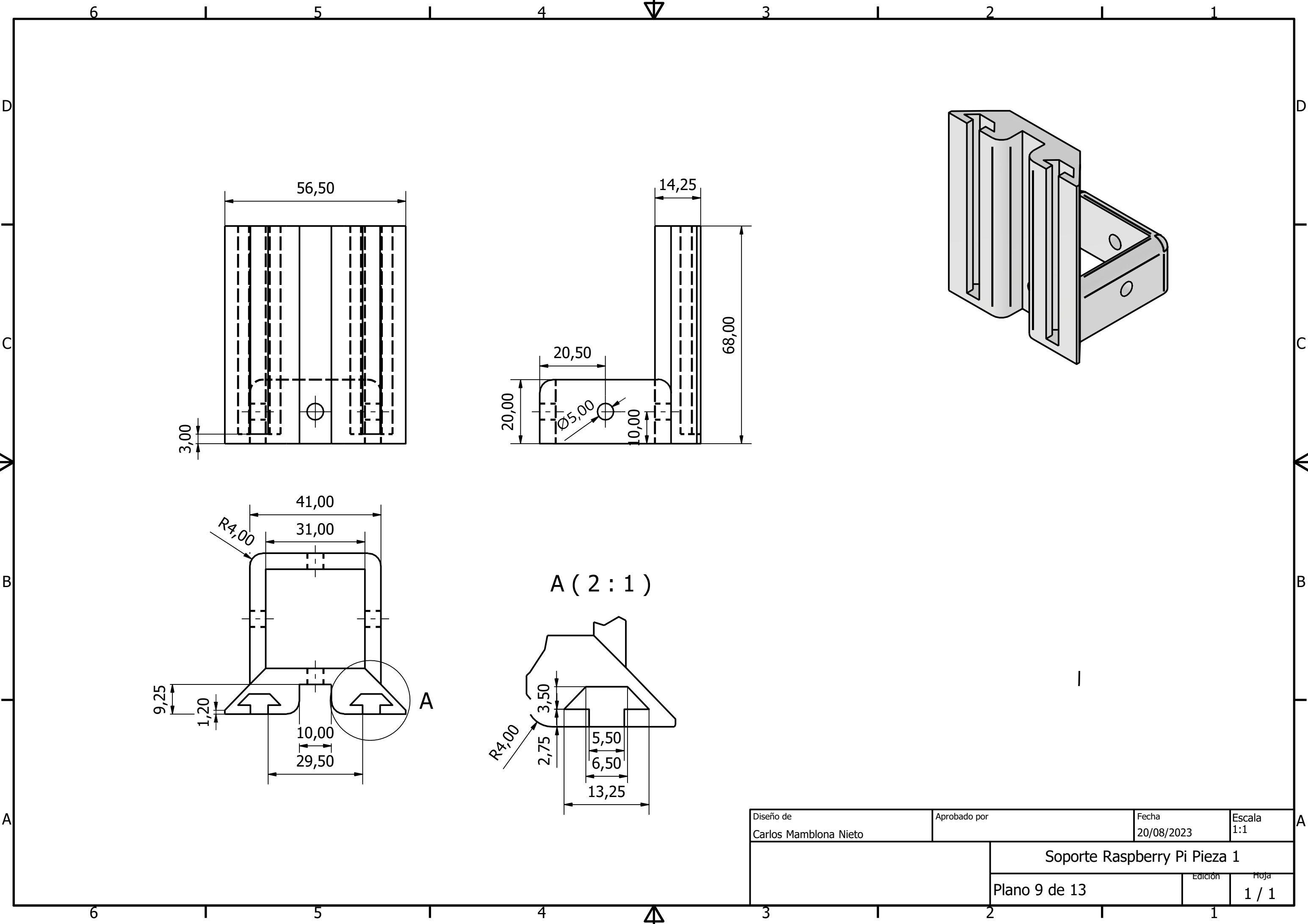
Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 1:1
Carcasa FaceCam			
Plano 6 de 13		Edición	Hoja 1 / 1



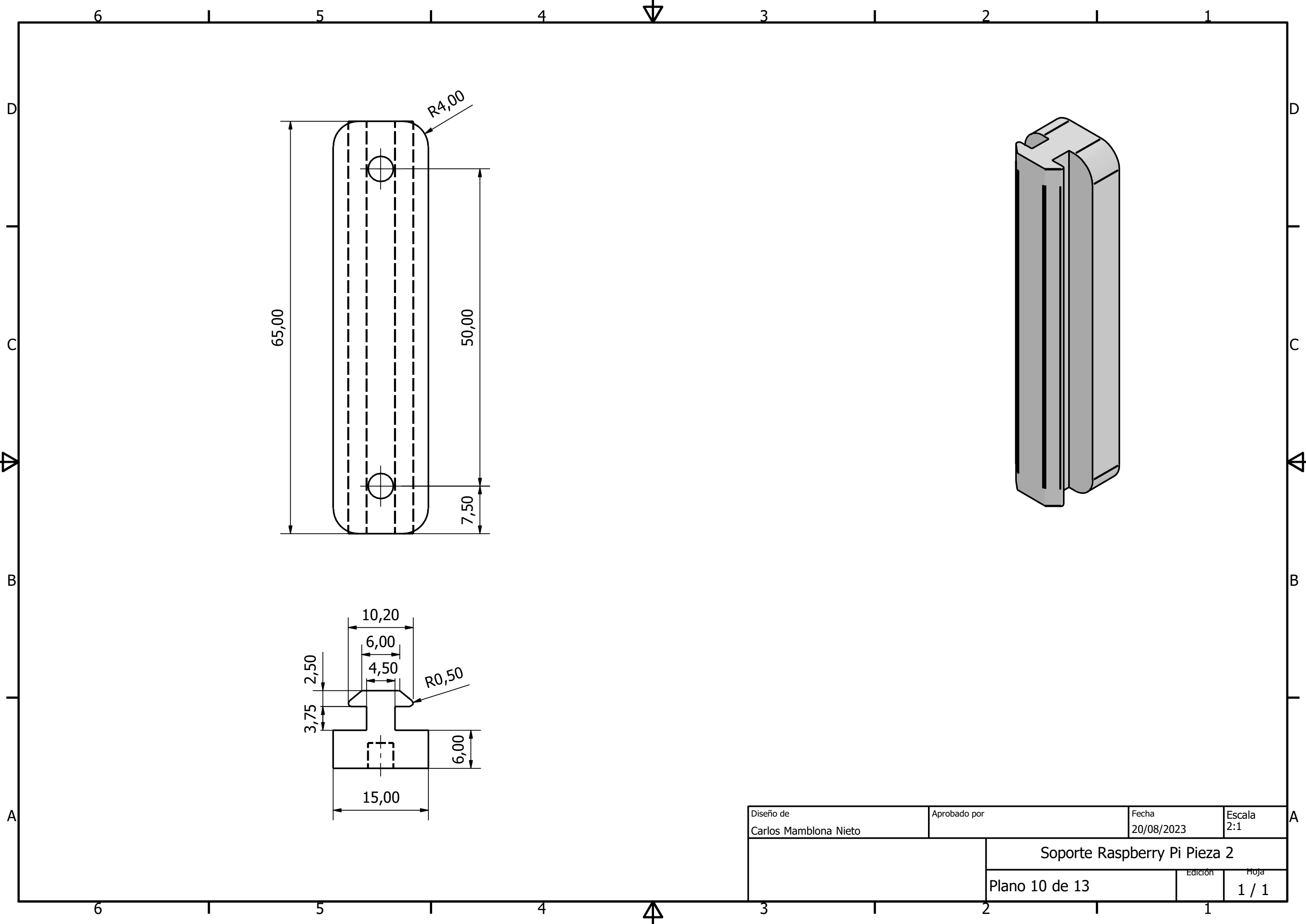
Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 2:1
		Pestaña de cierre carcasa FaceCam	
		Edición Plano 7 de 13	Hoja 1 / 1



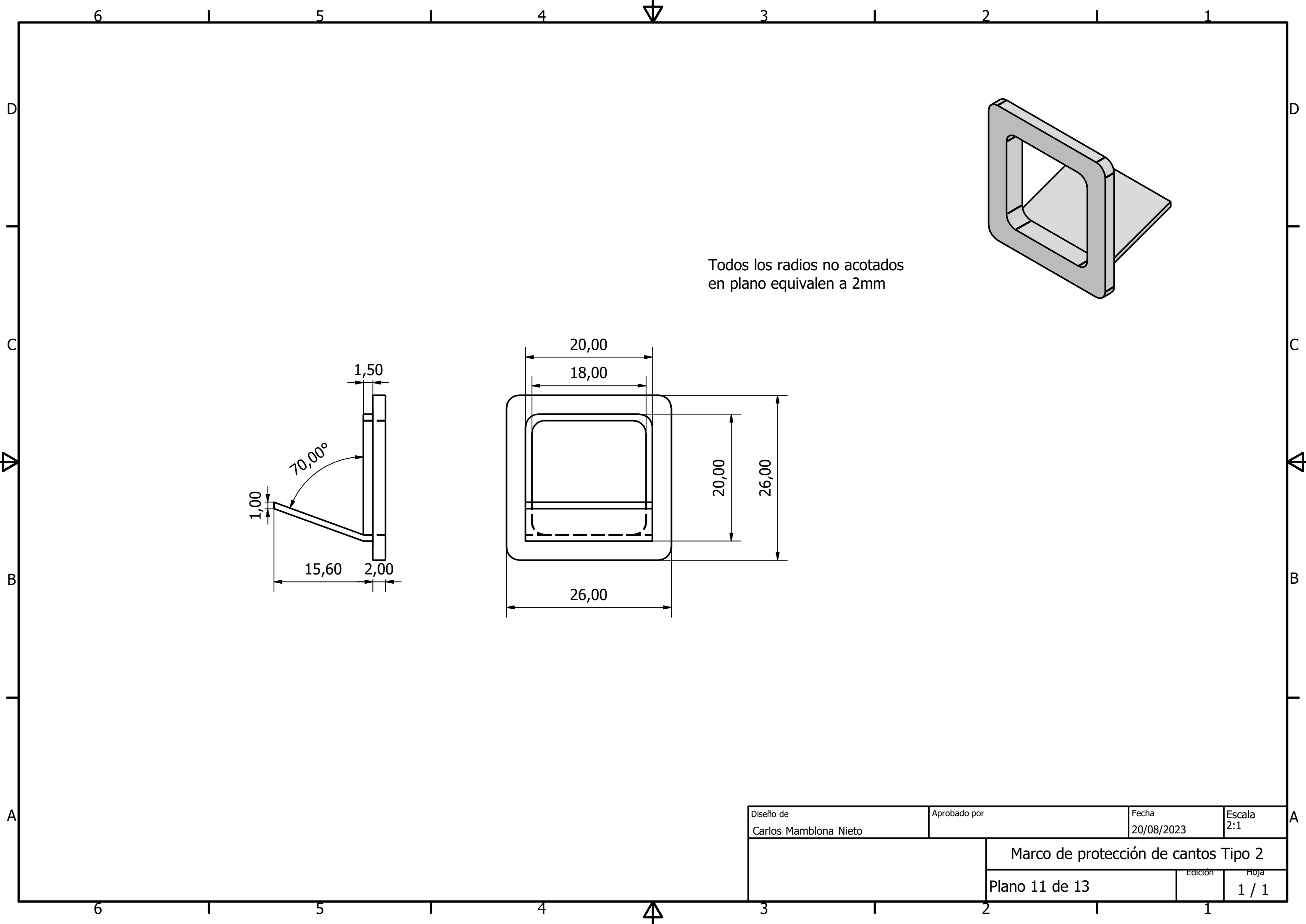
Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 2:1
		Marco de protección de cantos Tipo 1	
		Edición Plano 8 de 13	Hoja 1 / 1



Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 1:1
		Soporte Raspberry Pi Pieza 1	
		Edición Plano 9 de 13	Hoja 1 / 1

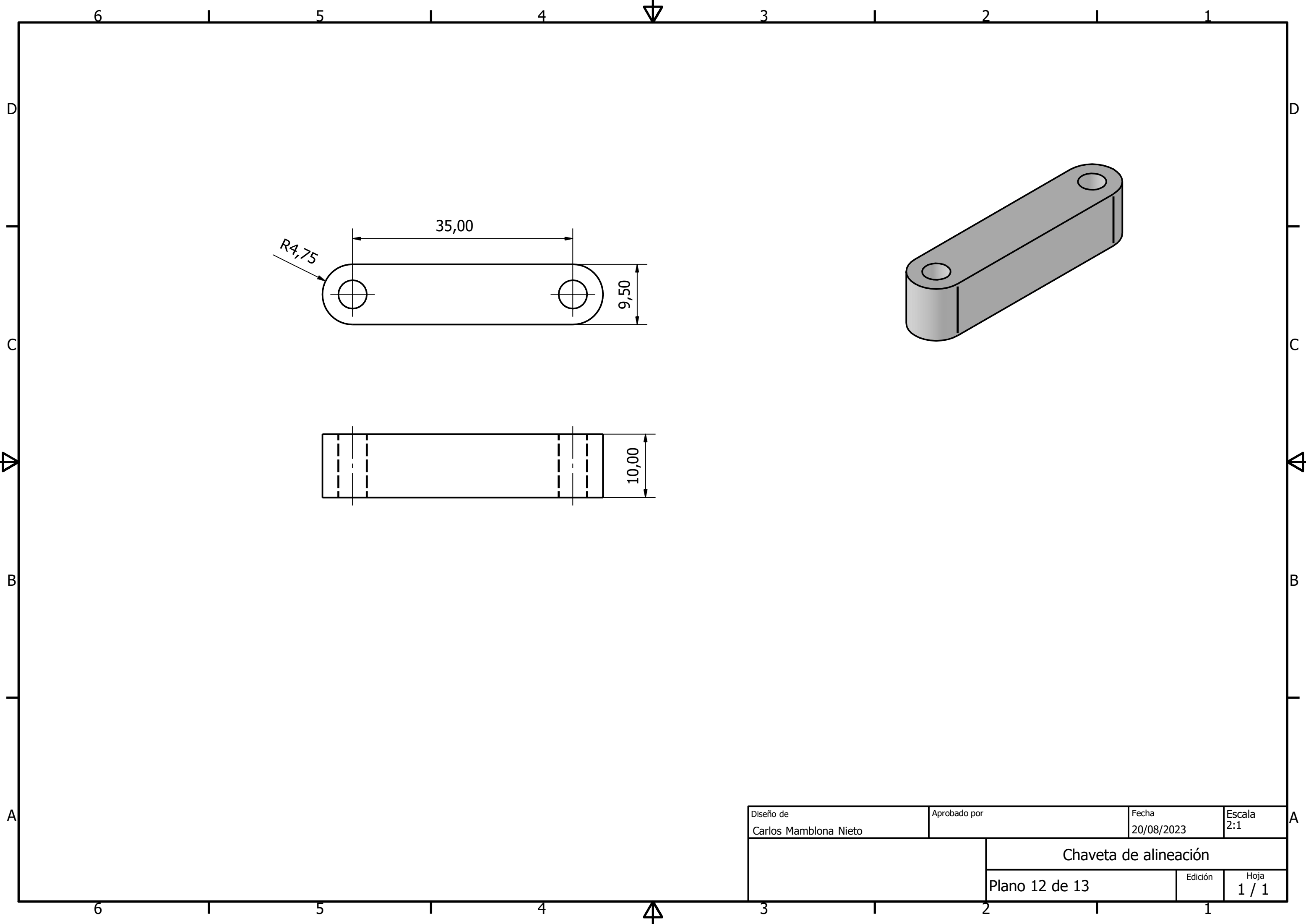


Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 2:1
		Soporte Raspberry Pi Pieza 2	
		Edición Plano 10 de 13	Hoja 1 / 1



Todos los radios no acotados
en plano equivalen a 2mm

Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 2:1
		Marco de protección de cantos Tipo 2	
		Edición Plano 11 de 13	Hoja 1 / 1



Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 2:1
		Chaveta de alineación	
		Plano 12 de 13	Edición Hoja 1 / 1

D

C

B

A

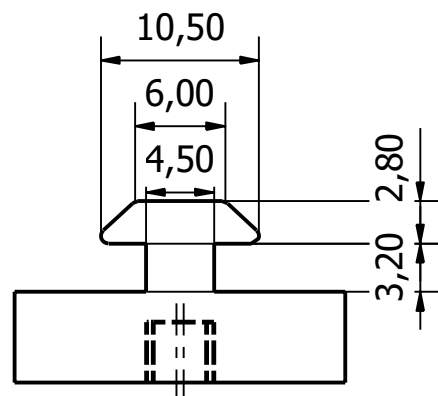
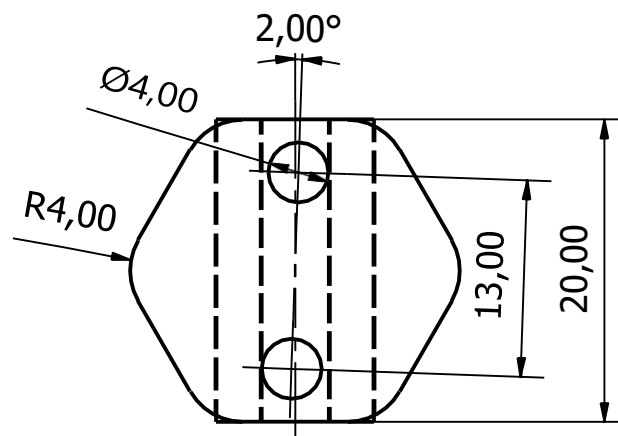
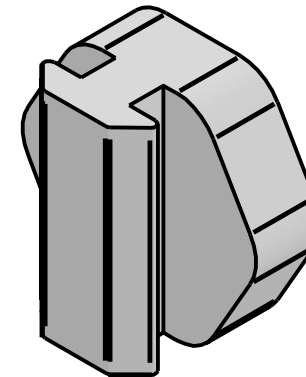
D

C

B

A

Nota:
Los taladros están desalineados por especificacion del diseño



Diseño de Carlos Mamblona Nieto	Aprobado por	Fecha 20/08/2023	Escala 2:1
		Unión cámara soporte	
		Plano 13 de 13	Edición Hoja 1 / 1



2.2. Códigos

2.2.1 Nodo de ROS2

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class ArucoCamNode(Node):

    def __init__(self):
        super().__init__("arucocam_node")
        self.create_timer(0.2, self.send_Output_Data)
        self.counter_ = 0
        self.Output_Data = "None"
        self.Output_Data_link =
        '/home/solrac/Documentos/GitHubRepositories/MUII/TFM/pipe.dat'
        self.Output_Data_Pub = self.create_publisher(String,
        "/outputdata_topic", 10)
        self.get_logger().info("ArucoCam_Node Activado")

    def send_Output_Data(self):
        Output_Data = String()
        Output_Data.data = self.Output_Data
        self.Output_Data_Pub.publish(Output_Data)
        self.get_logger().info("msg" + str(self.counter_) + ":" +
self.Output_Data)
        self.counter_ += 1
        with open(self.Output_Data_link, 'r') as pipe:
            self.Output_Data = pipe.read()

def main(args=None):
    rclpy.init(args=args)
    node = ArucoCamNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

2.2.2 Programa de detección de ArUcos

```

#* Programa de deteccion y estimacion de posicion de marcadores ArUco

#!=====||■ Configuration ■||=====

import cv2
import cv2.aruco as aruco
import numpy as np
import threading
import time

# Función periodica para la escritura de los datos de salida
Output_Data = 0
Output_T = 0.2 # Periodo s de escritura del resultado Output_Data

# Función de escritura periódica del resultado
def Print_Output():
    while True:
        if Output_Data:
            with open('pipe.dat', 'w') as pipe:
                pipe.write(Output_Data)
            print(f'msg: {Output_Data}')
            time.sleep(Output_T)

# Definición del hilo para mostrar los datos periodicamente
position_thread = threading.Thread(target=Print_Output)
position_thread.daemon = True
position_thread.start()

ArUcoID = True           # Visualizar la ID en el ArUco
Axis = True              # Visualizar los ejes en el ArUco
ArucoFrame = True        # Visualizar un marco en el ArUco
ArUcoPose = True         # Visualizar la Pose en el ArUco
visualizacion = True     # Mostrar la pantalla de visualización
hitbox = True            # Mostrar areas seguras de los robots
correccion = True        # Habilitar el cálculo de la corrección

SecDist = 400            # Distancia de seguridad entre dos robots

# Define 3D cube vertices para la visualizacion de areas seguras
# (se puede dibujar un cubo3D)
cube_points = np.array([[-1, -1, 0], [1, -1, 0],
                        [1, 1, 0], [-1, 1, 0],
                        [-1, -1, -3], [1, -1, -3],
                        [1, 1, -3], [-1, 1, -3]], dtype=np.float32)

```

```
# Cargar los parámetros de calibración
calibration_data = np.load(
    '/home/solrac/Documentos/GitHubRepositories/MUII/TFM/calibration_data.npy'
    , allow_pickle=True).item()
camera_matrix = calibration_data['camera_matrix']
distortion_coeff = calibration_data['distortion_coeff']

# Tamaño del marcador en milímetros
marker_size_mm = 70

# Diccionario de códigos ArUco 4x4
aruco_dict = aruco.Dictionary_get(aruco.DICT_4X4_100)

# Inicializar la cámara
cap = cv2.VideoCapture(0) # Inicializa la cámara

# !=====|| ■ MAIN ■ ||=====

while True:
    ret, frame = cap.read() # extrae un frame de video
    output_msg = ''
    if not ret:
        break

    # Detectar los marcadores ArUco
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    parameters = aruco.DetectorParameters_create()
    corners, ids, rejectedImgPoints = aruco.detectMarkers(gray, aruco_dict
    , parameters=parameters)

    if ids is not None: # Si hay ArUcos en la imagen:

        # Estimar la pose de los marcadores
        rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(corners,
        marker_size_mm, camera_matrix, distortion_coeff)

        if ArucoFrame: # Dibujar el contorno de los marcadores
            aruco.drawDetectedMarkers(frame, corners)

        for i in range(len(ids)):
            title = ''

            if Axis: # Dibujar ejes 3D en cada marcador
                aruco.drawAxis(frame, camera_matrix, distortion_coeff,
                rvecs[i], tvecs[i], marker_size_mm)

            if ArucoID: # Obtiene el ID y las coordenadas 3D del marcador
```

```

marker_id = ids[i][0]
title = title + (f'ID {marker_id} ')

if ArUcoPose:
    # Obtener la matriz de rotación a partir de rvec
    rotation_matrix, _ = cv2.Rodrigues(rvecs[i])
    # Calcular el ángulo de rotación alrededor del eje Z (yaw)
    w = np.degrees(np.arctan2(rotation_matrix[1, 0], rotation_matrix[0, 0]))+180
    # Calcular la posición detectada a partir de tvec
    xd, yd, zd = tvecs[i][0]

    if correccion:
        xd = -1*xd+798 # Correccion de offset
        yd = yd-85     # Correccion de offset

        x = ((xd-575) * 1.30) + 575 # Correccion de perspectiva
        y = yd - (xd-575) * 0.114   # Correccion de perspectiva
    else:
        x = xd
        y = yd

    title = title + (f'({x:.0f}, {y:.0f}, {w:.0f})')

    if 1 <= marker_id <= 10: # Si el marcador es de un robot:

# Añade al mensaje de salida las coordenadas del ultimo ID analizado
    output_msg = output_msg + (f'({marker_id:.0f}, {x:.0f}, {y:.0f}, {w:.0f})')

    colorhitbox = (0, 255, 0)
# Las areas seguras por defecto estan en verde
    for a in range(len(tvecs)):
        if 1 <= ids[a][0] <= 10 and a!=i:
# Calcula la distancia entre todos los ArUcos Robot detectados
            P1 = tvecs[i][0][:2]
            P2 = tvecs[a][0][:2]
            distancia = np.linalg.norm(P2-P1)
            if abs(distancia) < SecDist:
# Si la distancia es menor a la distancia de seguridad:
                colorhitbox = (0, 0, 255)
# Pinta el area segura de rojo
                break
    if hitbox == True:
        # Proyectar puntos de cubo sobre la imagen
        image_points, _ = cv2.projectPoints(cube_points*200, rvecs[
i], tvecs[i], camera_matrix, distortion_coeff)
        image_points = np.int32(image_points).reshape(-1, 2)
        top_face_corners = image_points[4:]

        # Dibujar bordes del area segura en la imagen
        for j in range(4):

```

```
        cv2.line(frame, tuple(image_points[j]),
                  tuple(image_points[(j + 1) % 4]),
colorhitbox, 2)

# Descomentar estas líneas para ver un cubo 3D que envuelve al robot
        cv2.line(frame, tuple(image_points[j]),
                  tuple(image_points[j + 4]), (0, 255,
0), 2)

        cv2.line(frame, tuple(image_points[j + 4]),
tuple(image_points[(j + 1) % 4 + 4])
                  , (0, 255, 0), 2)

    else:
        output_msg = ''

# Muestra el título con el número de marcador y las coordenadas 3D
        x, y = corners[i][0].mean(axis=0).astype(int)
        cv2.putText(frame, title, (x+10, y+10), cv2
.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)

    else:
        output_msg = ''

    if visualizacion:
        # Mostrar el frame con los marcadores y ejes 3D
        cv2.imshow('ArUco Detection', frame)

    if cv2.waitKey(1) & 0xFF == ord('a'):
        Axis = not(Axis)

    if cv2.waitKey(1) & 0xFF == ord('h'):
        hitbox = not(hitbox)

    if cv2.waitKey(1) & 0xFF == ord('c'):
        correccion = not(correccion)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    Output_Data = output_msg
# Copia el mensaje en la funcion periodica de escritura

cap.release()          # Libera la cámara
cv2.destroyAllWindows() # Cierra todas las ventanas emergentes
```


2.2.3 Programa de calibración de la cámara

```
# Programa de calibración por tablero de ajedrez

#!=====||■ Configuration ■||=====

import cv2
import numpy as np
import glob

# Dimensiones del patrón de calibración:
Ajedrez_DIM = (9, 6)    # El número de vértices internos de ancho y largo
Cuadrado_Tamaño = 38.0  # Tamaño de cada cuadro en milímetros
#Nota: Vertices internos = Nº Cuadrados - 1

# Crear una lista para almacenar los puntos de calibración 3D y 2D
obj_points = []  # Puntos 3D en el mundo real
img_points = []  # Puntos 2D en la imagen

# Dimensiona el patron de calibración a partir de las medidas introducidas
objp = np.zeros((Ajedrez_DIM[0] * Ajedrez_DIM[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:Ajedrez_DIM[0], 0:Ajedrez_DIM[1]].T.reshape(-1, 2)
objp *= Cuadrado_Tamaño

i = 0  # Contador de imagenes capturadas

# Inicializar la cámara
cap = cv2.VideoCapture(0)
# Usar la cámara predeterminada (puede cambiar el índice si tienes varias ca

#!=====||■ MAIN ■||=====

while True:

    # Genera una ventana emergente de video de la cámara
    ret, frame = cap.read()
    cv2.imshow('Capture Chessboard', frame)

    # Si se presiona la tecla "c":
    key = cv2.waitKey(1)
    if key == ord('c'):

        # Convierte la captura a escala de grises
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Busca los vertices del tablero de ajedrez y las guarda en "corners"
        ret, corners = cv2.findChessboardCorners(gray, Ajedrez_DIM, None)
```

```
        if ret:
            # Si la captura es correcta y contiene los vertices del tablero

            # Almacena los valores de los vertices de la imagen en la variable "img_point
            # en relación a los puntos del objeto 3D "obj_points"
            obj_points.append(objp)
            img_points.append(corners)

            # Se copia la captura y se dibujan los vertices en la imagen
            img_copy = frame.copy()
            cv2.drawChessboardCorners(img_copy, Ajedrez_DIM, corners, ret)

            # Guarda la imagen original en el siguiente archivo:
            img_ajedrez_original = f'img_ajedrez_original_{i}.jpg'
            cv2.imwrite(img_ajedrez_original, frame)

            # Guarda la imagen con los vertices dibujados en el siguiente archivo:
            img_ajedrez_procesado = f'img_ajedrez_procesado_{i}.jpg'
            cv2.imwrite(img_ajedrez_procesado, img_copy)

            print(f'Imagen original guardada como {img_ajedrez_original}')
            print(f'Imagen con esquinas dibujadas guardada como {
img_ajedrez_procesado}')
            i += 1

        elif key == ord('q'):    # si se presiona la tecla 'q': Salir
            break

    # Función de calibración
    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(obj_points, img_points,
gray.shape[:-1], None, None)

    # Guarda los parámetros de calibración en calibration_data.npy
    calibration_data = {
        'camera_matrix': mtx,
        'distortion_coeff': dist,
    }
    np.save('calibration_data.npy', calibration_data)

    print("Calibración completada")

cap.release()                # Libera la cámara
cv2.destroyAllWindows()      # Cierra todas las ventanas emergentes
```

2.2.4 Programa de conteo dinámico de puntos

```
#!/usr/bin/env python3
#* Programa de conteo dinamico de puntos

#=====■ Configuration ■=====

import cv2
import cv2.aruco as aruco
import numpy as np
import threading
import time

# Activar solo para depurar
visualizacion = True

Puntuacion = 0

# Rangos de color para amarillo, rosa y marrón
rango_amarillo = [(20, 100, 100), (30, 255, 255)] # Amarillo
rango_rosa = [(150, 50, 50), (170, 255, 255)] # Rosa
rango_marron = [(0, 50, 50), (20, 120, 120)] # Marrón

# Diccionario de códigos ArUco 4x4
aruco_dict = aruco.Dictionary_get(aruco.DICT_4X4_100)
marker_size_mm = 40 # Tamaño del marcador en milímetros

# Cargar los parámetros de calibración
calibration_data = np.load(
    '/home/solrac/Documentos/GitHubRepositories/MUII/TFM/calibration_data.npy',
    allow_pickle=True).item()
camera_matrix = calibration_data['camera_matrix']
distortion_coeff = calibration_data['distortion_coeff']

#=====■ MAIN ■=====

cap = cv2.VideoCapture(0) # Inicializa la cámara

ret, frame = cap.read() # extrae un frame de video

cap.release() # Libera la cámara

# Recortar un poco la imagen para no detectar ArUcos fuera del area de interés
frame = frame[100:1080,260:1660]
```

```
# Detectar los marcadores ArUco
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
parameters = aruco.DetectorParameters_create()
corners, ids, rejectedImgPoints = aruco.detectMarkers(gray, aruco_dict,
parameters=parameters)

if ids is not None: # Si hay ArUcos en la imagen:

    for i in range(len(ids)):

        # Estimar la pose de los marcadores
        rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(corners,
marker_size_mm, camera_matrix, distortion_coeff)

        # Obtener la matriz de rotación a partir de rvec
        rotation_matrix, _ = cv2.Rodrigues(rvecs[i])
        # Calcular el ángulo de rotación alrededor del eje Z (yaw)
        w = (np.degrees(np.arctan2(rotation_matrix[1, 0], rotation_matrix
[0, 0])))
        # Calcular la posición detectada a partir de tvec

        x, y = corners[i][0].mean(axis=0).astype(int)
        cv2.putText(frame, (f'ID {ids[i][0]}, w {w:.0f} '), (x+10, y+10
), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)

# Hay que ajustar la coordenada x en función de la orientación del ArUco
area_x = x - 25
area_x += int(np.sin(np.radians(w))*120)

# Guarda una copia de la sección donde se ven solo los colores de la tarta
frame_tarta = frame.copy()
frame_tarta = frame_tarta[(y+50):(y+400),area_x:(area_x + 50)]

cv2.imshow('tarta', frame_tarta)

# Convertir la imagen a HSV
imagen_hsv = cv2.cvtColor(frame_tarta, cv2.COLOR_BGR2HSV)

# Crea arrays NumPy a partir de los rangos de color seleccionados
rango_amarillo = np.array(rango_amarillo, dtype=np.uint8)
rango_rosa = np.array(rango_rosa, dtype=np.uint8)
rango_marron = np.array(rango_marron, dtype=np.uint8)

# Crea una máscara para cada rango de color
mascara_amarillo = cv2.inRange(imagen_hsv, rango_amarillo[0],
rango_amarillo[1])
mascara_rosa = cv2.inRange(imagen_hsv, rango_rosa[0], rango_rosa[
1])
mascara_marron = cv2.inRange(imagen_hsv, rango_marron[0],
rango_marron[1])
```

```
# Aplica las mascararas a la imagen original para obtener las areas de interes
    area_amarilla = cv2.bitwise_and(frame_tarta, frame_tarta, mask=
mascara_amarillo)
    area_rosa = cv2.bitwise_and(frame_tarta, frame_tarta, mask=
mascara_rosa)
    area_marron = cv2.bitwise_and(frame_tarta, frame_tarta, mask=
mascara_marron)

    # Contar cuantos colores se han detectado
    # solo si hay mas de 100px de ese color, para evitar ruido
    if cv2.countNonZero(mascara_amarillo) > 100:
        Puntuacion += 1
    if cv2.countNonZero(mascara_rosa) > 100:
        Puntuacion += 1
    if cv2.countNonZero(mascara_marron) > 100:
        Puntuacion += 1

    # Mostrar las areas detectadas
    cv2.imshow('Área Amarilla', area_amarilla)
    cv2.imshow('Área Rosa', area_rosa)
    cv2.imshow('Área Marrón', area_marron)

print(f'Ptos: {Puntuacion}')

if visualizacion:
    # Mostrar el frame con los marcadores y ejes 3D
    cv2.imshow('ArUco Detection', frame)
    cv2.waitKey(0)

#*----- Guardar Puntuacion-----

with open('ptos.dat', 'w') as pipe:
    pipe.write(str(Puntuacion))

cv2.destroyAllWindows()    # Cierra todas las ventanas emergentes
```

3. Presupuesto

- Estructura de soporte del sistema:

Qt	Referencia	Descripción	Tipo	Coste estimado
1 uds	SoporteCam_P1	Base SoporteCam Pieza 1	Impresión 3D	1,73 €/ud
1 uds	SoporteCam_P2	Columna Soporte	Comercial	14,95 €/ud
1 uds	SoporteCam_P3	Base SoporteCam Pieza 2	Impresión 3D	0,86 €/ud
1 uds	SoporteCam_P4	Barazo SoporteCam	Impresión 3D	1,18 €/ud
1 uds	SoporteCam_P5	Carcasa FaceCam	Impresión 3D	1,17 €/ud
2 uds	SoporteCam_P6	Unión cámara soporte	Impresión 3D	0,10 €/ud
2 uds	SoporteCam_P7	Pestaña de cierre carcasa FaceCam	Impresión 3D	0,05 €/ud
2 uds	SoporteCam_P8	Marco de protección de cantos Tipo 1	Impresión 3D	0,09 €/ud
1 uds	SoporteCam_P9	Soporte Raspberry Pi Pieza 1	Impresión 3D	0,94 €/ud
2 uds	SoporteCam_P10	Soporte Raspberry Pi Pieza 2	Impresión 3D	0,27 €/ud
1 uds	SoporteCam_P11	Marco de protección de cantos Tipo 2	Impresión 3D	0,04 €/ud
1 uds	SoporteCam_P12	Chaveta de alineación	Impresión 3D	0,15 €/ud
6 uds	TUER-01	Tuerca M4	Comercial	0,06 €/ud
8 uds	TUER-02	Tuerca para insercion M4	Comercial	0,10 €/ud
12 uds	TUER-03	Tuerca para insercion M3	Comercial	0,10 €/ud
1 uds	TUER-04	Tuerca de paloma M8	Comercial	0,58 €/ud
6 uds	TOR-01	Tornillo allen redondeado M4x20	Comercial	0,67 €/ud
4 uds	TOR-02	Tornillo allen redondeado M4x15	Comercial	0,46 €/ud
8 uds	TOR-03	Tornillo allen prisionero M4	Comercial	0,03 €/ud
12 uds	TOR-04	Tornillo allen avellanado M3x8	Comercial	0,30 €/ud
1 uds	TOR-05	Tornillo hexagonal M8x50	Comercial	0,30 €/ud

TOTAL	34,97 €
--------------	----------------

- Resto de componentes del sistema:

Qt	Artículo	Imagen	Tipo	Coste
1	RASPBERRY PI 4 - MODELO B - 8GB		Esencial	94,95 €/ud
1	Elgato Facecam - Webcam Full HD 1080p60 real para streaming, juegos, videoconferencia, sensor Sony, corrección de luz avanzada, controles avanzados, funciona con OBS, Zoom, Teams y demás, para PC/Mac		Esencial	139,00 €/ud
1	Elgato Facecam - Webcam Full HD 1080p60 real para streaming, juegos, videoconferencia, sensor Sony, corrección de luz avanzada, controles avanzados		Optativo	39,95 €/ud
1	cámara Raspberry Pi B 3/2 2592x1944 Resolución 5MP de Alta resolución, Chip OV5647, Gran Angular de 175 °		Optativo	22,08 €/ud

3	SanDisk Tarjeta microSDXC Extreme de 128 GB + adaptador SD + RescuePRO Deluxe de hasta 190 MB/s, con rendimiento de aplicación A2, UHS-I, Clase 10, U3, V30		Esencial	15,09 €/ud
1	Lector Tarjetas SD USB 3.0 Tipo C Micro USB Lector de Tarjetas con TF, SDxC, SDHC, MS, MMC, MSxC Tarjetas		Opcional	8,99 €/ud
1	USB Tipo C a USB 3.1 Gen 2 Cable 10Gbps Cable de Datos y Cable de Carga Rápida 3A		Esencial	9,99 €/ud
1	Cable USB Tipo C 30 cm, USB 3.1 Gen2 Cable USB C Carga Rápida, 10Gbps Cable Datos USB C		Opcional	8,99 €/ud
1	FUENTE ALIMENTACION 5,2V 3A 15,6W USB-C CON INTERRUPTOR		Esencial	8,50 €/ud
1	CABLE HDMI A MICRO HDMI (TIPO D) 2M. M/M NEGRO		Esencial	6,75 €/ud
1	DISIPADOR INTEGRAL CON VENTILADOR CONTROLADO POR SOFTWARE - PWM		Opcional	9,95 €/ud

TOTAL 394,42 €

Coste total de material asociado al proyecto	429,39 €
---	-----------------

4. Anexos

4.1 Normativa Eurobot 2023



Eurobot^{Open} 2023 Rules

30th edition of the robotic contest - Eurobot
OFFICIAL version



NOTE: all images in this document are provided as a guide to illustrate the various paragraphs. In no case they can serve as a reference. Only the dimensions, colors and materials indicated in the appendix shall be taken into consideration.

. Contents

A. NOTICE	3
B. CONTEST PRESENTATION	4
C. THEME PRESENTATION	5
D. PLAYING AREA AND ACTIONS	6
D.1. IMPORTANT INFORMATION	6
D.2. PLAYING AREA	7
D.3. STARTING AREAS	8
D.4. DESCRIPTION AND LAYOUT OF THE PLAYING ELEMENTS	9
D.5. MAKING CAKES	10
D.6. PUTTING THE CHERRY ON THE CAKE	11
D.7. DROPPING CHERRIES IN THE BASKET	12
D.8. PUTTING THE WHEELS IN THE PLATE	13
D.9. DISGUISE TO MAKE THE PARTY	13
D.10. ESTIMATE THE PERFORMANCE	15
E. PROJECT PRESENTATION	16
F. THE ROBOTS	17
F.1. FOREWORDS	17
F.2. DIMENSIONS	17
F.3. SECURITY CONSTRAINTS	19
F.4. COMMUNICATION SIGNALS	21
F.5. OTHER DESIGN CONSTRAINTS	21
G. BEACON TRACKING SYSTEM	23
G.1. GENERAL POINTS	23
G.2. EMBEDDED/ ON-BOARD BEACONS	23
G.3. FIXED BEACONS	24
G.4. CENTRAL TRACKING DEVICE	24
G.5. CONNECTIONS	25
G.6. ROBOT IDENTIFICATION	26
H. MATCHES	27
H.1. PREPARATION TIME	27
H.2. THE MATCH	27
H.3. END OF MATCH	27
H.4. COUNTING POINTS	28
I. THE CONTESTS	31
I.1. GENERAL INFORMATION	31
I.2. APPROVAL	31
I.3. QUALIFICATION PHASE	31
I.4. THE PLAY-OFF PHASE	32
I.5. THE FINAL PHASE	33
I.6. QUALIFICATION FOR THE NATIONAL MEETING	33
I.7. QUALIFICATION FOR THE EUROPEAN MEETING	33
J. APPENDIX	35

J.1. GENERAL DRAWINGS	35
J.2. MATERIAL REFERENCES	40
J.3. MANUFACTURING TOLERANCES	40
J.4. COLOUR REFERENCES	41

A. NOTICE

WARNING!

General remarks are annotated in the document. Please, pay special attention to these points.

This year the rules have been split up in several documents. Most parts remain the same for the Eurobot^{Open} and Eurobot^{Open} Junior contests, but to avoid confusion, each contest has its own rules as a single document.

Thus the particular cases specific to one of the competitions appears only in the document concerning it.

You will find the Eurobot^{Open} and the Eurobot^{Open} Junior rules and other information on the Eurobot^{Open} website (<http://www.eurobot.org/>)

Please note that the version of this release is noted down at the end of this page. For any inquiry, only an official version should be considered.

Changes or clarifications of the rules may be made during the year. We therefore strongly encourage all participants to check our website regularly (<http://www.eurobot.org/>) as well as your NOC's own website for news. You can also follow discussions, ask questions or get further assistance on our faq (<https://www.eurobot.org/faq/>).

Possible changes of the technical specifications will be announced on the Eurobot website, (<http://www.eurobot.org/>) or on the website of the National Organization Committee (NOC) in your country.

The forum responses from a referee are official responses taken into account for match refereeing and homologation stages.

In case of doubt regarding any point of the rules or the approval of robots, the referee committee may also be contacted at referee@planete-sciences.org.

Have a good reading!

B. CONTEST PRESENTATION

Eurobot^{Open} and Eurobot^{Open} Junior are two events addressed to young people interested in robotics. Teams are usually composed of students involved in an Eurobot^{Open} school project, a group of friends, or independent robotics clubs. They share the same goal: to offer young people an active learning process, to put into practice their knowledge and soft skills during a fun and friendly event. The technical challenge of Eurobot^{Open} is to build an autonomous robot alongside with an optional secondary autonomous robot.

The rules for both events (Eurobot^{Open} and Eurobot^{Open} Junior) are based on the same concept. As organisers, we intend to provide a common platform for the Eurobot^{Open} event. This platform is dedicated to autonomous robots for Eurobot^{Open} while for Eurobot^{Open} Junior, the robots are remote controlled. In this way, a Eurobot^{Open} organiser can easily set up a Eurobot^{Open} Junior contest and vice versa.

You're currently reading version
Eurobot^{Open} OFFICIAL of 2023rules.
(this version concerns fully autonomous robots)

The age limit for participating in the Eurobot^{Open} finals is 30 years. However, each team may have a supervisor whose age limit exceeds 30. It is important to note that teams that do not respect the age limit, will not be allowed to participate in the Eurobot^{Open} finals.

A team is a group of 2 young people at minimum who have built one or two robots for the event. A person can only represent one team. However, we encourage teams to share their expertise and knowledge.

An organization (club, school, etc.) can supervise and register several teams, if allowed by the registration requirements set by your National Organizing Committee. The acceptance of these requirements is compulsory to validate your registration and your entry.

The project can be supervised by someone over the age limit (teacher, parent, group leader, etc.), but all elements of the robot(s) must be designed by the participants. In this context, robots manufactured from a commercially purchased chassis or rolling base will not be accepted.

In the event that a robot was imagined, built or modified by the tutor alone, organisers can disqualify or reject the participation of the team in the competition. He can in exchange advise the students and guide them.

Eurobot^{Open} and Eurobot^{Open} Junior European finals gather teams selected after national qualifications. European finals take place in Europe, but all countries can participate. Countries where more than three teams are registered have to organize a national qualification to select teams that will attend the European finals.

Common parameters can change from one year to another. Accordingly, please read the rules carefully even if the chapters may seem familiar to you (playing area dimensions, robots dimensions, starting area dimensions, etc.).

Robotics contests are public events. Therefore, we ask the teams to respect our rules of decency and safety (electrical, sound level, manners, etc.). These rules apply to the participants, their supporters and all the equipment they bring.

C. THEME PRESENTATION

Happy birthday Eurobot contest! Grandma Monique wants to celebrate the event with a ton of cakes and she has asked you to help her to prepare them according to her legendary recipe and be sure that everyone has one of these fantastic cakes.

Sponge cake, cream, icing and the famous cherry, are all the ingredients necessary to work for Grandma Monique. Take your best wooden spoon!

The missions will be:

- **Making cakes,**
- **Putting cherry on the cake,**
- **Dropping the remaining cherries in the basket,**
- **Putting the wheels in the dish, at the end of the service,**
- **Deguisse to make the party,**
- **Estimating your performance.**

Warning! All actions are independent from one another and no specific sequence is imposed whatsoever. No single action is compulsory. Give careful thoughts to your strategy. It is strongly recommended to design simple and reliable systems with a limited number of actions.



Figure 1: Overview of the playing area

D. PLAYING AREA AND ACTIONS

D.1. IMPORTANT INFORMATION

The organisers are committed to build the playing area with as much accuracy as possible. Nevertheless, they reserve the right to do minor modifications and adjustments.

No complaints regarding dimensional deviations will be taken into account.

Teams are warned that the surface condition may differ from one playing area to another and may also degrade over time.

D.2. PLAYING AREA

The playing area is a horizontal rectangular plan of 3000 mm by 2000 mm with borders of 70 mm height on each side. Depending on the building process, it may consist of one or more pieces (eg 3 pieces of 1000 mm per 2000 mm).

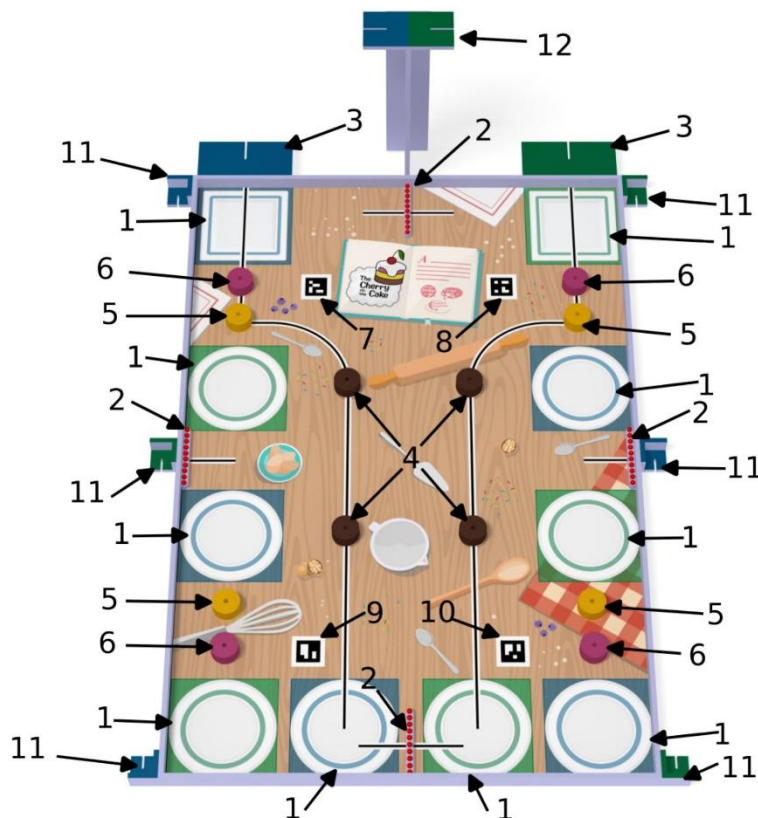


Figure 2: Detailed view of the playing area

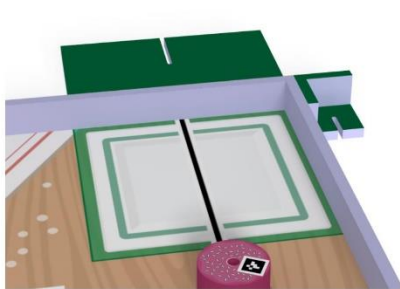
- | | |
|------------------------|-----------------------------|
| 1. Dropping areas | 7. ArUco marker number 20 |
| 2. Cherry dispensers | 8. ArUco marker number 21 |
| 3. Basket support | 9. ArUco marker number 22 |
| 4. 3 sponge cakes pile | 10. ArUco marker number 23 |
| 5. 3 cream pile | 11. Fixed beacon supports |
| 6. 3 icing pile | 12. Central tracking device |

Full specifications of the playing area and game elements (dimensions, positions at the beginning of the match, colors and other references) are listed in the appendix. In the remainder of this document, horizontal and vertical directions are stated relative to the playing area. Notions of “left”, “right”, “front” and “back” are stated with respect to the spectator’s point of view.

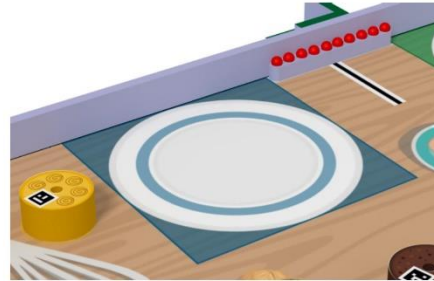
D.3. STARTING AREAS

D.3.a. DESCRIPTION

Each team has 5 dropping area, they are squared surfaces of 45 cm side delimited by a line on the table of the team’s color (the line is included in this zone). Thoses area are both the starting areas for their robots (it’s not mandatory to start all the robots from the same zone), the dropping area for the cakes, and the final area for the robots. (Check the distribution of the dropping areas in the general plan of the table)



(a) One of green dropping area



(b) One of blue dropping area

D.3.b. CONSTRAINTS

At the end of the setup time, the vertical projection of the robots must not exceed the limits of their starting area.

Make sure your robots can fully enter their starting area. The starting area does include the colored lines.

During the 3 minutes of preparation time, a robot can to change its starting zone among all the zones of the team. After the 3 minutes, the robot have to start from the zone in which one he is.

D.4. DESCRIPTION AND LAYOUT OF THE PLAYING ELEMENTS

Cake layer: The cake layers are solid discs, with an external diameter of 120 mm, a thickness of 20 mm and a maximum mass of 100 g. They are covered by vinyl on both sides, vinyl which contains an ArUco tag. They are 36 in number, and are initially located on predefined locations (grouped by 3 of identical color) placed by the markings on the playing area.

ArUco 4x4 tags are printed on the faces of cake layers in the following way:

- A tag number **47** on the face of the **icing** colored in pink.
- A tag number **13** on the face of the **cream** colored in yellow.
- A tag number **36** on the face of the **sponge cake** colored in brown.



Figure 4: The cake's 3 layers

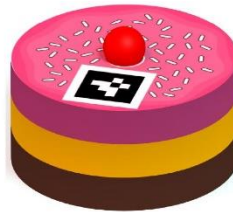


Figure 5: A cake with 3 layers and a cherry

The plates and platters: They are dropping areas of each team arranged along the edges of the playing area. The dropping area in the back corner are exclusively for the team usage, and are named "platters". The other dropping areas are named "plates". (Check the distribution of the dropping areas in the general plan of the table).

The cherries: The cherries are red foam balls compatible with Nerf RivalTM. They are 60 in total on the table (40 on supports, and 20 preloaded on robots).

Cherry support: The cherry support are drilled planks filled with 10 cherries. There are 4 scattered around the table.

Basket support: The horizontal surface of the basket support is leveled with the top of the playing area border, located at the back of the table, on the same side of the team.

Basket: Made by the team, the basket is the item in which teams deposit the cherries after picking them up. It must be fixed to the basket support during the preparation phase of the game.

D.5. MAKING CAKES

Robots have to build cakes with the different layers prepared by Grandma Monique and share them with all the guests. Watch out as your opponent can steal your cakes and serve them in their own plates! Who will have the most?

D.5.a. DESCRIPTION AND LAYOUT OF THE PLAYING ELEMENTS

For this action, only the cake layers are used with the dropping areas.

D.5.b. ACTIONS AND CONSTRAINTS

Actions:

- The robots must pick up the layers of cake, sort them, and assemble them by stacking the layers in a precise order. Once the cakes are done, robots will have to place them in the dropping areas provided for this purpose.

Constraints:

- For a cake to be valid, it must be composed of at least 1 layer, and at most 3 layers.
- A layer is considered part of a cake if it is in horizontal contact with the play area for the first layer, or if it rests mainly and by horizontal contact on a lower layer. A layer will count only for 1 cake.
- A cake can be valid with any type of layers.
- The legendary recipe is composed of the 3 following layers, in order, from top to bottom:
 - Icing
 - Cream
 - Sponge cakeRespecting this order grants bonus points on the cake.
- To be considered valid, a cake must have all or part of vertical projection of one of its element in a team's dropping area.
- A dropping area cannot accommodate more than 3 cakes, any additional cake will not be counted. In this case, the counting of points will be done to the advantage of the team.
- Only the cakes dropped in the dropping area at the bottom of the table (in front of the basket supports) will be for the use of the team exclusively.
- If a cake is still under a robot's control at the end of the match, it will not be counted.

D.5.c. POINTS

- **1 point** per layer in the cake
- **4 extra points** if the cake respects the legendary recipe

D.6. PUTTING THE CHERRY ON THE CAKE

The cherry-er the better for your cakes! It is up to you to put a cherry on top of every cake.

D.6.a. DESCRIPTION AND LAYOUT OF THE PLAYING ELEMENTS

The cakes and cherries are used for this action.

D.6.b. ACTIONS AND CONSTRAINTS

Actions:

- Collect the cherries to place them on the cakes

Constraints:

- For a cherry to be considered valid on a cake, it must be present on the top layer of a valid cake.
- The limit is 1 cherry per cake.
- Teams will be able to pre-load up to 10 cherries into their robots during preparation time. Among these cherries, the non pre-loaded ones will be taken out of the playing area for the all game. They must be easily recoverable at the end of the match.
- The rest of the cherries are placed on the cherry dispensers at the start of the match.

D.6.c. POINTS

- **3 points** for each cherry placed on a cake;

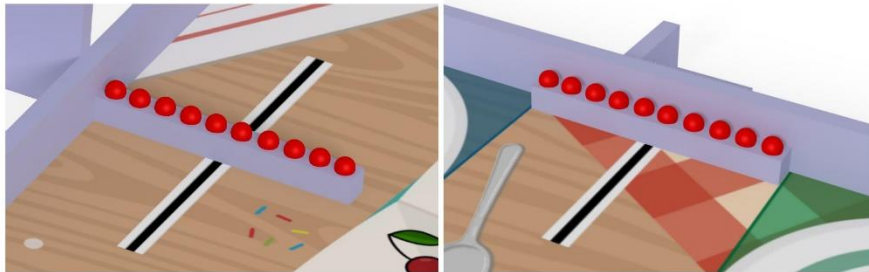


Figure 6: Cherry dispensers

D.7. DROPPING CHERRIES IN THE BASKET

Grandma Monique would like that all the ingredients are not lying around: you have to drop the cherries in the basket. Your robots are helped with an intelligent basket that counts the remaining cherries for inventory.

D.7.a. DESCRIPTION AND LAYOUT OF THE PLAYING ELEMENTS

The cherries and the basket are used for this action.

D.7.b. ACTIONS AND CONSTRAINTS

Actions:

- The team must drop the cherries in the basket of its colour.
- The basket, made by the team, must indicate the number of cherries it contains.

Constraints:

- A cherry is valid in the basket only if it is present in it at the end of the match.
- Cherries are not subject to the relative height restriction of game elements, so shooting is allowed.
- The cherries must remain visible and easily retrievable by the referees.
- The basket must indicate the number of cherries it contains via a visible and easily readable display, and must be expressed in decimal system. The counting must be done in direct reading of the value, the reading modes by calculation are not allowed. For the display, there are two exclusive options:
 - Pre-match evaluation on a static display: the team writes the number of cherries it intends to count during the match.
 - Evaluation during a match on a dynamic display device, which must continue to show the estimated number of cherries after the end of the match.
- Access of the cherries to the interior of the basket must be done above 300 mm in height from the level of the playing area.
- The vertical projection of the basket is not allowed to cross the limits of the basket stand at any time of the match.
- The basket has the following dimensional constraints:
 - Maximum depth: 222 mm.
 - Maximum width: 450 mm.
 - Maximum height: 430 mm.
- The weight of the basket must not exceed 3 kg.
- The horizontal plane of the support has a 10 mm wide groove running from the center of the support to the center of the back side. The groove must be used to fix the basket to the support by using threaded rod of 8 mm diameter and wing nut.
- Apart from the fixing screw and nut, no system or element is allowed to protrude under the support. The screw size is not counted in the height measurement of the basket.
- The action must not be dangerous for the public, the people around the table, the playground or the robots.
- The basket cannot be activated by an external element (team crew, remote control, ...).

D.7.c. POINTS

- **5 points** if the team places a basket during the preparation time;
- **1 point** per cherry in the basket
- **5 extra points** if the count is correct and not null

D.8. PUTTING THE WHEELS IN THE PLATE

Once their work is done, the robots will wait for the guests in their plates to party with them. As nobody wants a tire mark on their cakes: between a area full of cakes or a robot, it's time to choose!

D.8.a. DESCRIPTION AND LAYOUT OF THE PLAYING ELEMENTS

The depositing area zones are used for this action.

D.8.b. ACTIONS AND CONSTRAINTS**Actions:**

- At the end of the match, the robot must be stopped in their own dropping area.

Constraints:

- To be considered valid a robot vertical projection must be at least partially inside the limits of the dropping area of the team.
- Any other element in the team's dropping area will not be take into account.
- A team with multiple robots can only validate this action if both robots are in the same dropping area.

D.8.c. POINTS

- **15 points** if all team's robot are in its own dropping area;

D.9. DISGUISE TO MAKE THE PARTY**D.9.a. DESCRIPTION AND LAYOUT OF THE PLAYING ELEMENTS**

Only the robots are implicated for this action.

D.9.b. ACTIONS AND CONSTRAINTS**Actions:**

- After its job done, the robot must disguise to celebrate the party.

Constraints:

- The disguise must change the colour or the appearance of the robot.
- If a team have two robot, only one robot disguised is necessary to validated this action.
- The disguisement have to be contained inside the robots from the beginning to the end of the match.
- The disguise must do at least 50% of perimeter of the robot, in the limit of the deployed perimeter, and with a maximum height of 35cm.
- The disguise must be at least 15cm height after deployment.

- The action must be visible to the public.
- This action must be performed before the end of the match, and after that the disguised robot must stop and turn off all their actuators.

D.9.c. POINTS

- **5 points** if the funny action is performed;

D.10. ESTIMATE THE PERFORMANCE

D.10.a. DESCRIPTION AND LAYOUT OF THE PLAYING ELEMENTS

The device for displaying the score estimation during the match must be made by the team.

D.10.b. ACTIONS AND CONSTRAINTS

- The team must evaluate the number of points scored in the match by its robot(s). For this, there are two exclusive options:
 - Pre-match evaluation on a static display: the team writes the score it intends to make during the match.
 - Evaluation during a match on a dynamic display device, which must continue to show the estimated score after the end of the match.
- The display must be on the robots or the basket.
- The display area and its reading orientation must be easily visible and identifiable by the referees.
- The estimated score is an integer and must be expressed in decimal system.
- It is allowed for a team with two robots to design a display for each robot. In this case, the score assessment will be the sum of the values of the two displays; the type of reading and the order must be explained to the referees at the beginning of the match.
- Under no circumstances should the score change once the match has ended, otherwise the bonus will be lost!

D.10.c. POINTS

The assessment is based on all the previous actions.

The estimation bonus is calculated as follows: **Bonus = 20 points - Delta**

- The score is the one scored by the team during the match on standard actions.
- The delta is the difference between the score made by the team during the match and the score estimated by the team. This one is always positive (Absolute value).
- The bonus is an integer value (rounded up).
- The bonus is added to the points of the team.
- A negative bonus is reduced to 0.
- The bonus for not "forfeit" is not include in the performance estimation.
- The penalties are not include in the performance estimation.

E. PROJECT PRESENTATION

Both Eurobot^{Open} and Eurobot^{Open} Junior encourage participants to practice science in a funny and original way. Our main objective is to assist and value your projects conceived during the year. To achieve this, the teams must conceive a poster, and, organizers may require the teams to create a technical documentation of their robots.

We expect to see attractive, innovative robots that respect this edition's technical constraints and rules. Being creative and original will add value to your work as much as the performance of your robot(s) during the matches. By doing this presentation, you will increase the communication value of your project and the visual effect of your robots, for both the public coming to the events as well as for your own satisfaction. Having created something aesthetically and functionally complete, will strengthen your work attitude during and after the competition.

TECHNICAL POSTER

Each team is required to provide a technical poster of their robot. This poster should present information related to the design of the robot (drawings, technical references, design specifications, etc.). It should be at least size A1 (594x841 mm) and ideally it should be printed. This poster is intended to encourage exchange and communication between teams.

Special vulgarization efforts should be made to make the content of the poster accessible to the general public. Ideally, the poster should include pictures and charts to explain the concept to the public.

The poster must also include:

- the name of the team.
- the names of team members.
- the country of the team.
- the country flag of the team.

This poster will be posted on the booth of each team during the competition or in a place designated for the posters. For the international Finals, an English version may be requested. The chosen resolution must guarantee the legibility of all texts. The resulting PDF file must not exceed 25 MB. The PDF version of the poster may be sent to the organization prior to the meeting via your National Organizing Committee.

In general, the organization encourages teams to communicate around their projects on the Internet, social media, via forums, etc.

F. THE ROBOTS

F.1. FOREWORDS

Each team is allowed to register a maximum of two robots, called respectively "main robot" and "secondary robot". Each one have identical dimensional constraints, but it must be possible to distinguish them from each other from the public. The set of both robots have additional dimensional constraints.

For Eurobot^{Open}, both robots are autonomous.

The construction of a secondary robot is optional. The aim is to allow teams with a large number of members to work on a second project. It is recommended for beginning teams to concentrate on building a single functional machine. Having one robot that works well is better than having two that do not move.

A secondary robot can compete only with the main robot with which it was designed and approved. However it can compete alone if the main robot cannot participate. It cannot be re-approved with another main robot.

The main and secondary robots must be made up of elements that are fixed to each other (so robots cannot leave parts or elements on the playing area), with the exception of the playing elements.

Deliberately vibrating the table, attach themselves to it or any other irregular action exposes the team to a refusal of approval.

Each team must design a unique and original set of robots, specific to their team.

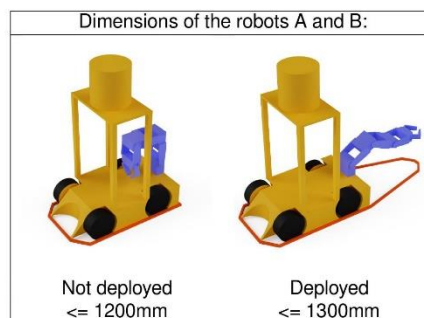
Be imaginative! For example, as an innovation but also to offer the public and the media an attractive show, your robot can use sounds, display expressions, etc.!

F.2. DIMENSIONS

Warning: the dimensions of Eurobot^{Open} robots are identical to those of Eurobot^{Open} Junior. Eurobot^{Open} Junior participants can more easily access the Eurobot^{Open} meetings. The Eurobot^{Open} Junior robot will only need modifications to make it autonomous.

Dimensions of the robots :

The perimeter of a robot is the perimeter of the convex shell of its vertical projection on the ground. It is measured by surrounding it as shown in the illustrations below:



The perimeter of a robot must not exceed 1200 mm at the departure time. The perimeter of the fully deployed robot shall not exceed 1300 mm during the match. It is allowed to change the convex hull shape during the game, as long as the perimeter of this one always respects the maximum perimeter constraint.

Not deployed dimensions of robots A and B	Deployed dimensions of robots A and B
 $A + B \leq 2050\text{mm}$	 $A + B \leq 2200\text{mm}$

The sum of the perimeters of the two robots at the start of the match must not exceed 2050 mm. The sum of the deployed perimeters of the two robots must not exceed 2200 mm. Be careful, the sum of the deployed perimeters is equal to the sum of the maximum of the deployment perimeter of each of the two robots during the entire duration of the match.

If a physical connection (mechanical, electrical, magnetic,...) exists between two robots of the same team, then the group formed by the two robots is in fact considered, for the perimeter constraints, as a single robot, until the group is dissociated. Note that a simple contact is not considered a physical connection.

At any time during the match the height of each robot and manipulated object must not exceed 350 mm. However, it can be tolerated that the emergency stop button exceeds this limit height to reach 375 mm.

This height excludes the beacon support mast, any sensors and electronic circuits integrated under the beacon holder mast.

Emergency stop button :

All systems including a source of stored electrical energy must be equipped with a red emergency stop button of at least 20 mm in diameter. It must be placed on the top of the system, in a visible position on a free surface and in a non-risking area to be immediately accessible by the referees at any time during the match.

The button may exceed the system's height by 25 mm. The emergency stop button must be operated by a simple downward movement (for example, by hitting it with the fist).

(Only exceptions to this rule, the beacons embedded on the beacon mast of the opposing robots are exempt from the emergency stop button).

Pressing this button must stop all system immediately. For robots, the non-cutting of control and avoidance systems is tolerated.

In order to avoid any risk of fire, attention should be paid to the diameters of the wires, depending on the intensity of the currents flowing through them. It is also strongly advised to protect the electrical installation with a fuse, wired close to the batteries.

F.3.c. LASERS

Only laser systems and classes defined according to the IEC60825 international standards are accepted. Teams using lasers must provide the manufacturer's document mentioning **the class of the device** (this information is normally always available on the system itself).

On the basis of this classification, class lasers:

- 1 and 1M are accepted without restriction
- 2 are tolerated only in case the laser beam do not exceed the play area, and if it is off when the robot does not touch the play area.
- 2M, 3R, 3B and 4 are strictly forbidden.

WARNING: Disassembling or modifying devices using laser sources often results in a change of class. Laser devices must not be altered and only be used in the state of their commercialization (laser device = source + optics + electronics).

F.3.d. HIGH POWER LIGHT SOURCES

When using a high intensity light source, the light intensity must not be dangerous to the human eye in case of direct contact. Note that some types of LEDs have warnings. Be responsible, as your machines are evolving in front of a general audience!

In the case of slightest doubt, the organization reserves the right to request the manufacturer's specifications to verify the non-dangerous nature of the lighting system used. If it turns out that the system is potentially dangerous, it may result in revocation of lasers class 2M and more.

F.3.e. COMPRESSED AIR SYSTEMS

Compressed air systems should not exceed 4 bars, except in pre-assembled commercial products, and only if :

1. these devices have not been modified.
2. They are compliant with european safety regulations.
3. they are safe.

The use of pressurised gas cartridges such as CO2 cartridges is prohibited

F.3. SECURITY CONSTRAINTS

F.3.a. GENERAL ASPECTS

All systems (robots, beacons and accessories) must comply with the European standards.

This is mandatory for the countries outside the EU that are organizing national meetings or send independent teams for the European Finals. Among other things, these security standards must respect safety rules and must not endanger participants, organizers nor public. They must not have protruding or pointed parts that could be dangerous or cause damage

The use of liquid, corrosive, fuel, pyrotechnic, radioactive, living beings or zombies is prohibited.

In general, any system deemed by the referees' committee as dangerous will not be approved, and must be replaced, in order to be accepted in the competition.

F.3.b. ENERGY SOURCES

The only sources of stored energy authorized in robots and auxillary systems are electrical chemical batteries, commercial batteries, springs and elastics, compressed air, gravitational energy. All other energy sources are prohibited.

If you have any doubt about unconventional energy sources, ask the referees' committee as soon as possible, providing the corresponding datasheets.

All robots must comply with standard "low voltage" regulations. As a result, **the on-board voltages must not exceed 48 V DC and 48 V peak to peak AC**.

Potential differences greater than 48 V may exist, but only within closed commercial devices (eg lasers, LCD backlights, etc.) and only if these devices have not been modified and comply with national and European regulations.

Batteries:

If the team chooses a battery power supply, we remind that only unmodified batteries can be used.

Teams must be able to play three games in a row. Note that this includes the time required to set up, during which the robot will be powered and awaiting the start.

Therefore, we strongly recommend that teams bring several sets of batteries and provide easy access to them in the robot for their replacement. Teams are reminded that it is essential to have a set of spare batteries, fully charged and available at all times.

Note on the use of Lithium-based batteries:

Lithium batteries are known for their lack of stability and can easily ignite when certain precautions are not taken.

This type of battery is therefore authorized under the following conditions:

- Suitable battery charger, which must be submitted for approval.
- Batteries kept in certified and unmodified fireproof bags.
- A system for detecting underloads is highly recommended.
- Exception in the case of the following batteries, authorized without the conditions listed above:
 - Lithium batteries for LEGO Mindstorm / laptop / mobile phone / power tools, not dismantled and used for the intended purpose of the manufacturer.
 - Lithium-Iron batteries (LiFePo4)

F.4. COMMUNICATION SIGNALS

To avoid interference between teams, it is recommended to encode the communication signals. We strongly recommend teams using infrared devices, to take into account the strong ambient light used during the encounters. In addition, this luminosity may vary in time and according to the emplacement of the playground in the hall.

We also remind teams that the organizing staff uses high-frequency radio devices and under no circumstances can they be held responsible for the malfunctions encountered by the robots.

CAUTION: Beyond the edges of the playing area, there may be elements that may interfere with color detection or communications signals such as:

- decor, lights and objects of the playing area
- people (referees, teams, etc.)
- electronic systems (microphones, cameras, etc.)

It is strictly forbidden to ask people to go away or move away objects/decor around the playing area !.

Wifi networks:

At multiple contest locations, the number of active Wifi devices can disturb the robots implementing this technology for communication. To answer this issue, it's recommended (but not mandatory) to use the 5 GHz frequency rather than the 2.4 GHz one.

F.5. OTHER DESIGN CONSTRAINTS

All systems (robots, beacons and accessories) must be presents on the table and are not allowed to communicate with systems outside the table during the matches.

Visibility : A rectangular, entire and non-deformable space of 100 x 70 mm per robot must be left free on one of the side faces. The teams will receive stickers printed by the organization (team number, sponsors etc), which they have to place on these open spaces.

It is also highly recommended (but not mandatory) to close empty spaces inside the robots to facilitate their detection by other robots.

Starting cord of autonomous robots: Robots must be equipped with a starting device easily accessible. This device has to be triggered by pulling the end of a cord at least 500 mm long. This cord must not stay attached to the robot after departure.

No other starting system (remote control, manual rocker switch, etc.) will be approved.

The start of one robot can launch the other robot.

Avoidance system : All teams are required to equip their robot(s) with a system for detecting opposing team's robots. This system is intended to prevent collisions between robots, during a match. This point will be systematically checked during homologation.

Warning: As most of the events are filmed, please adapt your avoidance systems so that it is not disturbed by autofocus cameras and filming.

Embedded/ on-board beacon support system : In order to facilitate robot identification in the field, robots must integrate an embedded beacon support system to identify the beacon of the opposing team. This support system must respect the following points:

- have a convex hull envelope over the entire height between the top of the robot and the beacon support, with a minimal size of a 70 mm diameter circle and with a maximum size of a 100 mm square;
- be solid and opaque (a material able to intercept the light at least in the infrared spectrum; and opaque for the visible spectrum too, except in case of a justification needed);
- have its upper surface flat, horizontal and positioned at a height of 430 mm above the playing area to allow the beacon of the opposing team to be placed in good conditions;
- the top of surface of the platform will be fully covered of VelcroTM (hook side);
- the vertical projection of the on-board beacon support must be located as centrally as possible and within a circle of 20 cm diameter around the center of the robot;
- the embedded beacon support can only accommodate sensor and telecommunication systems. The beacon support must be as hollow as possible. For teams using rotating devices, make sure that the portion of cylinder removed has a height of less than 2 cm (except for the minimum size of the envelope convex), and that it is not possible to see through the mat through this opening.
- the on-board beacon support must be stable, motionless regarding the robot structure and must be able to support a minimum weight of 400 g (the opposing team's beacon and the aruco tag), without flexing or compressing.

However, a team may choose not to equip their robot with an embedded beacon support system. In this case, if the opposing team requires a beacon support, and the use of it (either to detect the mast or to place a beacon), the team or the robot concerned may be revoked for that particular game.

G. BEACON TRACKING SYSTEM

G.1. GENERAL POINTS

Teams can develop a complete beacon localisation system; in this extent there are specific zones on the playing area where these beacons can be fixed.

At the opponent's request and only if justified, the robots could be equipped with a flagpole to fix an opponent's beacon above (see subsection F.5.).

All beacons (fixed beacons, embedded beacons and central tracking device) must remain in place on their supports throughout the duration of the match. All robot safety instructions equally apply to beacons. The beacons can only accommodate sensor and telecommunication systems.

Fixed beacons, center marker, beacons, embedded beacons and their respective supports are described below.

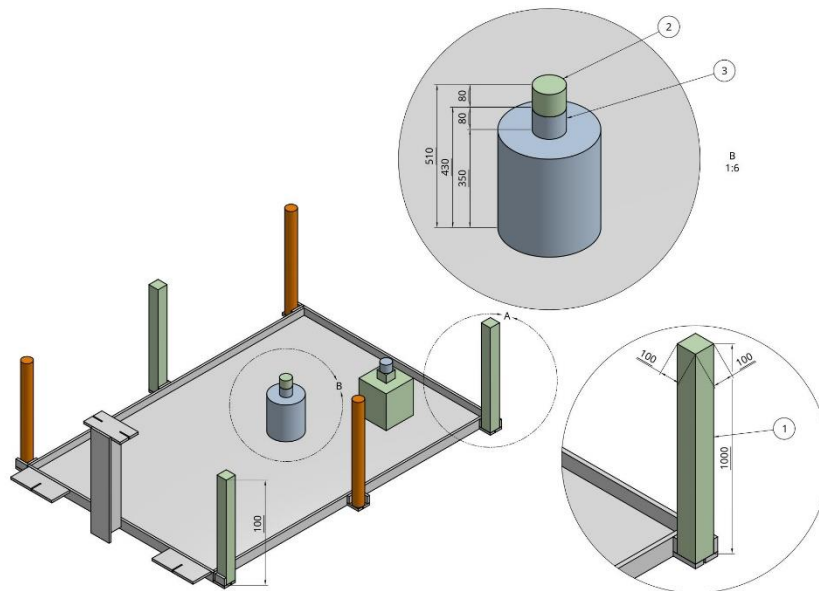


Figure 11 - Example of beacons positioning

Caption:

1. fixed beacons (maximum dimensions W x W x H: 100 x 100 x 1000 mm)
2. embedded beacon (maximum dimensions L x W x H: 100 x 100 x 80 mm)
3. mast of the support (cf. subsection F.5.)

G.2. EMBEDDED/ ON-BOARD BEACONS

A beacon can be placed on each of the adverse robots, in order to locate it. It must be placed on a dedicated mast, at an height of 430 mm above the playing area. The maximum size for an on-board location beacon is a

square based parallelepiped of 100 mm side and 80 mm height.

The upper side of the on-board beacon must be flat and covered with VelcroTM on the hook side and the under side of the beacons must be covered with velvet VelcroTM.

It is recommended that the color of the beacons be mainly white or very clear, so as to favor their detection on a dark background.

In the name of fair play, the elements used for this beacon must have real utility. Any "useless" or weighted beacon will be refused by the organization.

An embedded beacon must not exceed 300 g

G.3. FIXED BEACONS

Each team can place up to three fixed beacons on fixed supports. These fixed supports are allocated to the team upon demand and will be placed around the playing area.

Action-cams not useful for the course of the game are prohibited in fixed beacons.

G.3.a. DIMENSIONS

The fixed beacons shall be integrally contained in a square rectangular parallelepiped of 100 mm side and 1000 mm height.

A fixed beacon must not exceed 1.5 kg.

G.3.b. FIXATION

Given the potential height of the fixed beacons, they must have a solid fixing system.

The fixed beacon supports are located at the level of the playing area.

The horizontal plane of the fixed beacon support has a 10 mm wide groove. This groove must be used to secure the fixed beacons using an 8 mm diameter threaded rod and a wing nut.

The screw size is not counted into the dimensional constraints of the fixed beacons.

The absence of this fixing system will forbid the approval of the fixed beacons.

G.4. CENTRAL TRACKING DEVICE

In order to help identify the robots during the matches, a common platform located on the central symmetrical axis of the ground will be placed 1 m above the surface of the playing area (see plan). This platform can be used to place a robot tracking device with a top view.

The central tracking device should be placed on the area that corresponds to the color of the team. It must not exceed beyond a horizontal plane of 600 mm above the platform's upper surface. It must also not surpass a horizontal plane of 60mm below the platform's lower surface. On the sides, an offset of 60 mm is allowed: forward, at the edge not shared with the opponent and at the back. This offset allows: placing sensors above the ground, connecting the parts situated above and underneath the platform and setting up a slid fixing for the central tracking device. Its weight must be less than 2 kg.

It is forbidden to overpass on the opponent's area.

Please note that the central tracking device can be subject to vibrations due to the movements of robots on the playing area.

In appendix, the drawing of a mast supporting the platform are represented. However, only the dimensions and the position of the platform are contractual, the mast itself may be different from one meeting to another. Its design is left free to the organizer of the meeting.

Fixation :

The thickness of the fixing platform for the central tracking device is 22 mm.

The fixing device can surround the fastening platform from above, from below and from all three sides of the support so that it can be assembled and positioned quickly and without the risk of falling.

The mounting platform has a 10 mm wide groove from the center of the beacon support to the middle of the side. This groove must be used to secure the central tracking device using an 8 mm diameter threaded rod and a wing nut.

The absence of this fastening system will prevent the approval of the tracking system.

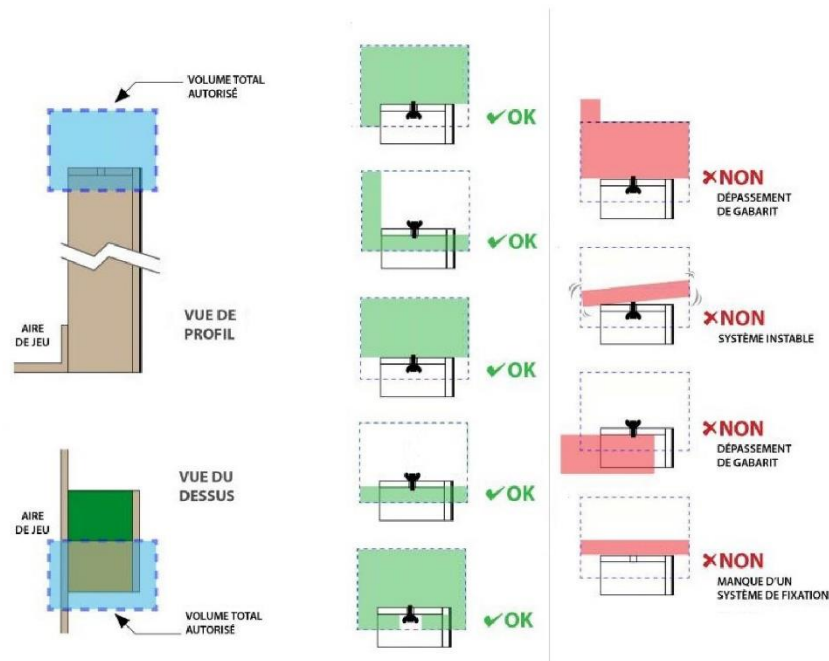


Figure 13 - How to install a central tracking device

G.5. CONNECTIONS

The fixed beacons and the central platform can be connected by a wired link. This connection must under no circumstances disturb the match. The installation of the whole system must be carried out before the match, during the preparation time. It should not disturb the opposing team's preparation.

During the preparation time, a temporary wired link may be pulled in between the robot and one or several beacons. This case scenario is accepted only if it doesn't cause any inconvenience for the opposing team.

G.6. ROBOT IDENTIFICATION

During each match, the robots will be assigned a marker (provided by the organizer) placed at the top of the beacon holder, or at the top of the on-board beacon (if present)(Figure 7b). This marker has two purposes:

- allow the public to identify the team to which a robot belongs;
- allow a vision system on the central mast to identify and locate each robot.

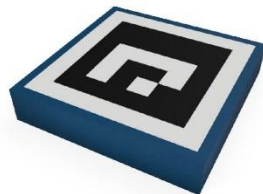
The marker is a square 10cm wide, 2cm thick ($\pm 2\text{mm}$) and with a mass less than or equal to 100g. Its upper side is covered with a vinyl with a 7cm ArUco 4x4 tag (centered). A 1cm wide white outline is drawn around the ArUco tag to facilitate its detection. A 0.5cm wide contour in the same colour as the team is drawn at the edge of the upper surface. The team's color also occupies the edge of the marker.(Figure 7a)

- Team blue robots will receive ArUco 4x4 tags between 1 and 5.
- Team green robots will receive ArUco 4x4 tags between 6 and 10.
- All robots will receive distinct markers. It will not be possible to choose them, nor to choose on which robot they will be placed.
- ArUco 4x4 tags from 11 to 50 are reserved for the playing area, but not all of them are necessarily used.

WARNING : to avoid visual disturbances, the teams are not allowed to use ArUco 4x4 tags from 0 to 50.

- ArUco 4x4 tags between 51 and 70 are reserved for team blue .
- ArUco 4x4 tags between 71 and 90 are reserved for team green .

However, other tag numbers over 90 and other dimensions than 4x4 are allowed.



(a) Robot identification marker (Tag n°1).



(b) Marker on the on-board beacon support.

H. MATCHES

Only two persons per team are permitted to go backstage and on stage to play the matches. During the homologation, if the teams justified the needs, the referees can allow the access to a third member of the team for the preparation.

To ensure that the contest runs smoothly, at least one member of the teams must be present on their booth with the robot(s) and ready to go 30 minutes before the start of each series and until that the game is played.

In case of a problem, it is tolerated by the organization to ask for a delay to go to the match but this delay can never exceed the end of the current series.

H.1. PREPARATION TIME

At the start of a match, the elements of the playing area and the playing area itself are installed as indicated in the diagrams in the appendix.

Upon arrival on the playing area, each team has a maximum of three minutes to proceed with the placement of the robots, the beacons and other equipment. At the end of the preparation time, the robot are not allowed to move until the start of the match.

A robot which is not ready at the end of this period exposes the team to a forfeit for the match.

Besides, the other team's robots will still play their own game on the playing area. The team will have to score points to be declared the winner.

When both teams have finished setting up, or when the preparation time is over, the referee asks the participants if they are ready. From this moment, teams are no longer allowed to touch their robots. It will be tolerated that the emergency stop button is pressed by the teams after the preparation time in order to preserve the batteries and actuators while waiting for the beginning of the match. No dispute can be made on the disposition of the elements of play after the beginning of the match.

H.2. THE MATCH

At the signal of the referee, each robot is switched on and it has 100 seconds to do these actions.

No one except the referee can touch the robots and the playing elements unless expressly indicated by the referees.

No elements taken out of the playing area can be put back on the table before the end of the match and the validation of the scores.

H.3. END OF MATCH

At the end of the 100 seconds, the robots must stop and turn off all their actuators. It is allowed to keep on any dynamic displays.

No one except the referee can touch the robots and the playing elements unless expressly indicated by the referees. The referees count the points; they give the result of the match, including the points to the teams. If they both agree, they validate the match sheet, can take back their robot(s) and join their booth. If the teams do not agree, they refer calmly to the referees. The robots remain in place until the dispute is resolved. Refereeing decisions are final.

In the event of a situation that is difficult to judge, the referees reserve the right to decide whether or not to replay the match.

The referees are allowed to pronounce the end of a game in advance, before the end of the regular time if both teams agree (if the robots are blocked for example).

H.4. COUNTING POINTS

Points summary

H.4.a. MAKING CAKE

- **1 point** per layer in the cake
- **4 extra points** if the cake respects the legendary recipe

H.4.b. PUT A CHERRY ON THE CAKE

- **3 points** for each cherry placed on a cake;

H.4.c. DROP CHERRIES IN THE BASKET

- **5 points** if the team places a basket during the preparation time;
- **1 point** per cherry in the basket
- **5 extra points** if the count is correct and not null

H.4.d. RETURN TO A ZONE

- **15 points** if all team's robot are in its own dropping area;

H.4.e. FUNNY ACTION

- **5 points** if the funny action is performed;

Warning! An element controlled by a robot, does not yield points. An element is considered to be controlled by a robot, if by moving the robot along its natural axis of movement the element is moved.

H.4.f. ESTIMATION POINT

The assessment is based on all the previous actions.

The estimation bonus is calculated as follows: **Bonus = 20 points - Delta**

- The score is the one scored by the team during the match on standard actions.
- The delta is the difference between the score made by the team during the match and the score estimated by the team. This one is always positive (Absolute value).
- The bonus is an integer value (rounded up).
- The bonus is added to the points of the team.
- A negative bonus is reduced to 0.
- The bonus for not "forfeit" is not include in the performance estimation.
- The penalties are not include in the performance estimation.

H.4.g. THE PENALTIES

Several actions during the match may result in penalties.

The following actions will result in a **warning** or a **point loss** on the final score if a warning has already been issued for the team (the warning rule is only valid during the series):

- loss of part or element of a robot on the playground: **loss of 20 points**.
- degradation of the table or a game element: **loss of 30 points**.
- non-functional avoidance system: **loss of 30 points**.
- false start: **loss of 50 points**.
- robot keeps moving when time runs out: **loss of 50 points**.
- excessive preparation time: **loss of 50 points**.
- robot change its start zone after 3 minute of preparation : **loss of 50 points**.
- unfair behavior: **loss of 50 to 100 points**.
- on arbitration decisions: **loss of 50 to 100 points**.
- on decisions of the organization: **loss of 50 to 100 points**.

The following actions will result in a **team forfeit**:

- no robot exit from its starting area.
- repeated excessive preparation time.
- dimensional limitations not respected.
- repeated false start.
- intentionally shoot at people in the vicinity.
- voluntary fixate, vibrate the table.
- intervention of a team member on the table, game elements or robots, after the preparation time (with the exception of the emergency stop button).
- intervention of a team member on the table, the game elements or the robots, during the match.
- the team is unable to play the game before the end of the series.
- following arbitration decisions.
- following decisions of the organization committee.

The following actions will result in **disqualification of the team from the competition**:

- voluntary deactivation of robot avoidance systems.
- design robots that are noticeably similar to robots from other teams (for example: rolling bases or identical actuators). If during the year you see a team building a robot similar to yours, report it to the organization as soon as possible.
- intentional degradation of robot belonging to other teams.
- following decisions of the organization committee.

Several penalties can be applied.

The score of a forfeited team or a negative score will be reduced to 0.

Only the referees are authorized to intervene on the table or the robots after the preparation time and during the match. In case of concern, ask the referee to intervene to avoid the forfeit.

In that cas, the appreciation of the penalties is upon the discretion of the referees. They cannot be disputed after the match.

Video refereeing is prohibited.

A general definition of anti-game: "If the goal is to harm without building, then it's anti-game". The assessment of the anti-game remains at the referee's judgement.

H.4.h. BONUS POINTS

1 bonus point is awarded to all teams that are not "forfeit". This bonus point is not included in the performance estimation.

I. THE CONTESTS

I.1. GENERAL INFORMATION

The Eurobot^{Open} meetings can be organized on three levels:

- regional: when they exist (e.g. in France, Eurobot^{Open} Junior), qualify a number of teams for the national meeting,
- national: it allows to qualify the teams for the European meeting,
- European: this last stage brings together, in the same friendly spirit, teams from different countries in Europe and elsewhere.

Each meeting has several successive steps:

- The static and dynamic approvals of all the robots;
- A qualification phase, with at least 3 series;
- An optional play-off phase;
- A final phase.

The organizers of each meeting can distribute awards, if they want, to the teams in order to reward an aspect of the participation of the team or its robot.

I.2. APPROVAL

I.2.a. STATIC APPROVAL

Before the start of the matches, robots are subject to the supervision of a referee who checks their compliance with the rules. Robots must be able to easily show all their mechanisms.

The ancillary systems (accessories, beacons, central tracking device, etc.) will also be subject to static control (size, mass, presence of mandatory elements, etc.).

I.2.b. DYNAMIC APPROVAL

The robots must, within 100 seconds, validate at least one action. The robots are put in a game situation but without the presence of an opposing team. Certain specific features provided for in the regulation can also be checked (timer, avoidance of opponents, etc.).

If the assembly consisting of the main robot and the secondary robot fulfills these conditions, it is declared approved. If one of the two robots is not approved, the other robot can play the match alone.

I.2.c. SIGNIFICANT TECHNICAL MODIFICATIONS AFTER THE APPROVAL

It is essential to inform the referees of any significant modifications (functional, structural, dimensional ...) brought to the robot(s) or any other element after approval. The referees will then check the modifications made and re-approve the robot if they deem it necessary. In the event of a breach, the team may be declared disqualified from the contest.

I.3. QUALIFICATION PHASE

During the qualification phase, the registered teams will have the possibility to play at least three games (often more, depending on the local organizers).

A ranking is established according to the accumulated points in order to select the qualified teams for the next phase.

The tied teams are tied by comparing their scores without taking into account their bonus points. Organizers may also use additional matches.

At the end of the qualifying phase, the first teams (according to the matches) are qualified for the next phase.

Number of participating teams	Minimum team selected
$B \leq 16$	4
$16 < N \leq 50$	8
$50 < N$	16

1.4. THE PLAY-OFF PHASE

An additional play-off phase may set up in the event that a meeting hosts two contests, the first of which qualifies for the second. For example:

- a regional meeting (A) and its national meeting (B)
- or a national meeting (A) and the European meeting (B)

The organizer may carry out the qualifying phase of both matches (A) and (B) either in parallel fashion or in a mixed fashion. In this case, a play-off phase may be organised in order to allow teams from the match (A) to qualify for the match (B), exempting them from having to catch up on all the matches from the qualifying phase of (B) in favour of this play-off phase.

The teams participating in this play-off phase are:

- the teams of the match (A), in a number corresponding to its qualification quota, and chosen in the order of ranking at the end of the final phase of (A) or through an award during the regional meeting.
- the teams of the match (B), in a number equivalent to the participants in this phase from (A), chosen from among the last teams normally qualified for the final phase of the match (B).

Example for a national meeting qualifying three teams for its European final whose European final phase has 16 teams. The teams participating in the play-off phase are:

- the first three teams of the national meeting at the end of the final phase of the national meeting;
- and the teams ranked 16th, 15th and 14th in the qualifying phase of the European meeting.

In this play-off phase, one match will be played for each participating team. The participating teams from the match (A) qualified through the ranking will play, in order of their ranking, with the teams from the match (B) with the lowest ranking. And the team(s) of (A) qualified for this play-off phase through an award will play against the top-ranked teams of (B) in a random order.

Each team that wins its play-off match will be included in the main draw of the final phase of the match (B). At the end of the play-off phase, the tree of the final phase of the match (B) presented in Figure 8 may be revised to more accurately represent the level of each team.

1.5. THE FINAL PHASE

At the end of the previous phase, the qualified teams form the table of the matches of the final phase. Depending to the meeting, only the teams composed of members less than 30 years of age will be able to enter the final stages.

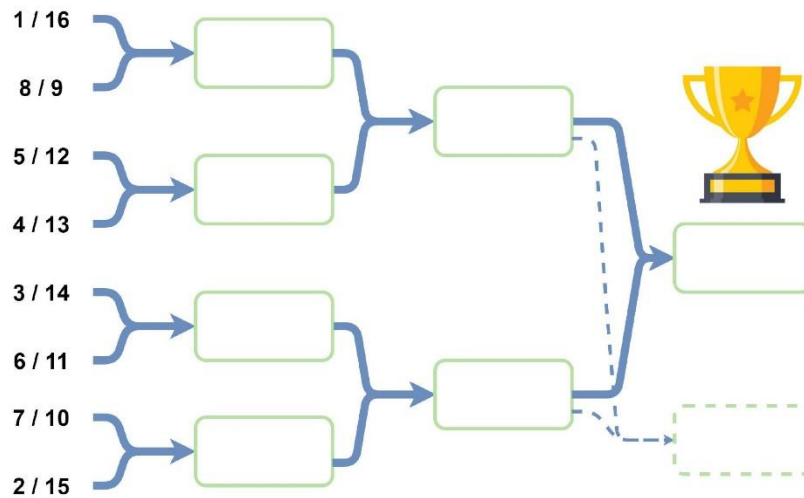


Figure 8: Tree of the final phase

The matches of the final phase are with knockout, unless otherwise organized on some meetings. In the event of double forfeit, double defeat or tie, the match is replayed immediately; if this second match is still a case of double forfeit, double defeat or equality, the winner is determined according to the points acquired at the end of the qualifying phase.

The final is played in two winning games. Be careful to provide batteries accordingly for autonomous robots.

1.6. QUALIFICATION FOR THE NATIONAL MEETING

When there are regional meetings, the number of teams qualified per regional meeting is proportional to the total number of teams registered at the national level.

The best teams in the ranking established at the end of the qualifying phase of each regional meeting, as well as at least one team chosen by the organizers from the special prizes (e.g. creativity, fair play, presentation, etc.).

1.7. QUALIFICATION FOR THE EUROPEAN MEETING

Each country participating in Eurobot^{Open} organizes a national meeting to determine the qualified teams for the international meeting.

The 3 finalist teams will qualify to take part in the European meeting.

For questions and comments, feel free to visit the Planète Sciences Forum.

<http://www.planete-sciences.org/forums/>

News and more information about Eurobot^{Open} and Eurobot^{Open} Junior are available on our website

www.eurobot.org

(It also contains links to your local organization)

The whole organization team of Eurobot^{Open} and Eurobot^{Open} Junior wishes you a lot of fun and success in the coming months, and looks forward to seeing you soon around our playing areas!

Robotic Regards,

The Eurobot^{Open} Eurobot^{Open} Junior organization committee.

J. APPENDIX

J.1. GENERAL DRAWINGS

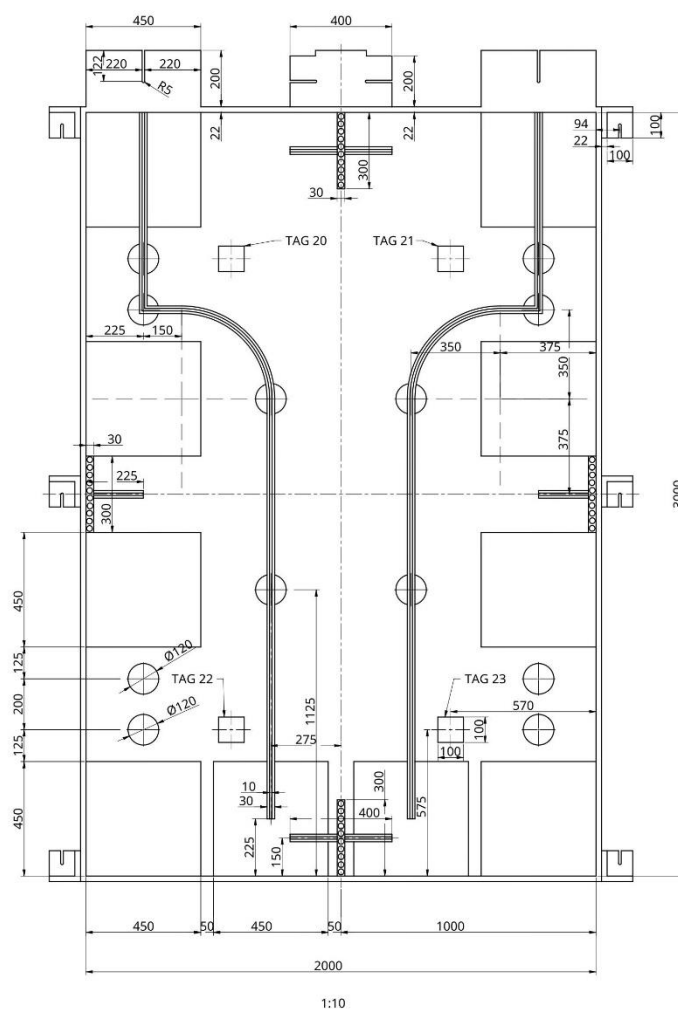


Figure 9: Top view of the playing area, symmetrical along both axes

J.1.a. CAKE STAGE

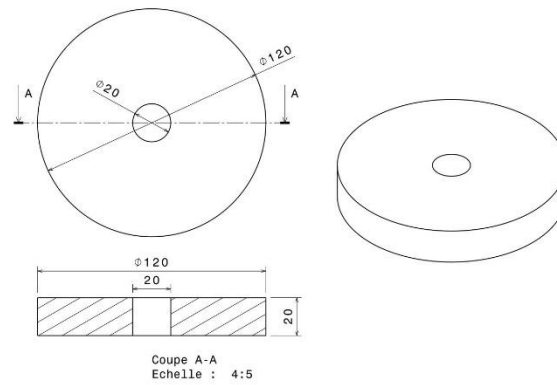


Figure 10: Cake layer

J.1.b. CHERRY SUPPORT

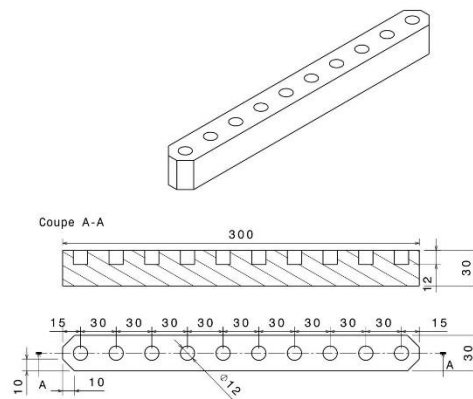


Figure 11: Cherry support

J.1.c. BASKET SUPPORT

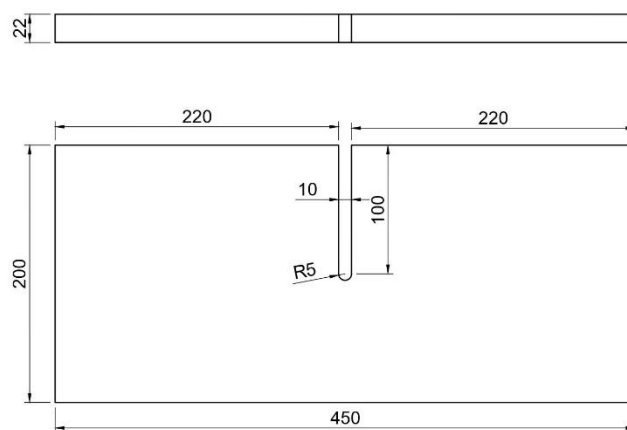


Figure 12: Basket support

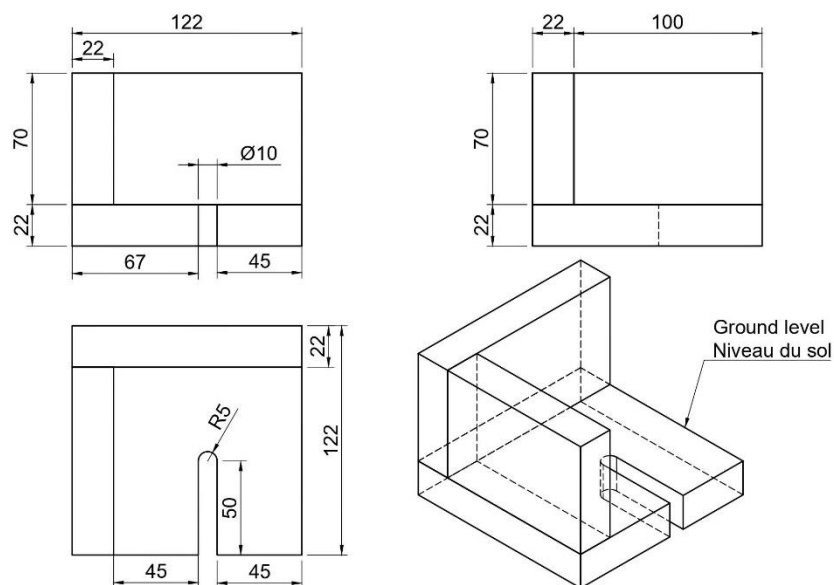


Figure 14: Left fixed beacon support

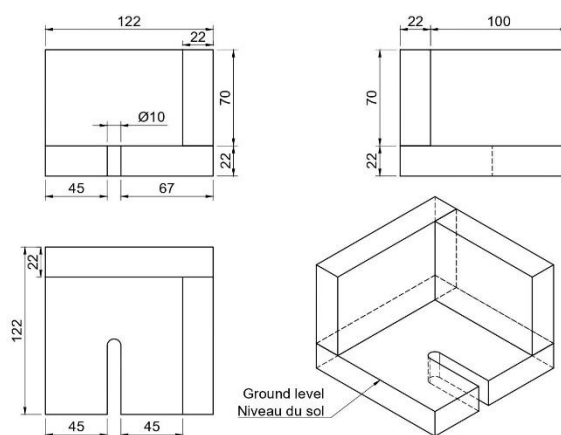


Figure 15: Right fixed beacon support

J.2. MATERIAL REFERENCES

Elements	Material or reference	Comments
Cake layer	Polyurethane foam board - Printed monomeric gripping vinyl	The cake layers can be composed of multiple layer of glued foam board
Game floor	Printed monomeric gripping vinyl	Ordering information will be provided by Planète Sciences
Showroom	Wood cover by printed monomeric gripping vinyl	
Beacon support	Wood cover by printed monomeric gripping vinyl	
Cherry	Red foam balls compatible with Nerf Rival TM	22.5mm in diameter on average
Distributors	Painted wood	
Robot identification marker	Polystyrene - Plastic - Vinyl - Velcro TM	

J.3. MANUFACTURING TOLERANCES

All dimensions are in millimeters (or mm). Manufacturing tolerances shall comply with the following rules, unless otherwise specified directly on the drawings.

Dimensions	General Tolerances
≤ 20	± 1.50
> 20 and ≤ 70	± 2.50
> 70 and ≤ 150	± 4.00
> 150	± 5.00

No objections regarding differences in dimensions will be taken into account.

The material's density can change from one country to another. It is highly recommended that the teams try different types of materials since the weight may differ significantly.

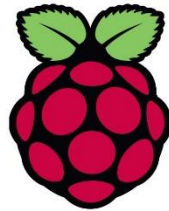
J.4. COLOUR REFERENCES

	Colors	References	CMYK
Team blue	Traffic Blue	RAL 5017 Mat	100% , 60% , 0% , 10%
Team green	Mint green	RAL 6029 Mat	100% , 5% , 90% , 30%
Cake layer 1	Telemagenta	RAL 4010 Mat	15% , 100% , 15% , 10%
Cake layer 2	Traffic yellow	RAL 1023 Mat	0% , 25% , 100% , 0%
Cake layer 3	Mahogany brown	RAL 8016 Mat	40% , 80% , 70% , 70%
Borders and non-colored elements	Pebble grey	RAL 7032 Mat	15% , 10% , 25% , 20%

RAL hues can vary from a printed soil mat to another.

4.2 Raspberry Pi 4 Datasheet

DATASHEET



Raspberry Pi 4 Model B

Release 1

June 2019

Copyright 2019 Raspberry Pi (Trading) Ltd. All rights reserved.



Raspberry Pi 4 Model B Datasheet
Copyright Raspberry Pi (Trading) Ltd. 2019

Table 1: Release History

Release	Date	Description
1	21/06/2019	First release

The latest release of this document can be found at <https://www.raspberrypi.org>



Raspberry Pi 4 Model B Datasheet
Copyright Raspberry Pi (Trading) Ltd. 2019

Contents

1	Introduction	5
2	Features	6
2.1	Hardware	6
2.2	Interfaces	6
2.3	Software	7
3	Mechanical Specification	7
4	Electrical Specification	7
4.1	Power Requirements	9
5	Peripherals	9
5.1	GPIO Interface	9
5.1.1	GPIO Pin Assignments	9
5.1.2	GPIO Alternate Functions	10
5.1.3	Display Parallel Interface (DPI)	11
5.1.4	SD/SDIO Interface	11
5.2	Camera and Display Interfaces	11
5.3	USB	11
5.4	HDMI	11
5.5	Audio and Composite (TV Out)	11
5.6	Temperature Range and Thermals	11
6	Availability	12
7	Support	12



Raspberry Pi 4 Model B Datasheet
Copyright Raspberry Pi (Trading) Ltd. 2019

List of Figures

1	Mechanical Dimensions	7
2	Digital IO Characteristics	8
3	GPIO Connector Pinout	9



Raspberry Pi 4 Model B Datasheet
Copyright Raspberry Pi (Trading) Ltd. 2019

List of Tables

1	Release History	1
2	Absolute Maximum Ratings	8
3	DC Characteristics	8
4	Digital I/O Pin AC Characteristics	8
5	Raspberry Pi 4 GPIO Alternate Functions	10



Raspberry Pi 4 Model B Datasheet
Copyright Raspberry Pi (Trading) Ltd. 2019

1 Introduction

The Raspberry Pi 4 Model B (Pi4B) is the first of a new generation of Raspberry Pi computers supporting more RAM and with significantly enhanced CPU, GPU and I/O performance; all within a similar form factor, power envelope and cost as the previous generation Raspberry Pi 3B+.

The Pi4B is available with either 1, 2 and 4 Gigabytes of LPDDR4 SDRAM.



2 Features

2.1 Hardware

- Quad core 64-bit ARM-Cortex A72 running at 1.5GHz
- 1, 2 and 4 Gigabyte LPDDR4 RAM options
- H.265 (HEVC) hardware decode (up to 4Kp60)
- H.264 hardware decode (up to 1080p60)
- VideoCore VI 3D Graphics
- Supports dual HDMI display output up to 4Kp60

2.2 Interfaces

- 802.11 b/g/n/ac Wireless LAN
- Bluetooth 5.0 with BLE
- 1x SD Card
- 2x micro-HDMI ports supporting dual displays up to 4Kp60 resolution
- 2x USB2 ports
- 2x USB3 ports
- 1x Gigabit Ethernet port (supports PoE with add-on PoE HAT)
- 1x Raspberry Pi camera port (2-lane MIPI CSI)
- 1x Raspberry Pi display port (2-lane MIPI DSI)
- 28x user GPIO supporting various interface options:
 - Up to 6x UART
 - Up to 6x I2C
 - Up to 5x SPI
 - 1x SDIO interface
 - 1x DPI (Parallel RGB Display)
 - 1x PCM
 - Up to 2x PWM channels
 - Up to 3x GPCLK outputs



Symbol	Parameter	Minimum	Maximum	Unit
VIN	5V Input Voltage	-0.5	6.0	V

Table 2: Absolute Maximum Ratings

Please note that VDD_IO is the GPIO bank voltage which is tied to the on-board 3.3V supply rail.

Symbol	Parameter	Conditions	Minimum	Typical	Maximum	Unit
V_{IL}	Input low voltage ^a	VDD_IO = 3.3V	-	-	TBD	V
V_{IH}	Input high voltage ^a	VDD_IO = 3.3V	TBD	-	-	V
I_{IL}	Input leakage current	TA = +85°C	-	-	TBD	μA
C_{IN}	Input capacitance	-	-	TBD	-	pF
V_{OL}	Output low voltage ^b	VDD_IO = 3.3V, IOL = -2mA	-	-	TBD	V
V_{OH}	Output high voltage ^b	VDD_IO = 3.3V, IOH = 2mA	TBD	-	-	V
I_{OL}	Output low current ^c	VDD_IO = 3.3V, VO = 0.4V	TBD	-	-	mA
I_{OH}	Output high current ^c	VDD_IO = 3.3V, VO = 2.3V	TBD	-	-	mA
R_{PU}	Pullup resistor	-	TBD	-	TBD	kΩ
R_{PD}	Pulldown resistor	-	TBD	-	TBD	kΩ

^a Hysteresis enabled

^b Default drive strength (8mA)

^c Maximum drive strength (16mA)

Table 3: DC Characteristics

Pin Name	Symbol	Parameter	Minimum	Typical	Maximum	Unit
Digital outputs	t_{rise}	10-90% rise time ^a	-	TBD	-	ns
Digital outputs	t_{fall}	90-10% fall time ^a	-	TBD	-	ns

^a Default drive strength, CL = 5pF, VDD_IO = 3.3V

Table 4: Digital I/O Pin AC Characteristics



Figure 2: Digital IO Characteristics



4.1 Power Requirements

The Pi4B requires a good quality USB-C power supply capable of delivering 5V at 3A. If attached downstream USB devices consume less than 500mA, a 5V, 2.5A supply may be used.

5 Peripherals

5.1 GPIO Interface

The Pi4B makes 28 BCM2711 GPIOs available via a standard Raspberry Pi 40-pin header. This header is backwards compatible with all previous Raspberry Pi boards with a 40-way header.

5.1.1 GPIO Pin Assignments

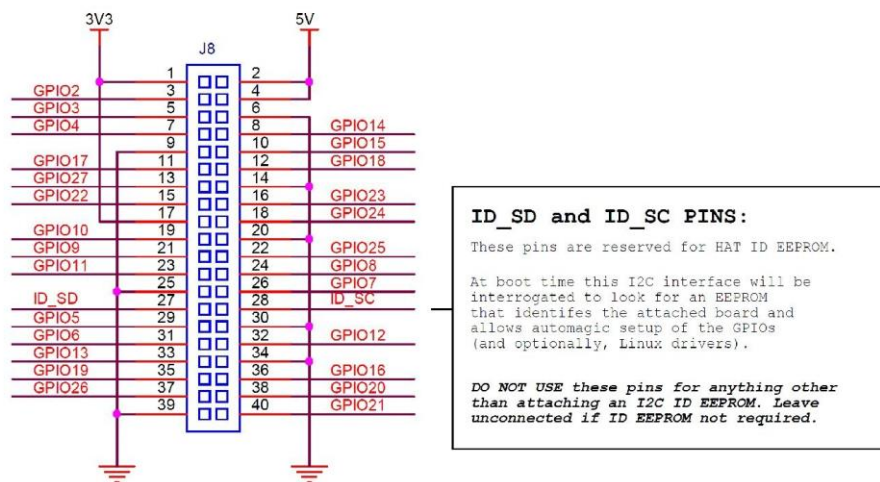


Figure 3: GPIO Connector Pinout

As well as being able to be used as straightforward software controlled input and output (with programmable pulls), GPIO pins can be switched (multiplexed) into various other modes backed by dedicated peripheral blocks such as I2C, UART and SPI.

In addition to the standard peripheral options found on legacy Pis, extra I2C, UART and SPI peripherals have been added to the BCM2711 chip and are available as further mux options on the Pi4. This gives users much more flexibility when attaching add-on hardware as compared to older models.



Raspberry Pi 4 Model B Datasheet
Copyright Raspberry Pi (Trading) Ltd. 2019

5.1.2 GPIO Alternate Functions

GPIO	Default Pull	ALT0	ALT1	ALT2	ALT3	ALT4	ALT5
0	High	SDA0	SA5	PCLK	SPI3_CE0_N	TXD2	SDA6
1	High	SCL0	SA4	DE	SPI3_MISO	RXD2	SCL6
2	High	SDA1	SA3	LCD_VSYNC	SPI3_MOSI	CTS2	SDA3
3	High	SCL1	SA2	LCD_HSYNC	SPI3_SCLK	RTS2	SCL3
4	High	GPCLK0	SA1	DPLD0	SPI4_CE0_N	TXD3	SDA3
5	High	GPCLK1	SA0	DPLD1	SPI4_MISO	RXD3	SCL3
6	High	GPCLK2	SOE_N	DPLD2	SPI4_MOSI	CTS3	SDA4
7	High	SPI0_CE1_N	SWE_N	DPLD3	SPI4_SCLK	RTS3	SCL4
8	High	SPI0_CE0_N	SD0	DPLD4	-	TXD4	SDA4
9	Low	SPI0_MISO	SD1	DPLD5	-	RXD4	SCL4
10	Low	SPI0_MOSI	SD2	DPLD6	-	CTS4	SDA5
11	Low	SPI0_SCLK	SD3	DPLD7	-	RTS4	SCL5
12	Low	PWM0	SD4	DPLD8	SPI5_CE0_N	TXD5	SDA5
13	Low	PWM1	SD5	DPLD9	SPI5_MISO	RXD5	SCL5
14	Low	TXD0	SD6	DPLD10	SPI5_MOSI	CTS5	TXD1
15	Low	RXD0	SD7	DPLD11	SPI5_SCLK	RTS5	RXD1
16	Low	FL0	SD8	DPLD12	CTS0	SPI1_CE2_N	CTS1
17	Low	FL1	SD9	DPLD13	RTS0	SPI1_CE1_N	RTS1
18	Low	PCM_CLK	SD10	DPLD14	SPI6_CE0_N	SPI1_CE0_N	PWM0
19	Low	PCM_FS	SD11	DPLD15	SPI6_MISO	SPI1_MISO	PWM1
20	Low	PCM_DIN	SD12	DPLD16	SPI6_MOSI	SPI1_MOSI	GPCLK0
21	Low	PCM_DOUT	SD13	DPLD17	SPI6_SCLK	SPI1_SCLK	GPCLK1
22	Low	SD0_CLK	SD14	DPLD18	SD1_CLK	ARM_TRST	SDA6
23	Low	SD0_CMD	SD15	DPLD19	SD1_CMD	ARM_RTCK	SCL6
24	Low	SD0_DAT0	SD16	DPLD20	SD1_DAT0	ARM_TDO	SPI3_CE1_N
25	Low	SD0_DAT1	SD17	DPLD21	SD1_DAT1	ARM_TCK	SPI4_CE1_N
26	Low	SD0_DAT2	TE0	DPLD22	SD1_DAT2	ARM_TDI	SPI5_CE1_N
27	Low	SD0_DAT3	TE1	DPLD23	SD1_DAT3	ARM_TMS	SPI6_CE1_N

Table 5: Raspberry Pi 4 GPIO Alternate Functions

Table 5 details the default pin pull state and available alternate GPIO functions. Most of these alternate peripheral functions are described in detail in the BCM2711 Peripherals Specification document which can be downloaded from the hardware documentation section of the website.



5.1.3 Display Parallel Interface (DPI)

A standard parallel RGB (DPI) interface is available the GPIOs. This up-to-24-bit parallel interface can support a secondary display.

5.1.4 SD/SDIO Interface

The Pi4B has a dedicated SD card socket which supports 1.8V, DDR50 mode (at a peak bandwidth of 50 Megabytes / sec). In addition, a legacy SDIO interface is available on the GPIO pins.

5.2 Camera and Display Interfaces

The Pi4B has 1x Raspberry Pi 2-lane MIPI CSI Camera and 1x Raspberry Pi 2-lane MIPI DSI Display connector. These connectors are backwards compatible with legacy Raspberry Pi boards, and support all of the available Raspberry Pi camera and display peripherals.

5.3 USB

The Pi4B has 2x USB2 and 2x USB3 type-A sockets. Downstream USB current is limited to approximately 1.1A in aggregate over the four sockets.

5.4 HDMI

The Pi4B has 2x micro-HDMI ports, both of which support CEC and HDMI 2.0 with resolutions up to 4Kp60.

5.5 Audio and Composite (TV Out)

The Pi4B supports near-CD-quality analogue audio output and composite TV-output via a 4-ring TRS 'A/V' jack.

The analog audio output can drive 32 Ohm headphones directly.

5.6 Temperature Range and Thermals

The recommended ambient operating temperature range is 0 to 50 degrees Celcius.

To reduce thermal output when idling or under light load, the Pi4B reduces the CPU clock speed and voltage. During heavier load the speed and voltage (and hence thermal output) are increased. The internal governor will throttle back both the CPU speed and voltage to make sure the CPU temperature never exceeds 85 degrees C.

The Pi4B will operate perfectly well without any extra cooling and is designed for sprint performance - expecting a light use case on average and ramping up the CPU speed when needed (e.g. when loading a webpage). If a user wishes to load the system continually or operate it at a high temperature at full performance, further cooling may be needed.



Raspberry Pi 4 Model B Datasheet
Copyright Raspberry Pi (Trading) Ltd. 2019

6 Availability

Raspberry Pi guarantee availability Pi4B until at least January 2026.

7 Support

For support please see the hardware documentation section of the Raspberry Pi website and post questions to the Raspberry Pi forum.

4.3 Elgato Facecam

Especificaciones técnicas

Modelo: elgato 6470494 Facecam Full HD Webleva

Resoluciones admitidas (sin comprimir): 1080p60, 1080p30, 720p60, 720p30, 540p60, 540p30



Óptica: Lente Elgato Prime (totalmente de vidrio)

Rango de enfoque: 30 - 120 cm | 11.8 - 47.2 in

Apertura: f / 2.4

Longitud focal: 24 mm

Campo de View: 82 ° (diagonal)

Sensor: Sony® STARVIS™ CMOS

Conexión: USB 3.0 (o mejor), Tipo C

Dimensiones: An 79 x Al 48 x Pr 58 mm

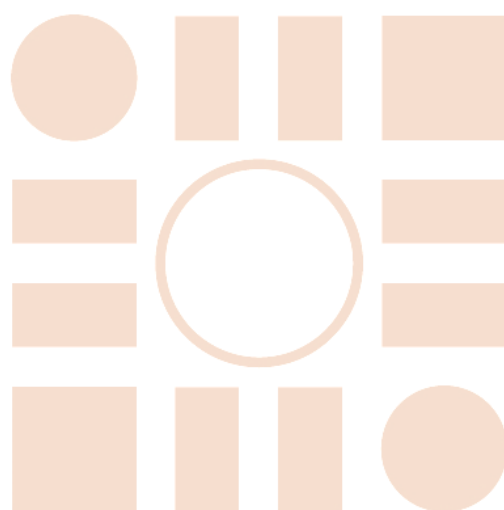
3.1 x 1.9 x 2.3 in (sin soporte)

Peso: 96 g | 0.21 lb (sin obturador de privacidad ni soporte)

5. Bibliografía

- [1] Raspberry Pi. (2012). Introducing the Raspberry Pi Cameras (1st ed.) [Online]. Available: https://www.raspberrypi.com/documentation/computers/camera_software.html
- [2] W. Ondara. (2022, August 9). How to Install ROS 2 Humble on Ubuntu 22.04. [Online]. Available: <https://linuxopsys.com/topics/install-ros-2-humble-on-ubuntu>
- [3] Ubuntu MATE Team. (2014). Ubuntu Mate home [Online]. Available: <https://ubuntu-mate.org/>
- [4] Eurobot Association (2004). International Students Robotic Contest: "Home" [Online]. Available: <https://www.eurobot.org/>
- [5] OpenCV. (2023, September 12). Detection of ArUco Markers [Online]. Available: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html
- [6] ROS. (2021). Robot Operating System (ROS) [Online]. Available: <https://www.ros.org/>
- [7] Clearview. (2013). Parámetros Ópticos para Aplicaciones de Visión Artificial, parte 1 [Online]. Available: <https://www.clearview-imaging.com/es/blog/par%C3%A1metros-%C3%B3pticos-para-aplicaciones-de-visi%C3%B3n-artificial-parte-1-de-2>
- [8] A. Babinec et al. "Visual Localization of Mobile Robot Using Artificial Markers", ScienceDirect 1, 96, pp 1-9. 2014.
- [9] OpenAI. "ChatGPT: A Large-Scale Generative Model for Language Understanding and Generation." 2021. [Online]. Available: <https://www.openai.com/chatgpt>
- [10] Edouard Renard. "ROS2 For Beginners (ROS Foxy, Humble - 2023)." Udemy, 2023. [Online]. Available: https://www.udemy.com/course/ros2-for-beginners/?couponCode=SEP_23
- [11] NASA. (2023). NASA - National Aeronautics and Space Administration. [Online]. Available: <https://www.nasa.gov/>

Universidad de Alcalá
Escuela Politécnica Superior



ESCUELA POLITECNICA
SUPERIOR



Universidad
de Alcalá